

Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software

Juan José Medicis Zambrano

Proyecto de grado entregado para obtener el título de
Magister en Ingeniería de Software

Dirigida por
MsC Juan Camilo Parra Martinez

Pontificia Universidad Javeriana Cali
Facultad de Ingeniería y Ciencias
Maestría en Ingeniería de Software
Santiago de Cali
octubre de 2026

Santiago de Cali, octubre de 2026.

Señores
Pontificia Universidad Javeriana Cali.
Ph.D. Luisa Rincón
Directora Maestría en Ingeniería de Software.
Cali.

Cordial Saludo.

Por medio de la presente hago constar que en mi calidad de director de trabajo de grado he revisado el proyecto titulado "Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software" realizado por el estudiante de Magister en Ingeniería de Software Juan José Medicis Zambrano (cod: [CÓDIGO ESTUDIANTE]), el cual se encuentra terminado y considero que cumple con los requisitos para ser sustentado.
Atentamente,

MsC Juan Camilo Parra Martinez

Santiago de Cali, octubre de 2026.

Señores
Pontificia Universidad Javeriana Cali
Ph.D. Luisa Rincón
Directora Maestría en Ingeniería de Software
Cali.

Cordial Saludo.

Me permito presentar a su consideración el proyecto de grado titulado "Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software" con el fin de cumplir con los requisitos exigidos por la Universidad y para que sea sometido a revisión del jurado y cumpla su aprobación, para conseguir posteriormente el título de Magister en Ingeniería de Software.

Atentamente,

Juan José Medicis Zambrano
Código: [CÓDIGO ESTUDIANTE]

Ficha Resumen
Trabajo de Grado Maestría en Ingeniería de Software

TÍTULO: Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software

1. Énfasis: Ingeniería de Software
2. Área de trabajo: Arquitectura de software, análisis estático de código y gobernanza técnica
3. Tipo de proyecto (Aplicado, Innovación, Investigación): Investigación aplicada
4. Estudiante: Juan José Medicis Zambrano
5. Correo electrónico: [COMPLETAR: correo institucional del estudiante]
6. Dirección y teléfono: [COMPLETAR: dirección y teléfono del estudiante]
7. Director: MsC Juan Camilo Parra Martinez
8. Vinculación del director: [COMPLETAR: Planta / Cátedra / Externo]
9. Correo electrónico del director: [COMPLETAR: correo institucional del director]
10. Co-Director (Si aplica): No aplica
11. Grupo o empresa que lo avala (Si aplica): No aplica
12. Otros grupos o empresas: No aplica
13. Palabras clave (al menos 5): deriva arquitectónica; análisis estático; grafos de dependencia; *Fitness Functions*; gobernanza arquitectónica; conformidad estructural
14. ODS que aplica al proyecto (Agenda 2030): ODS 9 (Industria, innovación e infraestructura)
15. Fecha de inicio: Mayo de 2026
16. Resumen: Este proyecto aborda la deriva arquitectónica, entendida como la divergencia progresiva entre la arquitectura prevista de un sistema de software y su implementación real, un fenómeno que incrementa el costo de mantenimiento y erosiona la calidad del producto. Su importancia radica en que las pequeñas y medianas empresas de software, que concentran buena parte del desarrollo en la industria, rara vez disponen de documentación arquitectónica actualizada ni de los recursos para adoptar herramientas de verificación sofisticadas, por lo que acumulan deuda técnica de manera silenciosa. Ante esta problemática, se propone diseñar,

construir y evaluar un mecanismo automatizado que detecta derivas arquitectónicas combinando el análisis estático de grafos de dependencia con la ejecución de *Fitness Functions*, definidas como verificaciones objetivas que cuantifican qué tan alineada está la implementación con las reglas estructurales declaradas. El objetivo general consiste en desarrollar y validar experimentalmente dicho mecanismo para clasificar el estado de conformidad estructural de un proyecto sin depender de documentación arquitectónica externa, mientras que los objetivos específicos abarcan la formulación de un catálogo de *Fitness Functions*, la implementación de un componente de análisis estático, la construcción de un motor de validación basado en una Tasa de Conformidad y la ejecución de un experimento controlado sobre un caso de estudio. La evaluación se realizó sobre cinco escenarios de un mismo sistema: una implementación de referencia y cuatro versiones alteradas, tres de ellas con una única deriva inducida para medir la sensibilidad de cada regla por separado. El mecanismo detectó la totalidad de las derivas y no produjo ningún hallazgo sobre la implementación de referencia, superando el criterio de éxito establecido; como complemento, detectó también las 31 derivas inyectadas en siete sistemas Java reales —cuatro bibliotecas y tres aplicaciones en capas—, sin hallazgos espurios, y encontró deriva real en la evolución publicada de dos bibliotecas. Los escenarios aislados revelaron, además, que la Tasa de Conformidad diluye las violaciones localizadas por ser una proporción no ponderada, lo que motivó incorporar la severidad del peor hallazgo como criterio complementario de clasificación. Entre las lecciones aprendidas destacan la viabilidad del enfoque con recursos limitados, la importancia de aislar las variables en el diseño experimental —fue esa decisión la que reveló la limitación de la métrica— y la necesidad de fijar los umbrales con un procedimiento estadístico y no a juicio, y de contrastar la evidencia propia con una herramienta independiente.

Resumen

La deriva arquitectónica es la divergencia progresiva entre la arquitectura prevista de un sistema y su implementación real, y constituye una de las principales fuentes de deuda técnica en las pequeñas y medianas empresas de software, que rara vez disponen de documentación arquitectónica actualizada o de recursos para adoptar plataformas de verificación sofisticadas. Este trabajo propone un mecanismo automatizado que detecta dicha deriva comparando dos representaciones de un mismo sistema: la arquitectura prevista, que el equipo declara en un catálogo mínimo de capas y dependencias admitidas, y la implementada, que se extrae del código compilado como un grafo de dependencias entre paquetes con reglas explícitas. La comparación se realiza mediante tres *Fitness Functions* seleccionadas a partir de la literatura —integridad de capas, ausencia de ciclos y concentración de dependencias salientes—, cuyos resultados se agregan en una Tasa de Conformidad que clasifica el estado del sistema como aceptable, en observación o no conforme y que acompaña cada hallazgo de la evidencia que lo origina y de una guía para interpretarlo. La propuesta se evaluó mediante un experimento controlado sobre cinco escenarios de un caso de estudio —una implementación de referencia y cuatro versiones alteradas, tres de ellas con una única deriva inducida— y se complementó inyectando derivas conocidas en siete sistemas Java reales —cuatro bibliotecas de hasta dos mil clases y tres aplicaciones en capas con arquitectura documentada—, siguiendo treinta y cinco versiones publicadas de tres de ellos y contrastando la extracción con una herramienta independiente. El mecanismo detectó la totalidad de las derivas inducidas e inyectadas sin producir hallazgos espurios, encontró deriva real en la evolución de dos bibliotecas y extrajo el mismo grafo que la herramienta de contraste salvo diferencias explicadas por sus reglas de extracción. La evaluación reveló, además, que la Tasa de Conformidad, al ser una proporción no ponderada, diluye las violaciones aisladas y puede clasificar como aceptable un sistema con una desviación real; ese hallazgo, que se reprodujo en los sistemas reales, motivó un refinamiento que incorpora la gravedad del peor hallazgo junto a la extensión que mide la métrica, con el cual los cinco escenarios pasaron a clasificarse conforme a lo esperado; la formulación por reglas que preveía el anteproyecto, calculada en paralelo, no se diluye pero satura y no registra la tendencia. El umbral de concentración se fijó con un procedimiento de percentiles, frente al que la clasificación resultó robusta. La hipótesis de trabajo quedó confirmada en la detección y en la diferenciación de estados, y solo parcialmente en la utilidad de la evidencia para decidir, que exige una evaluación con equipos reales. Los resultados evidencian la viabilidad del enfoque para la gobernanza continua sin depender de documentación arquitectónica externa, y precisan los límites que toda métrica de conformidad agregada arrastra.

Palabras Clave: deriva arquitectónica, análisis estático, grafos de dependencia, *Fitness Functions*, gobernanza arquitectónica.

Abstract

Architectural drift is the progressive divergence between a system's intended architecture and its actual implementation, and it represents one of the main sources of technical debt in small and medium-sized software enterprises, which seldom keep architectural documentation up to date or can afford sophisticated verification platforms. This work proposes an automated mechanism that detects such drift by comparing two representations of the same system: the intended architecture, which the team declares in a minimal catalog of layers and allowed dependencies, and the implemented one, extracted from compiled code as a package dependency graph under explicit rules. The comparison relies on three *Fitness Functions* selected from the literature —layer integrity, absence of cycles, and outgoing dependency concentration—, whose results are aggregated into a Conformance Rate that classifies the system as acceptable, under observation, or non-conformant, attaching to every finding the evidence that originates it and guidance to interpret it. The proposal was evaluated through a controlled experiment over five scenarios of a case study —a reference implementation and four altered versions, three of which introduce a single drift each— and complemented by injecting known drifts into seven real Java systems —four libraries of up to two thousand classes and three layered applications with documented architectures—, by following thirty-five published versions of three of them, and by checking the extraction against an independent tool. The mechanism detected every induced and injected drift without spurious findings, found genuine drift in the evolution of two libraries, and extracted the same graph as the reference tool except for differences explained by its extraction rules. The evaluation further revealed that the Conformance Rate, being an unweighted proportion, dilutes isolated violations and may classify as acceptable a system that contains a genuine deviation; this finding, which reappeared in the real systems, motivated a refinement that combines the severity of the worst finding with the extent that the metric measures, under which all five scenarios were classified as expected; the rule-based formulation foreseen in the proposal, computed in parallel, does not dilute but saturates and shows no trend. The concentration threshold was set through a percentile procedure, to which the classification proved robust. The working hypothesis was confirmed regarding detection and the distinction between states, and only partially regarding the usefulness of the evidence for decision-making, which requires an evaluation with real teams. The results evidence the feasibility of the approach for continuous governance without relying on external architectural documentation, and delimit the constraints that any aggregate conformance metric carries.

Keywords: architectural drift, static analysis, dependency graphs, *Fitness Functions*, architectural governance.

Índice general

1. Introducción **1**
- 1.1. Antecedentes y contexto . . . 2
- 1.2. Definición del problema . . . 3
  - 1.2.1. Planteamiento del problema . . . 3
  - 1.2.2. Sistematización del problema . . . 3
  - 1.2.3. Hipótesis de trabajo . . . 3
- 1.3. Objetivos del proyecto . . . 4
  - 1.3.1. Objetivo general . . . 4
  - 1.3.2. Objetivos específicos . . . 4
- 1.4. Delimitaciones y alcances . . . 4
- 1.5. Justificación del trabajo de grado . . . 5
- 1.6. Metodología de la investigación . . . 5
  - 1.6.1. Ejecución del cronograma . . . 7
- 1.7. Resultados obtenidos . . . 7
- 1.8. Estructura del documento . . . 8

2. Marco de referencia **11**
- 2.1. Marco Teórico . . . 11
  - 2.1.1. Arquitectura de software y atributos de calidad . . . 11
  - 2.1.2. Deriva y erosión arquitectónica . . . 11
  - 2.1.3. Deuda técnica arquitectónica y *smells* . . . 12
  - 2.1.4. *Fitness Functions* . . . 13
  - 2.1.5. Análisis estático y grafos de dependencia . . . 13
  - 2.1.6. Métricas de acoplamiento . . . 14
  - 2.1.7. Conformidad arquitectónica y gobernanza continua . . . 15
- 2.2. Estado del Arte . . . 15
- 2.3. Resumen del capítulo . . . 18

3. Desarrollo del Proyecto **19**
- Parte I. El proceso de detección . . . 19
- 3.1. Visión general del proceso . . . 19
- 3.2. Origen y selección de las *Fitness Functions* . . . 21
- 3.3. Declaración de la arquitectura prevista . . . 23
- 3.4. Extracción del grafo de dependencias . . . 26
- 3.5. Evaluación: cómo se obtiene la deriva . . . 28
- 3.6. Unidades de medida y agregación . . . 29
- 3.7. Interpretación de los resultados . . . 31

Parte II. La instrumentación del proceso . . . 32
3.8. Requisitos . . . 32
3.9. Arquitectura del prototipo . . . 34
3.10. Implementación . . . 34
3.11. Representación gráfica del grafo . . . 36
3.12. Interfaz gráfica . . . 37
3.13. Verificación del prototipo . . . 37
3.14. Escenarios de evaluación . . . 39
3.15. Usos y escenarios de aplicación . . . 41
3.16. Trazabilidad entre requisitos e implementación . . . 42
3.17. Resumen del capítulo . . . 43

4. Evaluación **45**
4.1. Diseño de la evaluación . . . 45
4.1.1. Matriz de validación . . . 46
4.1.2. Criterios de éxito y procedimiento . . . 46
4.2. Resultados de la evaluación . . . 47
4.2.1. Contraste con la matriz de validación . . . 47
4.2.2. Sensibilidad por dimensión . . . 48
4.2.3. Trazabilidad del cálculo . . . 48
4.2.4. Interpretación de los resultados por escenario . . . 49
4.3. Un hallazgo sobre la métrica: la dilución de las violaciones localizadas . . . 50
4.3.1. La formulación del anteproyecto: la CR por regla . . . 51
4.4. Refinamiento del artefacto . . . 51
4.5. Contraste con los indicadores de la metodología . . . 52
4.5.1. Criterios de cierre . . . 53
4.6. Aplicación a un sistema externo . . . 54
4.7. Validación de aplicabilidad en entorno controlado sobre sistemas reales . . . 55
4.8. Deriva real a lo largo de la evolución . . . 60
4.9. Calibración del umbral de concentración . . . 61
4.10. Contraste con un oráculo independiente . . . 63
4.11. Aporte frente a la verificación con *ArchUnit* . . . 64
4.12. Contraste de la hipótesis de trabajo . . . 65
4.13. Amenazas a la validez . . . 66
4.14. Resumen del capítulo . . . 68

5. Conclusiones **69**
5.1. Conclusiones . . . 69
5.2. Contribuciones del trabajo . . . 71
5.3. Aplicación en la industria . . . 72
5.4. Trabajos futuros . . . 73

5.5. Lecciones aprendidas 74

6. Riesgos éticos **77**
6.1. Riesgos éticos 77
6.2. Principios orientadores 77
6.3. Riesgos identificados y tratamiento aplicado 77
6.4. Riesgos residuales 79

Bibliografía **81**

A. Reproducción del experimento **85**
A.1. Entorno y dependencias 85
A.2. Estructura del repositorio 85
A.3. Pasos para reproducir el experimento 86
A.4. Figuras de grafos y sistema externo 87
A.5. Evidencia generada 87
A.6. Determinismo de los resultados 88

B. Exploración del asistente de catálogo **89**
B.1. Pregunta y procedimiento 89
B.2. Resultados 90
B.3. Alcance de la exploración 91

C. Registro de decisiones **93**
C.1. Desviaciones respecto del anteproyecto 93
C.2. Índice del registro 94

Índice de figuras

3.1. Proceso de detección de deriva arquitectónica. El catálogo interviene en la extracción —delimita qué paquetes forman el sistema— y en la asignación de capas. Las flechas discontinuas son las dos salidas de una decisión: corregir el código, cuando el hallazgo es deriva, o actualizar la intención declarada, cuando la arquitectura cambió de forma deliberada. 21
3.2. Módulos del prototipo, el paso del proceso que instrumenta cada uno y el sentido de sus dependencias. Se omiten, por legibilidad, las aristas que saltan niveles; ningún módulo depende de otro situado por encima de él, de modo que el grafo interno es acíclico. 35
3.3. Interfaz gráfica del prototipo analizando el escenario combinado (C4). La barra lateral guía los tres pasos —proyecto, catálogo y criterio de clasificación—; el panel principal muestra los indicadores, la conformidad por regla y el grafo, en el que se ha seleccionado el paquete `negocio`: lo que no le concierne se atenúa y la ficha de la derecha detalla su capa, sus acoplamientos, sus vecinos y los hallazgos en que interviene. 38
3.4. Grafo de dependencias de la implementación de referencia (C0), dibujado por el propio mecanismo: 4 paquetes y 5 dependencias, sin violaciones. Cada banda corresponde a una capa declarada; el dominio compartido, que no pertenece a ninguna, ocupa la columna lateral. 39
3.5. Grafo del escenario C2, dibujado por el propio mecanismo. El ciclo queda contenido en la capa de negocio: ambos paquetes comparten fila y su dependencia mutua se traza como dos arcos, uno por encima y otro por debajo. Ninguna arista remonta la estratificación. 40
3.6. Grafo del escenario combinado (C4), dibujado por el propio mecanismo: 5 paquetes y 11 dependencias. Las dos dependencias que la persistencia dirige hacia capas superiores violan la estratificación y rodean los nodos por fuera; el ciclo que cierran se marca en los tres paquetes de capa, y el paquete que concentra dependencias salientes aparece en ámbar en la columna lateral. 41

4.1. Grafo de dependencias de *ArchUnit* 1.3.0 con las exenciones declaradas, dibujado por el propio mecanismo a partir del archivo `.jar`. Las bandas siguen la estratificación hipotética `library` → `lang` → `core` → `base`; el núcleo de anotaciones exento ocupa la columna lateral y sus dependencias se dibujan atenuadas. 56

Índice de tablas

2.1. Comparación de los trabajos relacionados con la propuesta. Leyenda: ✓ lo cumple, ∼ lo cumple parcialmente, × no lo cumple. 17

3.1. Entidades del proceso de detección. 20
3.2. Candidatas a *Fitness Function* y decisión de selección. 22
3.3. Catálogo de *Fitness Functions* implementadas. 24
3.4. Formas de referencia entre clases y su efecto en el grafo, verificadas por la batería de pruebas sobre código compilado. 28
3.5. Evaluación del escenario C4, paso a paso. 29
3.6. Unidades en que se expresa la deriva. 30
3.7. Guía de interpretación del diagnóstico. 33
3.8. Trazabilidad entre requisitos, implementación y evidencia de cumplimiento. 43

4.1. Matriz de validación: comportamiento esperado de cada *Fitness Function* por escenario. 46
4.2. Resultados agregados del mecanismo sobre los cinco escenarios. 47
4.3. Contraste entre el comportamiento esperado y el observado. Cada celda indica el número de hallazgos reportados por la regla. 47
4.4. Hallazgos detectados en el escenario combinado (C4). 48
4.5. Conformidad local por *Fitness Function* en cada escenario (verificaciones superadas sobre verificaciones realizadas). 48
4.6. Clasificación de los cinco escenarios bajo ambos criterios. La columna «esperado» recoge el estado que el escenario merece según las derivas efectivamente inducidas. . 52
4.7. Contraste entre los indicadores metodológicos y los resultados observados. 53
4.8. Diagnóstico de *ArchUnit* 1.3.0 antes y después de declarar las exenciones. 54
4.9. Inyección controlada de derivas en siete sistemas reales. La variación de la CR corresponde a las versiones con una sola deriva. 58
4.10. Evolución de la conformidad respecto del catálogo declarado para la versión de referencia. «Posteriores» resume los hallazgos que aparecieron después de ella. 60
4.11. Distribución del acoplamiento eferente de los paquetes verificados por FF3 (percentiles por rango más cercano). 62
4.12. Dependencias entre paquetes extraídas por el mecanismo y por JDepend sobre los mismos binarios. 63
4.13. Las reglas del caso de estudio escritas con *ArchUnit* y declaradas en el catálogo del mecanismo. 64
4.14. Contraste de los componentes de la hipótesis de trabajo. 65

5.1. Cumplimiento de los criterios de éxito comprometidos en el anteproyecto. 70

6.1. Riesgos éticos, tratamiento aplicado y estado al cierre del proyecto. 79

B.1. Exploración del asistente de catálogo: 5 escenarios × 3 repeticiones por serie, sin contexto. Las derivas que siguen detectándose se cuentan sobre los catálogos válidos. 90

C.1. Decisiones que se apartan de la letra del anteproyecto. 93
C.2. Registro de decisiones del trabajo. 94

Capítulo 1

Introducción

La arquitectura de un sistema de software no es una propiedad estática: se moldea, se negocia y, con frecuencia, se deteriora a medida que el código evoluciona. Cuando las decisiones de implementación se apartan de la estructura prevista sin que nadie lo registre, aparece la deriva arquitectónica, un distanciamiento paulatino entre la arquitectura deseada y la que efectivamente quedó plasmada en el código fuente (Perry and Wolf, 1992; Li et al., 2022). Se trata de un fenómeno silencioso, acumulativo y costoso, que rara vez se manifiesta de golpe; por el contrario, avanza mediante pequeñas concesiones —una dependencia que cruza una capa, un atajo entre módulos, una clase que concentra demasiadas responsabilidades— que, tomadas de forma aislada, parecen inofensivas.

Las consecuencias de ese deterioro han sido ampliamente documentadas. La erosión arquitectónica incrementa de forma sostenida el esfuerzo de mantenimiento, encarece la incorporación de nuevas funcionalidades y reduce la capacidad del equipo para predecir el impacto de sus cambios (Pigazzini, 2021). Estudios empíricos recientes confirman que ciertos *smells* arquitectónicos correlacionan de manera negativa con la mantenibilidad del software (Jolak et al., 2025), y que los propios desarrolladores perciben la deriva como una fuente persistente de fricción en su trabajo cotidiano (Anthony et al., 2024). El problema no es menor: en un estudio con desarrolladores de la industria, replicado sobre un segundo conjunto de datos independiente, Besker et al. (2019) estimaron que la deuda técnica consume en promedio el 23 % del tiempo de desarrollo, y la revisión sistemática de Li et al. (2022) identifica entre las consecuencias de la erosión el deterioro de la calidad y una evolución cada vez más rígida y frágil.

El escenario se agudiza en las pequeñas y medianas empresas (PyMES) de software. A diferencia de las grandes organizaciones, estas compañías suelen operar con equipos reducidos, ciclos de entrega ajustados y una documentación arquitectónica escasa, desactualizada o, en muchos casos, inexistente (Hamed et al., 2023). Carecen, además, de los recursos humanos y económicos para desplegar las plataformas de verificación sofisticadas que sí están al alcance de corporaciones con mayor madurez. El resultado es una paradoja preocupante: precisamente donde la gobernanza técnica sería más rentable, es donde menos se practica. Frente a este panorama, resulta pertinente preguntarse si es posible trasladar los beneficios de la verificación arquitectónica a un entorno de recursos limitados, mediante un mecanismo ligero, automatizado y que no dependa de documentación externa.

Este trabajo propone responder esa inquietud a través de un mecanismo que combina el análisis estático de grafos de dependencia con la ejecución de *Fitness Functions*. Estas últimas, popularizadas en la práctica de la ingeniería de software, consisten en verificaciones objetivas y automatizables que expresan, en forma de criterios cuantificables, las propiedades que la arquitectura debe preservar (Paul and Wang, 2019; Ford et al., 2017, 2022). En lugar de confiar en un documento de

arquitectura que probablemente esté desactualizado, el mecanismo infiere la estructura real del sistema a partir de su propio código, la contrasta contra un conjunto acotado de reglas estructurales y emite un diagnóstico sobre el estado de conformidad del proyecto. De esta forma, la gobernanza arquitectónica deja de ser un ejercicio documental y se convierte en una práctica continua, verificable y de bajo costo.

1.1. Antecedentes y contexto

La preocupación por mantener la coherencia entre el diseño previsto y el código construido no es reciente. Desde que la ingeniería de software reconoció a la arquitectura como una disciplina con identidad propia, a comienzos de la década de 1990, quedó establecido que la estructura de un sistema condiciona de manera decisiva sus atributos de calidad: rendimiento, mantenibilidad, seguridad y capacidad de evolución (Perry and Wolf, 1992; Bass et al., 2012). La arquitectura dejó de entenderse como un dibujo accesorio y pasó a concebirse como el andamiaje sobre el que se sostienen todas las decisiones posteriores; de ahí que su deterioro no sea un asunto cosmético, sino una amenaza concreta contra la viabilidad del producto.

Paralelamente, la evidencia empírica consolidó una constatación incómoda: los sistemas reales cambian, y cambian mucho. Las leyes de evolución del software enunciadas por Lehman (1980) anticiparon que todo programa que resulte útil a su comunidad se verá sometido a una presión continua de modificación, y que, a medida que crece, su estructura tiende a degradarse a menos que se invierta un esfuerzo explícito en preservarla. La deriva arquitectónica es, en buena medida, la manifestación contemporánea de esa ley: el costo de no intervenir se paga, tarde o temprano, en forma de complejidad estructural y deuda acumulada.

El concepto de deuda técnica, acuñado por Cunningham (1992) para describir las consecuencias de aplazar decisiones de diseño, proporcionó el vocabulario con el que la industria pudo nombrar el fenómeno. Desde entonces, la gestión de esa deuda se convirtió en un campo de estudio activo, y su variante estructural —la deuda técnica arquitectónica— en uno de sus frentes más difíciles de atender, precisamente porque sus síntomas no se manifiestan en una línea de código aislada, sino en la relación entre componentes que, por separado, parecen saludables.

En el ámbito de las pequeñas y medianas empresas, esta problemática se cruza con otra restricción. La literatura sobre transformación digital de PyMES coincide en señalar que estas organizaciones necesitan soluciones de bajo costo, de adopción incremental y que no exijan especialistas dedicados (Patrício and Varela, 2025; Jamil, 2026). Los marcos de gobernanza pensados para corporaciones resultan, por su peso y su demanda de roles, poco realistas para equipos pequeños. Esta tensión entre la necesidad de control estructural y la escasez de recursos constituye el terreno fértil sobre el que se levanta la presente propuesta.

1.2. Definición del problema

1.2.1. Planteamiento del problema

El punto de partida es una situación concreta y, a la vez, extendida: los sistemas de software de las PyMES crecen sin que exista un mecanismo que vigile, de manera continua, la coherencia entre lo que se pretendía construir y lo que realmente se construyó. Los síntomas son reconocibles para cualquier profesional del área: dependencias que cruzan fronteras que no debían cruzarse, módulos que se acoplan de forma imprevista, componentes que se vuelven imprescindibles para todo el sistema y una dificultad creciente para estimar cuánto costará el siguiente cambio. Las causas son múltiples, pero se concentran en la ausencia de documentación arquitectónica viva, la presión por entregar con rapidez y la falta de herramientas de verificación asequibles para equipos pequeños (Hamed et al., 2023; Klein et al., 2022).

Si la situación se mantiene, los efectos negativos se profundizan. La deuda técnica se acumula en forma de *smells* arquitectónicos que, según la evidencia disponible, terminan comprometiendo la mantenibilidad y la longevidad de los productos (Sas and Avgeriou, 2023; Jolak et al., 2025). Cada desviación no detectada encarece las siguientes: el equipo invierte tiempo en comprender un código cuya estructura real ya no se corresponde con ninguna representación mental compartida. La organización, en consecuencia, ve erosionada su capacidad de respuesta ante el mercado, justo cuando más la necesita.

1.2.2. Sistematización del problema

A partir de lo anterior, la investigación se orienta mediante una pregunta central, formulada de manera abierta para que actúe como guía y no como una búsqueda de una única causa:

¿Es posible diseñar y evaluar un mecanismo automatizado, basado en análisis estático, grafos de dependencia y Fitness Functions, que permita detectar tempranamente derivas arquitectónicas y clasificar el estado de conformidad estructural de un sistema de software para PyMES sin depender de documentación arquitectónica externa?

De esta pregunta se desprenden interrogantes subsidiarios que delimitan las etapas del trabajo: qué propiedades estructurales deben vigilarse para que el diagnóstico sea útil y acotado; de qué manera puede obtenerse, a partir del código fuente, un grafo de dependencias lo bastante fiel para soportar la evaluación; cómo debe traducirse el resultado de las verificaciones en una clasificación del estado arquitectónico que sea comprensible para un equipo de desarrollo; y, finalmente, cómo evidenciar, mediante un experimento controlado, que el mecanismo efectivamente detecta las derivas que se le inducen.

1.2.3. Hipótesis de trabajo

El anteproyecto formuló la hipótesis que orienta la evaluación: si se implementa un mecanismo automatizado que extraiga dependencias del código, ejecute *Fitness Functions* sobre un grafo estructural y clasifique los resultados mediante umbrales explícitos de conformidad, entonces será posible (i) detectar tempranamente derivas arquitectónicas relevantes en un proyecto de software

de PyME, (ii) diferenciar de manera objetiva un estado aceptable de uno problemático y (iii) producir evidencia suficiente para apoyar decisiones de mantenimiento y refactorización, sin depender de documentación arquitectónica externa. Los tres componentes se contrastan por separado, con la evidencia que corresponde a cada uno y declarando lo que no quedó demostrado, en la sección 4.12.

1.3. Objetivos del proyecto

1.3.1. Objetivo general

Desarrollar y evaluar experimentalmente un mecanismo automatizado para la detección de derivas arquitectónicas, basado en grafos de dependencia y en la ejecución de *Fitness Functions*, que permita clasificar el estado de conformidad estructural de proyectos de software de PyMES sin depender de documentación arquitectónica externa.

1.3.2. Objetivos específicos

- **OBJ1.** Formular un catálogo acotado de *Fitness Functions* que traduzcan, en criterios verificables, las propiedades estructurales esenciales que deben preservarse en un sistema de software (integridad de capas, ausencia de ciclos y control de la concentración de dependencias salientes).
- **OBJ2.** Implementar un componente de análisis estático capaz de construir, a partir del código fuente, un grafo de dependencias entre paquetes que sirva de insumo común para todas las verificaciones.
- **OBJ3.** Construir un motor de validación que ejecute las *Fitness Functions* sobre el grafo, agregue sus resultados en una Tasa de Conformidad (CR) y clasifique el estado arquitectónico del sistema en niveles interpretables.
- **OBJ4.** Ejecutar un experimento controlado sobre un caso de estudio para comprobar que el mecanismo detecta las derivas inducidas y clasifica correctamente tanto una implementación de referencia como una versión alterada.

1.4. Delimitaciones y alcances

El trabajo se circunscribió a un conjunto de fronteras definidas de antemano, con el propósito de hacer viable la propuesta en el tiempo y con los recursos disponibles. En primer lugar, el mecanismo se orientó exclusivamente al ecosistema Java, aprovechando la madurez de las herramientas de análisis estático disponibles para esa plataforma; no se abordaron otros lenguajes ni entornos de ejecución. En segundo lugar, la detección se limitó a tres dimensiones estructurales —integridad de capas, dependencias cíclicas y concentración de dependencias salientes—, dejando por fuera otras manifestaciones de la deriva, como la erosión a nivel de servicios distribuidos o las desviaciones en los patrones de diseño. En tercer lugar, el grafo de dependencias se construyó a nivel de paquetes:

las violaciones se clasifican con esa granularidad, si bien la evidencia que las acompaña desciende hasta la referencia concreta entre clases que las origina. Por último, el experimento controlado se realizó sobre un caso de estudio sintético, como aprobó el anteproyecto, y se complementó con validaciones sobre código real —cuatro bibliotecas y tres aplicaciones en capas de código abierto, y treinta y cinco versiones publicadas de tres de ellas—, sin desplegar el mecanismo en un entorno productivo ni contar con la participación de sus equipos; la generalización a organizaciones con arquitecturas heterogéneas y la valoración de la utilidad del reporte por equipos reales quedaron, por tanto, planteadas como trabajo futuro y no como parte del alcance cumplido.

1.5. Justificación del trabajo de grado

Resolver el problema planteado aporta valor en varios frentes que se refuerzan entre sí. Desde la perspectiva económica, un mecanismo que detecte la deriva de forma temprana permite contener el crecimiento del costo de mantenimiento, cuya magnitud ilustra el cuarto del tiempo de desarrollo que, según Besker et al. (2019), se pierde por la deuda técnica, y que la erosión arquitectónica tiende a agravar (Pigazzini, 2021; Li et al., 2022). Anticipar la detección equivale a corregir las desviaciones cuando todavía son baratas de revertir, en lugar de pagar su factura cuando ya comprometen la viabilidad del producto.

Desde el punto de vista técnico, la propuesta traslada a las PyMES una capacidad que hasta ahora se concentraba en organizaciones de mayor tamaño: la verificación continua de la conformidad arquitectónica. La evidencia disponible subraya la importancia de esta práctica para sostener la calidad de los sistemas a lo largo del tiempo (Klein et al., 2022), y herramientas como *ArchUnit* ya demostraron que es posible expresar reglas arquitectónicas como pruebas ejecutables en Java (TNG Technology Consulting, 2026). Este trabajo aprovecha esa base conceptual, pero la orienta hacia un diagnóstico agregado y clasificatorio, más sencillo de interpretar para un equipo sin especialistas dedicados a la arquitectura.

En el plano social y organizacional, la solución empodera a los equipos de desarrollo, a los arquitectos y a los líderes técnicos de las PyMES, quienes pasan de depender de la intuición o de documentos obsoletos a disponer de una señal objetiva y reproducible sobre el estado estructural de sus proyectos. La propuesta se alinea, además, con las iniciativas de gobernanza técnica liviana que vienen cobrando fuerza en el ecosistema de las pequeñas organizaciones, en las que los marcos de control deben ser proporcionales a los recursos disponibles (Modawski and Wolniak, 2025).

1.6. Metodología de la investigación

La investigación se desarrolló, tal como se definió en el anteproyecto, como una investigación aplicada de enfoque predominantemente cuantitativo bajo la estrategia de *Design Science Research* (DSR), adecuada cuando el propósito no es solo comprender un fenómeno, sino también diseñar y evaluar un artefacto que lo aborde (Hevner et al., 2004; Peffers et al., 2007). Para que el trabajo fuera alcanzable en el tiempo previsto, la estrategia se ejecutó mediante ciclos iterativos cortos, cada uno con un entregable verificable, organizados en las cinco fases que fijó el anteproyecto:

1. **Delimitación del caso de estudio y catálogo de reglas.** Se seleccionó el ecosistema Java, se construyó el sistema base y se seleccionaron, a partir de la literatura, las tres *Fitness Functions* del catálogo con su matriz de reglas (secciones 3.2 y 3.14).

2. **Diseño técnico del prototipo.** Se definieron el proceso de detección y sus entradas y salidas, la forma de declarar la arquitectura prevista, la de calcular e interpretar la Tasa de Conformidad y la arquitectura interna del prototipo, antes de programar (secciones 3.1, 3.3, 3.6, 3.7 y 3.9).

3. **Construcción del componente de extracción.** Se definieron las reglas con que se obtiene del código el grafo de dependencias entre paquetes y se implementó el módulo que las aplica (secciones 3.4 y 3.10).

4. **Construcción del motor de validación.** Sobre el grafo se implementaron la evaluación de las reglas, el cálculo de la CR, la clasificación por umbrales y el reporte de violaciones por tipo, ubicación y severidad (secciones 3.5 e 3.10).

5. **Evaluación y análisis de resultados.** Se ejecutó el experimento controlado sobre la línea base y las versiones con derivas inducidas, y se contrastaron los resultados con la matriz de validación y con los indicadores comprometidos (capítulo 4). Como complemento, y sin ampliar el alcance de OBJ4, se repitió la prueba en entorno controlado sobre siete sistemas reales, inyectándoles derivas conocidas (sección 4.7); se siguió la evolución de tres de ellos a lo largo de sus versiones publicadas para observar deriva real (sección 4.8); se calibró el umbral de concentración con un procedimiento estadístico (sección 4.9), y se contrastó la extracción con una herramienta independiente (sección 4.10).

El ciclo no fue lineal, como la propia estrategia DSR prevé: la quinta fase reveló una limitación de la métrica de conformidad que obligó a volver a la cuarta, refinar el criterio de clasificación y reejecutar el experimento, de modo que el artefacto que se presenta es el producto de una segunda iteración y no de un único paso de construcción.

Metodológicamente, el trabajo articuló procedimientos de distinta naturaleza. La construcción del grafo de dependencias se apoyó en técnicas de análisis estático, que extraen la estructura del sistema sin necesidad de ejecutarlo. La evaluación de las propiedades estructurales se formalizó mediante un catálogo de *Fitness Functions*, cada una de las cuales define sus propios criterios de verificación e incumplimiento. Finalmente, la agregación de resultados se resolvió con una métrica de conformidad acompañada de umbrales de clasificación, lo que permitió traducir los hallazgos individuales en un veredicto global sobre el estado del sistema. La validación se realizó mediante un experimento controlado que contrastó una implementación de referencia contra cuatro versiones deliberadamente alteradas, tres de ellas con una única deriva inducida para poder medir la sensibilidad de cada regla por separado, siguiendo las convenciones de diseño y reporte habituales en la investigación empírica en ingeniería de software (Wohlin et al., 2012; Runeson and Höst, 2009).

Las fases guardan con los objetivos específicos la correspondencia que estableció el anteproyecto, y que otorga trazabilidad al trabajo. La primera fase responde a OBJ1, la formulación del catálogo

de *Fitness Functions*; la segunda y la tercera, a OBJ2, la construcción de la base técnica del análisis estático y del grafo; la cuarta, a OBJ3, el motor de validación con la CR y sus umbrales; y la quinta, a OBJ4, el experimento controlado y el análisis de sus resultados. Los resultados de esa última fase retroalimentaron la cuarta, cerrando el ciclo iterativo que caracteriza a la investigación en diseño.

Se emplearon los cuatro instrumentos previstos. La **revisión bibliográfica focalizada** consolidó los criterios de deriva y la selección de las reglas (capítulo 2). La **matriz de reglas arquitectónicas** registró, para cada regla, su racionalidad, su evidencia esperada y su criterio de incumplimiento, y es el catálogo de la tabla 3.3. La **bitácora de desarrollo y ejecución** quedó en el historial de versiones del repositorio, que conserva cada iteración del prototipo, y en la evidencia que genera cada corrida, de modo que las decisiones y los resultados pueden reconstruirse; las decisiones con consecuencias conceptuales se registran además como *Architecture Decision Records* en la carpeta `decisiones/` del repositorio, con su contexto, las alternativas descartadas y su respaldo (anexo C). Las **tablas comparativas de resultados** registran el número de violaciones, la CR, el tipo de deriva y la clasificación de cada escenario frente a la línea base (sección 4.2).

1.6.1. Ejecución del cronograma

El anteproyecto planificó cinco meses, de mayo a septiembre de 2026, con 192 horas repartidas en 32 de revisión y definición de reglas, 80 de diseño e implementación, 42 de evaluación y ajustes y 38 de documentación y cierre, y previó al cierre de cada mes una revisión con el director para contrastar horas planeadas y reales. Los entregables de cada mes se produjeron en el orden previsto: el protocolo y el catálogo inicial (capítulo 3, primera parte), el diseño del prototipo y la extracción (secciones 3.4 y 3.9), las reglas ejecutables, la detección de ciclos con el cálculo de la CR y el reporte, y el estudio de caso con su análisis comparativo (capítulo 4). El trabajo se extendió hasta octubre de 2026, mes en que se añadieron las validaciones complementarias sobre código real y se consolidó el documento.

Dos desviaciones se declaran en lugar de omitirse. La primera es que **las revisiones mensuales con el director no se realizaron** durante la ejecución, de modo que la bitácora no contiene sus observaciones y la validación con él se concentra en la revisión de este documento. La segunda es que, a falta de esas revisiones, el contraste mensual entre horas planeadas y reales que preveía la metodología no quedó documentado; el historial de versiones permite, en cambio, fechar cada entregable. Ninguna de las dos afecta a los resultados técnicos, pero ambas restan la supervisión externa que el anteproyecto había previsto como control de calidad del proceso, y por eso se registran como decisión en el anexo C.

1.7. Resultados obtenidos

El mecanismo desarrollado logró construir, a partir del código compilado, un grafo de dependencias entre paquetes y ejecutar sobre él las tres *Fitness Functions* previstas. Sobre la implementación de referencia obtuvo una Tasa de Conformidad del cien por ciento, sin un solo hallazgo, y la clasificó como *aceptable*. Sobre las cuatro versiones alteradas detectó la totalidad de las seis derivas

inducidas —rupturas de la integridad de capas, dependencias cíclicas y concentración excesiva de dependencias salientes—, superando el criterio de éxito del ochenta por ciento fijado en el anteproyecto. En el escenario que combina las tres derivas, la conformidad descendió al cincuenta y siete por ciento y el sistema fue clasificado como *no conforme*.

La evaluación aportó, además, un hallazgo que no se había anticipado en su magnitud. Los escenarios construidos con una única deriva revelaron que la Tasa de Conformidad, al ser una proporción no ponderada, diluye las violaciones aisladas: en dos de esos escenarios el mecanismo detectó correctamente la desviación pero la métrica se mantuvo por encima del umbral de aceptación, de modo que el sistema quedaba clasificado como aceptable pese a contener una violación real. Ese resultado motivó un refinamiento del criterio de clasificación —que incorpora la gravedad del peor hallazgo junto a la extensión que mide la métrica, sin alterar su cálculo— con el cual los cinco escenarios pasaron a clasificarse conforme a lo esperado.

Al margen del experimento controlado, el mecanismo se aplicó sobre una biblioteca de terceros de casi mil clases para observar su comportamiento ante código que nadie preparó para él. El análisis completo tardó poco más de un segundo y la regla de integridad de capas no produjo un solo falso positivo. Reveló, en cambio, una segunda limitación: sobre código real, una regla puramente topológica confunde un componente deteriorado con uno que concentra dependencias por el papel que desempeña, hasta el punto de que dos tercios de sus hallazgos resultaron ser ruido. La respuesta fue incorporar exenciones declaradas y registradas en el reporte, que eliminan ese ruido sin silenciar los hallazgos genuinos.

Finalmente, para comprobar que lo observado en el caso de estudio se sostiene fuera de él, se inyectaron derivas conocidas en siete sistemas Java reales —cuatro bibliotecas de entre cuatrocientas y dos mil clases y tres aplicaciones en capas cuya arquitectura documentan sus propios proyectos—. El mecanismo detectó las treinta y una derivas inyectadas sin producir un solo hallazgo espurio y en menos de un segundo por sistema, y la dilución de la Tasa de Conformidad reapareció con mayor claridad: en el sistema más conforme, una deriva aislada la dejaba por encima del noventa y siete por ciento. Sobre treinta y cinco versiones publicadas de tres bibliotecas, el mecanismo encontró además deriva real, que nadie inyectó: una dependencia nueva de la introspección hacia la deserialización en Jackson Databind 2.19.0 y una concentración nueva en Commons Lang 3.21.0, mientras que Guava conservó su arquitectura a lo largo de doce versiones publicadas. Una herramienta independiente, JDepend, extrajo el mismo grafo salvo veintiuna aristas, todas explicadas por las reglas de extracción documentadas, y las mismas reglas escritas con *ArchUnit* detectaron lo mismo con más del doble de especificación y sin veredicto agregado. Los capítulos de desarrollo y evaluación detallan los resultados y el razonamiento que condujo a cada ajuste.

1.8. Estructura del documento

El presente documento se organiza en seis capítulos y tres anexos. El capítulo 2, *Marco de referencia*, expone las bases teóricas sobre deriva arquitectónica, *Fitness Functions*, análisis estático, grafos de dependencia y métricas de acoplamiento, junto con el estado del arte. El capítulo 3, *Desarrollo del proyecto*, es el núcleo del trabajo y se divide en dos partes: la primera explica el

proceso de detección —qué entidades intervienen, de dónde proceden y por qué se seleccionaron las *Fitness Functions*, cómo se declara la arquitectura prevista, con qué reglas se extrae el grafo, cómo se obtiene la deriva, en qué unidades se mide y cómo se interpreta—, y la segunda describe el prototipo que lo instrumenta y los escenarios del caso de estudio. El capítulo 4, *Evaluación*, presenta el experimento controlado, su interpretación, el refinamiento que motivó, la aplicación a sistemas reales y las amenazas a la validez. El capítulo *Conclusiones* responde a los objetivos, precisa la aplicación del mecanismo en la industria y traza las líneas de trabajo futuro, y el capítulo 6, *Riesgos éticos*, examina las salvaguardas asociadas a su uso. El anexo A reúne las instrucciones para reproducir todos los resultados, el anexo B documenta una exploración del asistente de catálogo con modelos de lenguaje reales, y el anexo C resume el registro de decisiones del trabajo.

Capítulo 2

Marco de referencia

Este capítulo sienta las bases conceptuales que sostienen el resto del documento. En primer lugar, se delimitan los conceptos centrales —deriva arquitectónica, deuda técnica arquitectónica y *smells*— para, a continuación, presentar las *Fitness Functions* como instrumento de verificación, el análisis estático y los grafos de dependencia como insumo estructural, las métricas de acoplamiento que permiten cuantificarlos y la noción de gobernanza continua como marco de aplicación. Finalmente, se revisan los trabajos previos más relevantes y se contrastan mediante una tabla comparativa, con el fin de evidenciar el vacío que este proyecto pretende ocupar.

2.1. Marco Teórico

2.1.1. Arquitectura de software y atributos de calidad

Antes de abordar la deriva, conviene precisar qué se entiende por arquitectura de software y por qué su integridad importa. La arquitectura es el conjunto de estructuras principales de un sistema, junto con las relaciones que las conectan y los principios que gobiernan su diseño y evolución (Bass et al., 2012). No es un producto tangible, sino una propiedad emergente que se materializa en las dependencias, las capas, los módulos y los contratos que articulan el código. Su papel es doble: por un lado, sirve de vehículo para alcanzar los atributos de calidad deseados; por otro, actúa como marco de comunicación entre quienes construyen y quienes mantienen el sistema (Perry and Wolf, 1992).

La relevancia de la arquitectura para la mantenibilidad ha sido documentada de forma consistente. Cuando la estructura se degrada, los atributos de calidad no decaen todos a la vez ni de manera uniforme: la mantenibilidad y la evolucionabilidad suelen ser los primeros en resentirse, precisamente porque dependen de que las dependencias sean predecibles y de que los cambios puedan acotarse. Un sistema cuya arquitectura se ha erosionado obliga a sus desarrolladores a razonar sobre un territorio que ya no corresponde a ningún mapa compartido, lo que eleva el costo cognitivo y material de cada modificación (Klein et al., 2022). Esta relación entre estructura y costo de cambio es la razón de fondo por la que la vigilancia arquitectónica no puede considerarse un lujo.

2.1.2. Deriva y erosión arquitectónica

La deriva arquitectónica describe el distanciamiento gradual entre la arquitectura concebida para un sistema y la que efectivamente se materializa en su implementación (Perry and Wolf, 1992; Li et al., 2022). Se distingue de la erosión arquitectónica, término que suele reservarse para el deterioro

estructural ya consumado y observable: la deriva es el proceso, la erosión es su huella acumulada. La distinción y sus múltiples denominaciones fueron sistematizadas por de Silva and Balasubramaniam (2012), quienes al revisar las estrategias disponibles para controlar la erosión las agruparon en tres familias —minimizarla mediante disciplina de proceso, prevenirla mediante restricciones impuestas por construcción y repararla una vez detectada— y señalaron que las aproximaciones basadas en verificación automática de conformidad se sitúan entre la primera y la segunda. Ambos conceptos comparten una misma raíz: las decisiones de implementación que, tomadas de forma incremental y bajo presión de entrega, van apartando al código de las restricciones estructurales que deberían gobernarlo (Anthony et al., 2024). La literatura coincide en señalar que este fenómeno no es un accidente puntual, sino una consecuencia casi inevitable de la evolución de los sistemas, lo que convierte su vigilancia en una tarea permanente y no en una actividad esporádica (Klein et al., 2022). Esta lectura es coherente con los trabajos clásicos sobre mantenimiento de la consistencia arquitectónica, que ya advertían del costo de permitir que la implementación y el diseño se separen (Rosik, 2007), y con las leyes de evolución del software, que prevén la degradación estructural como desenlace natural de todo sistema que crece (Lehman, 1980). De ahí que la deriva no deba tratarse como un fallo excepcional del proceso, sino como un riesgo estructural que reclama mecanismos de vigilancia integrados en el propio ciclo de desarrollo. El estudio de mapeo sistemático de Li et al. (2022), que reunió 73 estudios primarios publicados entre 2006 y 2019, ofrece además una orientación concreta sobre qué vigilar: de las cuatro familias de síntomas de erosión que identifica —estructurales, de violación, de calidad y de evolución—, las más frecuentes son las estructurales y las de violación, y entre sus ejemplos característicos figuran las dependencias cíclicas y la violación del patrón de capas. Son precisamente dos de las tres dimensiones que el anteproyecto fijó para este trabajo.

2.1.3. Deuda técnica arquitectónica y *smells*

El concepto de deuda técnica, acuñado originalmente para describir el costo de las decisiones de diseño postergadas (Cunningham, 1992), trasladado al plano estructural, da lugar a la deuda técnica arquitectónica: el costo diferido que una organización asume al postergar la corrección de las desviaciones que degradan la arquitectura. Los *smells* arquitectónicos —ciclos de dependencia, componentes excesivamente acoplados, violaciones de capas, concentraciones de responsabilidad— son los síntomas observables de esa deuda (Sas and Avgeriou, 2023; Arcelli Fontana et al., 2016). Su costo no es solo teórico: en un estudio con desarrolladores de la industria, replicado y validado con un segundo conjunto de datos independiente, Besker et al. (2019) estimaron que la deuda técnica consume en promedio el 23 % del tiempo de desarrollo. La evidencia empírica ha establecido una relación negativa entre la presencia de estos *smells* y la mantenibilidad de los sistemas, lo que refuerza la necesidad de instrumentos que los detecten de forma objetiva (Jolak et al., 2025; Sas et al., 2019). Para las PyMES, cuya capacidad de reacción ante la deuda acumulada es limitada, el reto no es solo detectar los síntomas, sino hacerlo antes de que su corrección resulte prohibitiva (Hamed et al., 2023).

2.1.4. *Fitness Functions*

Una *Fitness Function* es una verificación objetiva y automatizable que expresa, en términos cuantificables, una propiedad que el sistema debe preservar. El término, tomado de la biología evolutiva, fue adoptado por la ingeniería de software para describir pruebas que no verifican una funcionalidad, sino una característica estructural o de calidad de la arquitectura (Paul and Wang, 2019). La formulación de referencia es la de Ford et al. (2017), quienes las sitúan en el centro de las llamadas arquitecturas evolutivas y proponen clasificarlas según varios ejes: por su alcance, distinguiendo las *atómicas* —que evalúan una única dimensión, como una regla de dependencia entre módulos— de las *holísticas*, que evalúan la interacción de varias; y por su cadencia, separando las que se ejecutan de forma continua en cada integración de las que se activan de manera puntual o desencadenada por un evento. Las tres reglas que este trabajo implementa son, en esa taxonomía, funciones atómicas de ejecución desencadenada, y la Tasa de Conformidad que las agrega constituye una lectura holística construida sobre ellas. A diferencia de las métricas convencionales, que se limitan a describir el estado del sistema, las *Fitness Functions* incorporan un criterio de juicio: producen un veredicto de cumplimiento o incumplimiento sobre una regla declarada. Esta propiedad las hace idóneas para la gobernanza arquitectónica, pues permiten codificar las decisiones de diseño como comprobaciones ejecutables que se revalúan en cada iteración del desarrollo. La segunda edición de la obra de referencia amplía precisamente ese uso, al tratar las *Fitness Functions* como el instrumento de una gobernanza arquitectónica automatizada (Ford et al., 2022). Herramientas como *ArchUnit* han popularizado esta idea en el ecosistema Java, al permitir expresar reglas arquitectónicas como pruebas unitarias (TNG Technology Consulting, 2026). En la práctica industrial se distingue, además, entre reglas que bloquean una integración de inmediato y aquellas que solo registran la desviación para su revisión posterior, distinción que la literatura técnica de divulgación denomina *guardrails* arquitectónicos (Rossini, 2022). El énfasis en convertir decisiones de diseño en comprobaciones conecta las *Fitness Functions* con la documentación explícita de las decisiones arquitectónicas, cuya justificación —hacer transparentes la arquitectura y su razón de ser para todos los interesados— formularon Tyree and Akerman (2005), y que la práctica actual registra en *Architecture Decision Records* (ADR) (AOE Technology Radar, 2026). Este trabajo registra así sus propias decisiones (anexo C).

2.1.5. Análisis estático y grafos de dependencia

El análisis estático consiste en examinar el código fuente —o sus artefactos compilados— sin ejecutar el programa, con el fin de extraer propiedades estructurales. Cuando el resultado de ese examen se organiza como un grafo dirigido, en el que los nodos representan componentes (paquetes, clases o módulos) y las aristas representan dependencias entre ellos, se obtiene una representación compacta y verificable de la arquitectura real del sistema (Manglaras et al., 2026). Este enfoque es particularmente valioso en el contexto de este trabajo porque permite prescindir de la documentación arquitectónica: la estructura se infiere directamente del código, que es la única fuente de verdad siempre disponible. La elección entre analizar el código fuente o el *bytecode* compilado responde a un compromiso práctico: el primero ofrece mayor riqueza semántica, mientras que el segundo simplifica

la resolución de dependencias al operar sobre una representación ya consolidada. En el ecosistema Java, herramientas como *ArchUnit* ejemplifican este segundo enfoque, al importar clases compiladas y construir sobre ellas un modelo navegable de dependencias (TNG Technology Consulting, 2026), al igual que *JDepend*, que calcula las métricas de paquetes de Martin directamente sobre el *bytecode* (Clark, 2015), y que *Arcan*, que detecta *smells* arquitectónicos sobre grafos extraídos de proyectos Java compilados (Arcelli Fontana et al., 2016). Los modelos de reflexión de Murphy et al. (2001) y los mecanismos de sincronización arquitectónica (Diaz-Pace et al., 2011) constituyen antecedentes directos de esta manera de contrastar el código con las expectativas estructurales.

Representar la estructura como un grafo dirigido no es una comodidad de notación: habilita el uso de algoritmos con propiedades conocidas. La detección de ciclos, en particular, se reduce a hallar los componentes fuertemente conexos del grafo, problema que Tarjan (1972) resolvió con un algoritmo de una sola pasada en profundidad y complejidad lineal en el número de nodos y aristas. Un componente fuertemente conexo de tamaño mayor que uno es, por definición, un conjunto de paquetes mutuamente alcanzables, es decir, un ciclo. Esta correspondencia entre un concepto arquitectónico y una construcción de la teoría de grafos es la que permite que la verificación sea exacta y no heurística.

2.1.6. Métricas de acoplamiento

Algunas propiedades estructurales no se expresan como una prohibición binaria sino como una cuestión de grado, y para ellas hace falta una medida. Martin (2003) introdujo dos métricas que se han vuelto estándar en el análisis de dependencias entre paquetes: el **acoplamiento eferente** (Ce), que cuenta cuántos paquetes distintos conoce un paquete dado, y el **acoplamiento aferente** (Ca), que cuenta cuántos dependen de él. La primera mide la exposición del paquete a los cambios ajenos; la segunda, su responsabilidad sobre los cambios propios.

Un valor elevado de Ce señala un paquete que ha ido acumulando conocimiento de todo el sistema, y es el indicio habitual de lo que la literatura denomina componente dios: un módulo del que todo depende y que depende de todo, cuya modificación repercute en cascada y cuyo reemplazo se vuelve inviable (Sas and Avgeriou, 2023). Conviene subrayar, no obstante, que la métrica es puramente topológica y no distingue el papel que el paquete desempeña: una raíz de composición o una fachada presentan por diseño un Ce alto sin que ello constituya una deriva. De ahí que el umbral a partir del cual un valor se considera excesivo no sea universal, sino un parámetro que debe calibrarse según el sistema y el rol de cada componente. Para esa calibración existe un procedimiento establecido: derivar el umbral de la distribución que la métrica presenta en sistemas de referencia y fijarlo en un percentil alto, en lugar de elegirlo a juicio (Lanza and Marinescu, 2006). Alves et al. (2010) lo sistematizaron sobre cien sistemas y propusieron los percentiles 70, 80 y 90 como fronteras de riesgo moderado, alto y muy alto. Es el procedimiento que se aplica en la sección 4.9 para fijar el umbral de concentración.

2.1.7. Conformidad arquitectónica y gobernanza continua

La conformidad arquitectónica es el grado en que la implementación de un sistema se adhiere a las reglas estructurales que se han definido para él (Knodel and Popescu, 2007; Klein et al., 2022). Su verificación no es un fin en sí mismo, sino el insumo de una práctica más amplia: la gobernanza continua, entendida como la vigilancia sistemática y automatizada de las propiedades arquitectónicas a lo largo de todo el ciclo de vida del producto. En organizaciones pequeñas, donde los marcos de control deben ser livianos y proporcionados a los recursos disponibles, la gobernanza continua solo es viable si se apoya en instrumentos de bajo costo y de interpretación sencilla (Modawski and Wolniak, 2025). Esta constatación orienta el diseño del mecanismo que se describe en los capítulos siguientes: un instrumento que agregue las verificaciones individuales en una señal global comprensible para un equipo de desarrollo sin especialistas dedicados. A la vigilancia automatizada se suma, como complemento, la documentación de decisiones arquitectónicas. Los *ADR* proporcionan el registro deliberado de por qué se adoptó una estructura y no otra, y constituyen el insumo natural contra el cual contrastar las desviaciones detectadas por el mecanismo (AOE Technology Radar, 2026). De esta manera, la gobernanza continua no se reduce a un acto de verificación, sino que articula un ciclo de documentación, monitoreo y corrección.

2.2. Estado del Arte

La detección de la deriva arquitectónica ha sido abordada desde múltiples frentes, que conviene examinar para situar la contribución de este proyecto. Un primer grupo de trabajos se concentra en la verificación de conformidad mediante modelos explícitos. Los modelos de reflexión de Murphy et al. (2001) comparan una representación de alto nivel con la estructura extraída del código, pero presuponen la existencia y el mantenimiento de ese modelo de alto nivel, un requisito que las PyMES rara vez satisfacen. Knodel and Popescu (2007) sistematizaron esta familia al comparar los tres enfoques disponibles para la verificación estática de conformidad —modelos de reflexión, matrices de dependencias y lenguajes de reglas— y concluyeron que ninguno domina a los demás en todas las dimensiones: los modelos de reflexión ofrecen mayor poder expresivo a cambio de exigir una arquitectura de referencia, mientras que los lenguajes de reglas resultan más económicos de adoptar pero cubren un espectro menor de restricciones. Esa constatación es directamente pertinente aquí, pues el presente trabajo se sitúa en la tercera familia y asume conscientemente ese compromiso. En la misma línea, Diaz-Pace et al. (2011) exploran la sincronización entre la arquitectura documentada y la implementación, y San Martín et al. (2024), en un trabajo aún en forma de preprint, estudian la verificación de conformidad en sistemas autoadaptativos mediante un lenguaje específico para declarar la arquitectura planificada; ambos asumen igualmente una especificación arquitectónica de referencia.

Un segundo grupo aborda la deriva a través de sus síntomas. Arcelli Fontana et al. (2016) detectan automáticamente *smells* de inestabilidad —dependencias cíclicas, concentradoras (*hub-like*) e inestables— sobre el grafo de dependencias de proyectos compilados, Sas et al. (2019) investigan su evolución, y Sas and Avgeriou (2023) proponen un índice de deuda técnica arquitectónica basa-

do en aprendizaje de máquina aplicado a la detección de *smells*. Estos trabajos aportan evidencia valiosa sobre el diagnóstico, pero se orientan a la caracterización del deterioro más que a ofrecer un veredicto de conformidad accionable para un equipo de desarrollo cotidiano.

Un tercer grupo lo constituyen las propuestas centradas en las *Fitness Functions* como práctica. Paul and Wang (2019) y Ford et al. (2022) formalizan la idea de verificar propiedades arquitectónicas mediante comprobaciones automatizadas, y *ArchUnit* la materializa en el ecosistema Java (TNG Technology Consulting, 2026). Estos trabajos definen el *cómo* verificar, pero no proponen un mecanismo que agregue los resultados en una clasificación global del estado del sistema. Por último, trabajos recientes como Manglaras et al. (2026) reconstruyen arquitecturas de sistemas distribuidos a partir del código, lo que confirma el interés creciente por extraer estructura sin depender de documentación.

Un cuarto grupo, más reciente, agrupa las herramientas y guías orientadas a hacer práctica la verificación en el día a día de los equipos. Los *guardrails* arquitectónicos, difundidos como complemento de las *Fitness Functions* (Rossini, 2022), y las soluciones de verificación por convención para distintos ecosistemas —como *NetArchTest* para .NET (Panneerselvam, 2023) o *ArchUnit* para Java (codecentric AG, 2026)— evidencian que la industria ya demanda instrumentos ligeros de control estructural. Estas fuentes se citan como testimonio de la práctica, no como respaldo de las decisiones del trabajo, que descansan en la literatura arbitrada. Sin embargo, estas herramientas comparten una limitación: exigen que alguien escriba y mantenga las reglas, y no ofrecen por sí solas una lectura agregada del estado global del sistema.

La tabla 2.1 sintetiza esta comparación en torno a cuatro criterios: el enfoque adoptado, si la propuesta detecta derivas de forma automatizada, si prescinde de documentación arquitectónica externa y si resulta asequible para una PyME sin especialistas dedicados.

Tabla 2.1: Comparación de los trabajos relacionados con la propuesta. Leyenda: ✓ lo cumple, ∼ lo cumple parcialmente, × no lo cumple.

Trabajo	Enfoque	Automática	Sin doc. externa	Veredicto agregado	Asequible PyME
Murphy et al. (2001)	Modelos de reflexión	×	×	×	×
Knodel and Popescu (2007)	Comparación de enfoques de conformidad	✓	×	×	∼
Diaz-Pace et al. (2011)	Sincronización arquitectónica	∼	×	×	×
San Martín et al. (2024)	Verificación de conformidad (MAPE-K)	✓	×	×	×
Arcelli Fontana et al. (2016)	Detección de *smells* (*Arcan*)	✓	✓	×	∼
Sas et al. (2019)	Evolución de *smells* de inestabilidad	∼	✓	×	∼
Sas and Avgeriou (2023)	Índice de deuda técnica con aprendizaje de máquina	∼	✓	✓	∼
Ford et al. (2017, 2022); Paul and Wang (2019)	*Fitness Functions* como práctica	✓	∼	×	∼
TNG Technology Consulting (2026)	Reglas arquitectónicas como pruebas	✓	∼	×	∼
Manglaras et al. (2026)	Reconstrucción de arquitectura	✓	✓	×	∼
Esta propuesta	*Fitness Functions* sobre grafos, con tasa de conformidad y severidad	✓	✓	✓	✓

La columna «veredicto agregado» merece una precisión, porque es la que sostiene el argumento de la brecha. Solo Sas and Avgeriou (2023) producen una cifra global, pero lo hacen mediante un modelo de aprendizaje de máquina cuyo resultado no es reconstruible por el equipo que lo recibe; el índice indica cuánta deuda hay, no qué reglas se incumplieron ni dónde. La agregación que aquí se propone persigue el objetivo contrario: una cifra deliberadamente simple, calculada como un cociente que cualquier miembro del equipo puede rehacer a mano a partir de los hallazgos que la componen.

El examen anterior revela una brecha consistente: los enfoques basados en modelos exigen documentación que las PyMES no mantienen; los enfoques basados en *smells* describen el deterioro sin ofrecer un veredicto agregado; y las propuestas de *Fitness Functions* aportan el mecanismo de verificación sin resolver la agregación en un diagnóstico clasificatorio. Ninguno de los trabajos revisados combina, en un solo instrumento ligero y sin dependencias de documentación externa, la extracción del grafo de dependencias, la evaluación mediante *Fitness Functions* y la clasificación del estado de conformidad en niveles interpretables. Es precisamente ese espacio el que ocupa la presente investigación. Conviene precisar desde ahora en qué sentido la propuesta prescinde de la documentación: no de toda declaración de la intención —la deriva solo puede medirse contra una arquitectura prevista—, sino de la documentación arquitectónica extensa que los enfoques basados

en modelos presuponen, que se sustituye por un catálogo mínimo de capas y dependencias admitidas que vive junto al código y se verifica en cada ejecución.

2.3. Resumen del capítulo

Este capítulo definió los conceptos fundamentales que articulan el proyecto: deriva y erosión arquitectónica, deuda técnica arquitectónica y *smells*, *Fitness Functions*, análisis estático con grafos de dependencia, las métricas de acoplamiento que permiten cuantificar la concentración de responsabilidades y la gobernanza continua como marco de aplicación. Dos correspondencias resultan especialmente relevantes para los capítulos siguientes: la que vincula la noción de ciclo de dependencia con los componentes fuertemente conexos de un grafo, que permite verificarla de forma exacta (Tarjan, 1972), y la que vincula el componente dios con el acoplamiento eferente, que permite medirlo aunque exija calibrar su umbral (Martin, 2003). Sobre esa base, se revisaron los trabajos previos agrupándolos en cuatro corrientes —verificación basada en modelos, detección de *smells*, *Fitness Functions* como práctica y herramientas de verificación por convención— y se los contrastó mediante una tabla comparativa. La comparación evidenció la brecha que motiva la propuesta: la ausencia de un mecanismo que integre, de forma asequible y sin documentación externa, la detección y una clasificación agregada del estado de conformidad que siga siendo reconstruible por quien la recibe.

Capítulo 3

Desarrollo del Proyecto

Este capítulo describe el mecanismo de detección de deriva arquitectónica en dos partes. La primera explica el **proceso**: qué entidades intervienen y cómo se relacionan, de dónde proceden las *Fitness Functions* y por qué se eligieron, cómo declara una persona la arquitectura prevista de su sistema, con qué reglas se extrae el grafo de dependencias, cómo se combinan esos elementos para obtener la deriva, en qué unidades se expresa y cómo se interpreta. La segunda describe el **prototipo** que instrumenta ese proceso, que es el medio y no el fin del trabajo. En términos de la metodología, el capítulo recoge las cuatro primeras fases del proyecto —la delimitación del caso de estudio y del catálogo de reglas, el diseño técnico, la construcción del componente de extracción y la del motor de validación—; la quinta, la evaluación, ocupa el capítulo siguiente.

Parte I. El proceso de detección

3.1. Visión general del proceso

La deriva arquitectónica es la divergencia entre la arquitectura prevista de un sistema y la que su código materializa (Perry and Wolf, 1992; San Martín et al., 2024). Para detectarla de forma automática hay que convertir esa definición en una comparación ejecutable, y eso exige disponer de los dos términos de la comparación: la **arquitectura prevista**, que solo puede declarar quien diseña el sistema, y la **arquitectura implementada**, que puede extraerse del código sin intervención humana. El mecanismo representa la primera como un *catálogo de reglas* y la segunda como un *grafo de dependencias entre paquetes*, y los compara mediante tres *Fitness Functions*: verificaciones objetivas que expresan, cada una, una propiedad estructural que el sistema debe preservar (Ford et al., 2017). La tabla 3.1 resume las entidades que intervienen.

Tabla 3.1: Entidades del proceso de detección.

Entidad	**Qué representa**	**Origen**
Código compilado	La arquitectura implementada, tal como existe.	El sistema analizado.
Catálogo de reglas	La arquitectura prevista: capas, dependencias admitidas, umbral y exenciones.	La persona que diseña el sistema (sección 3.3).
Grafo de dependencias	Los paquetes del sistema y las dependencias entre ellos, con su evidencia.	Extracción automática (sección 3.4).
Fitness Function	Una propiedad estructural verificable: integridad de capas, ausencia de ciclos, concentración acotada.	Catálogo fijo del mecanismo (sección 3.2).
Verificación	Cada comprobación atómica que una regla realiza sobre una arista o un paquete.	Evaluación (sección 3.5).
Hallazgo	Un incumplimiento, con su ubicación, su evidencia y su severidad.	Evaluación.
Tasa de Conformidad	Proporción de verificaciones superadas.	Agregación (sección 3.6).
Estado arquitectónico	Clasificación en *aceptable*, *en observación* o *no conforme*.	Agregación.
Tendencia	Variación de la CR respecto del análisis anterior del mismo sistema.	Historial de análisis.

Estas entidades se encadenan en seis pasos (figura 3.1). Dos de ellos corresponden a la persona y cuatro al mecanismo. La persona **declara** la arquitectura prevista (P1) y, al final, **interpreta** el diagnóstico y decide qué hacer con él (P6). El mecanismo **extrae** el grafo del código (P2), **asigna** cada paquete a la capa que el catálogo le atribuye (P3), **evalúa** las tres reglas sobre el grafo, produciendo verificaciones y hallazgos con su evidencia (P4), y **agrega** los resultados en una Tasa de Conformidad, una severidad máxima y un estado (P5). La deriva es, por tanto, el conjunto de hallazgos de P4: las dependencias y estructuras del código que contradicen lo declarado en P1 o las propiedades que toda arquitectura debe preservar. P5 la resume en cifras, y P6 la convierte en decisiones.

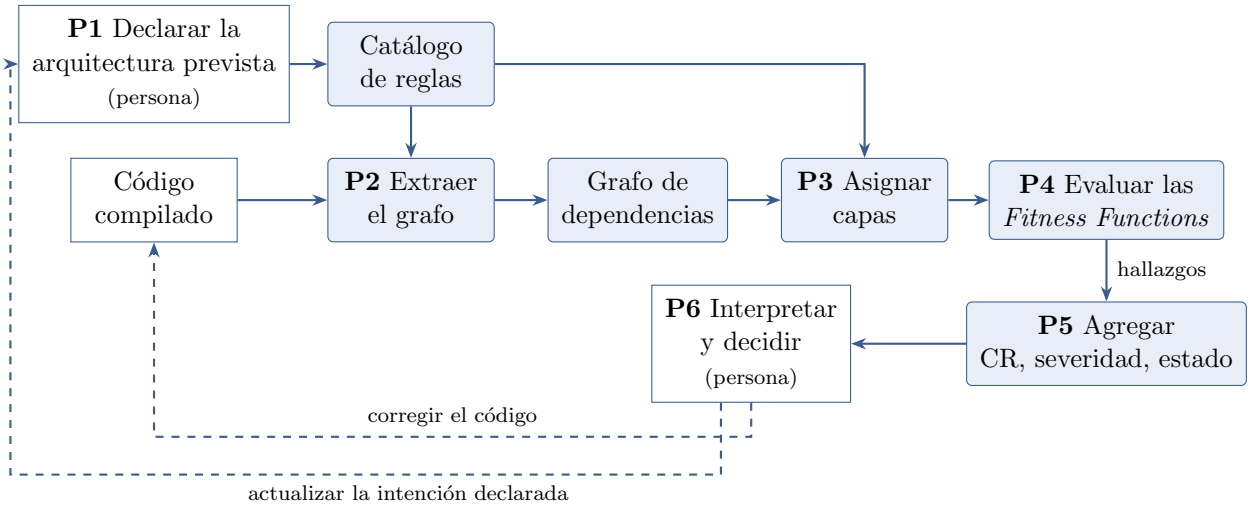


Figura 3.1: Proceso de detección de deriva arquitectónica. El catálogo interviene en la extracción —delimita qué paquetes forman el sistema— y en la asignación de capas. Las flechas discontinuas son las dos salidas de una decisión: corregir el código, cuando el hallazgo es deriva, o actualizar la intención declarada, cuando la arquitectura cambió de forma deliberada.

El proceso no termina en P6: se repite en cada iteración del desarrollo, y la comparación entre una ejecución y la anterior —la tendencia— es la que revela si la deriva avanza o se contiene. Las secciones siguientes desarrollan cada paso.

3.2. Origen y selección de las *Fitness Functions*

Las *Fitness Functions* del mecanismo no son una elección arbitraria ni una configuración del usuario: proceden de lo que la literatura identifica como manifestaciones estructurales de la deriva, y su selección quedó acotada por el primer objetivo específico del anteproyecto, que fijó tres familias —violaciones de capas, ciclos de dependencia y concentración excesiva de dependencias salientes—. Esta sección reconstruye ese razonamiento.

Dónde se manifiesta la deriva. Perry and Wolf (1992) distinguen la deriva —la insensibilidad progresiva a la arquitectura prevista— de la erosión, su violación directa, y ambas se manifiestan en la estructura de dependencias del código. La literatura ofrece dos maneras complementarias de detectarlas. La primera es la **verificación de conformidad**: comparar las dependencias del código con un modelo de alto nivel declarado por las personas. Es la idea de los modelos de reflexión de Murphy et al. (2001), que identifican como *divergencias* las dependencias presentes en el código y ausentes del modelo, y la de los enfoques de verificación de conformidad comparados por Knodel and Popescu (2007) y revisados en la encuesta sobre control de la erosión de de Silva and Balasubramaniam (2012), que coinciden en que las reglas de dependencia entre módulos o capas son

el control más extendido. La segunda es la detección de ***smells* arquitectónicos**: estructuras que degradan el sistema con independencia de su intención. Sas et al. (2019) estudian la evolución de los tres *smells* de inestabilidad más investigados —la dependencia cíclica, la dependencia concentradora (*hub-like*) y la dependencia inestable—, y Sas and Avgeriou (2023) los emplean como base de un índice de deuda técnica arquitectónica. Finalmente, los principios de diseño de paquetes de Martin (2003) —dependencias acíclicas, dependencias estables y abstracciones estables— y sus métricas de acoplamiento ofrecen el vocabulario cuantitativo con que se expresan esas estructuras.

Criterios de selección. De esas fuentes se derivan candidatas, que se contrastaron con cinco criterios coherentes con el propósito y el alcance del trabajo: (C1) que la literatura las asocie a la deriva o a la erosión; (C2) que puedan verificarse estáticamente sobre un grafo de paquetes, sin ejecutar el sistema; (C3) que su criterio de incumplimiento sea objetivo y explicable a un profesional sin formación específica; (C4) que su configuración sea barata para una PyME; y (C5) que, juntas, cubran dimensiones distintas, incluyendo al menos una que dependa de la intención declarada y otras que no dependan de ella. La tabla 3.2 recoge el contraste.

Tabla 3.2: Candidatas a *Fitness Function* y decisión de selección.

Candidata	**Fuente**	**Decisión y motivo**
Reglas de dependencia entre capas	Murphy et al. (2001); Knodel and Popescu (2007); de Silva and Balasubramaniam (2012)	**Incluida (FF1).** Es la única que mide la deriva en sentido estricto —contra la intención declarada— y la forma más extendida de verificación de conformidad (C1, C5).
Dependencia cíclica	Martin (2003); Sas et al. (2019)	**Incluida (FF2).** Viola el principio de dependencias acíclicas, es el *smell* más estudiado y su detección es exacta (C1–C4). No requiere parámetros.
Concentración de dependencias salientes	Martin (2003); Arcelli Fontana et al. (2016); Sas et al. (2019)	**Incluida (FF3).** El *smell* de dependencia concentradora (*hub-like*) combina dependencias entrantes y salientes; FF3 toma de él solo el componente que el anteproyecto fijó —el acoplamiento eferente Ce—, que requiere un único umbral (C1–C4).
Dependencia inestable	Martin (2003); Sas et al. (2019)	**Excluida.** Exige comparar la inestabilidad relativa de cada par de paquetes; su incumplimiento es menos inmediato de explicar (C3) y su relación con la intención es indirecta.
Distancia a la secuencia principal	Martin (2003)	**Excluida.** Requiere medir la abstracción de cada paquete y no tiene un umbral de consenso (C3, C4); mide calidad de diseño, no conformidad.
Reglas a nivel de clase y patrones de diseño	TNG Technology Consulting (2026)	**Excluida.** Cambia la granularidad del análisis y queda fuera del alcance aprobado; se plantea como trabajo futuro.

Las tres reglas seleccionadas se complementan de una forma que conviene subrayar. FF1 es **prescriptiva**: solo tiene sentido respecto de una arquitectura declarada, y es la que detecta la deriva propiamente dicha. FF2 y FF3 son **intrínsecas**: señalan estructuras perjudiciales en cualquier

arquitectura, y por eso pueden evaluarse incluso sin catálogo. Las tres son, en la taxonomía de Ford et al. (2017), funciones atómicas —evalúan una sola dimensión— de ejecución desencadenada, y la Tasa de Conformidad que las agrega constituye una lectura holística sobre ellas.

El catálogo resultante. La tabla 3.3 formaliza cada regla con los campos que exigía el primer objetivo específico —racionalidad, evidencia esperada y criterio de incumplimiento— y con los que la evaluación necesita: la propiedad que protege, la unidad sobre la que verifica, su severidad, su parámetro y su respaldo bibliográfico.

La severidad responde a cuánto compromete cada incumplimiento la capacidad de cambiar el sistema. Un ciclo es **crítico** porque funde varios paquetes en una única unidad de cambio: ninguno puede modificarse, probarse ni liberarse por separado, y la estructura tiende a crecer, porque cada paquete que se añade al ciclo arrastra a los demás. Una violación de capas o una concentración son **mayores**: degradan la separación de responsabilidades, pero de forma localizada —una arista, un paquete— y reversible sin reorganizar el resto del sistema. Esa diferencia es la que justifica que, en el criterio de clasificación de la sección 3.6, un solo ciclo baste para declarar un sistema no conforme. La jerarquía es coherente con la literatura: Martin (2003) formula la ausencia de ciclos como un principio y no como una preferencia, Arcelli Fontana et al. (2016) y Sas et al. (2019) tratan la dependencia cíclica como el más dañino de los *smells* de inestabilidad por su tendencia a crecer, y el índice de Sas and Avgeriou (2023), validado con profesionales, asigna a los ciclos mayor severidad estimada que a los demás *smells*. Aun así, las severidades son una convención declarada y no una medida: el modo `cr` permite prescindir de ellas.

3.3. Declaración de la arquitectura prevista

El catálogo es la única entrada que aporta la persona, y es la que convierte un análisis de estructura en un análisis de deriva: sin él, el mecanismo puede señalar estructuras perjudiciales, pero no desviaciones respecto de una intención. Por eso el catálogo no se deduce del código, sino que se declara.

Qué se declara. El catálogo es un archivo de propiedades de unas pocas decenas de líneas con seis elementos: la **raíz** de paquetes que delimita el sistema; las **capas**, cada una con un nombre y uno o más prefijos de paquete; la **matriz de dependencias permitidas**, que indica de qué otras capas puede depender cada una; el **umbral de concentración**; las **exenciones**; y, opcionalmente, los umbrales de la CR y el criterio de clasificación. El del caso de estudio es el siguiente:

```
raiz=com.caso
capas=presentacion,negocio,persistencia
capa.presentacion=com.caso.presentacion
capa.negocio=com.caso.negocio
capa.persistencia=com.caso.persistencia
permitido.presentacion=negocio
```

Tabla 3.3: Catálogo de *Fitness Functions* implementadas.

	FF1 — Integridad de capas	**FF2 — Ausencia de ciclos**	**FF3 — Concentración de dependencias**
Propiedad protegida	La estratificación declarada se respeta.	El grafo de dependencias entre paquetes es acíclico.	Ningún paquete concentra más dependencias salientes de las admisibles.
Racionalidad	Una dependencia que salta o invierte capas anula la separación de responsabilidades y propaga los cambios fuera de su ámbito previsto.	Los paquetes de un ciclo no pueden compilarse, probarse ni comprenderse por separado: forman una unidad de cambio que ninguna decisión de diseño declaró.	Un acoplamiento eferente elevado señala un paquete que asumió más responsabilidades de las que le corresponden; su modificación arrastra al sistema entero (Martin, 2003).
Unidad de verificación	Cada arista entre dos capas declaradas distintas.	Cada paquete del grafo.	Cada paquete del grafo.
Criterio de incumplimiento	El par (capa origen, capa destino) no figura en la matriz de adyacencias permitidas.	El paquete pertenece a un componente fuertemente conexo de tamaño mayor que uno.	El acoplamiento eferente Ce del paquete supera el umbral configurado.
Evidencia esperada	Arista infractora, par de capas, adyacencias autorizadas y referencias entre clases que la originan.	Ruta cíclica concreta, paquetes del componente y referencias entre clases que cierran el ciclo.	Valores de Ce y Ca, umbral aplicado y lista de paquetes destino.
Severidad	Mayor	Crítica	Mayor
Parámetro	Matriz `permitido.*`	Ninguno	`umbral.concentracion`
Respaldo	Violación de la arquitectura prevista (Perry and Wolf, 1992); divergencias en los modelos de reflexión (Murphy et al., 2001); reglas de dependencia en la verificación de conformidad (Knodel and Popescu, 2007; de Silva and Balasubramaniam, 2012).	Principio de dependencias acíclicas (Martin, 2003); dependencia cíclica como *smell* arquitectónico (Sas et al., 2019); detección exacta mediante componentes fuertemente conexos (Tarjan, 1972).	Acoplamiento eferente Ce (Martin, 2003); componente eferente del *smell* de dependencia concentradora (Arcelli Fontana et al., 2016; Sas et al., 2019); umbral por percentiles (Alves et al., 2010).

```
permitido.negocio=persistencia
permitido.persistencia=
excluir=
rol.compartido=
umbral.concentracion=3
umbral.cr.aceptable=85
umbral.cr.observacion=70
clasificacion.modo=cr+severidad
```

Cómo se interpreta. Cada elemento tiene una semántica precisa, que es la que la persona debe conocer para declarar lo que pretende:

- **Asignación por prefijo.** Un paquete pertenece a la primera capa, en el orden declarado, cuyo prefijo coincide con él o lo contiene; los subpaquetes heredan la capa del prefijo. Si un prefijo contiene a otro, la capa más específica debe declararse antes.
- **Paquetes sin capa.** Un paquete que no encaja en ningún prefijo —un dominio compartido, unas utilidades— queda fuera de FF1: al no pertenecer a la estratificación, no puede afirmarse que la vulnere. Sigue evaluándose con FF2 y FF3.
- **Matriz explícita y no transitiva.** Las dependencias dentro de una misma capa se admiten siempre; entre capas, solo las declaradas. Que la presentación pueda depender del negocio y el negocio de la persistencia no autoriza a la presentación a depender de la persistencia: eso debe declararse, de modo que la matriz distingue una estratificación estricta de una relajada.
- **Exenciones.** La propiedad `excluir` retira del análisis código que **no forma parte del sistema** aunque el empaquetado lo sitúe bajo la raíz —bibliotecas embebidas, código generado—, con todos sus subpaquetes. La propiedad `rol.compartido` señala paquetes que **sí pertenecen al sistema** pero cuyo papel —una raíz de composición, una fachada, un núcleo de anotaciones— hace que concentren dependencias por diseño: permanecen en el grafo, pero quedan exentos de FF2 y FF3. Esta exención es exacta y no se hereda, porque un papel lo desempeña un paquete concreto y no un subárbol; propagarla eximiría al sistema entero cuando el paquete compartido es la raíz, que es el caso más frecuente.

Las exenciones son la forma en que el anteproyecto previó contener el riesgo de formalismo vacío: mediante un *registro de excepciones justificadas*. Por eso viven en el catálogo, donde son visibles y discutibles, y no en el código de las reglas, y por eso el reporte enumera en su sección de alcance todas las aplicadas. Una excepción declarada puede auditarse; una codificada dentro de la regla, no.

Cómo la define el usuario. La declaración sigue un procedimiento que se apoya en lo que el equipo sabe de su sistema, no en lo que el código hace hoy:

1. **Delimitar el sistema.** Fijar la raíz de paquetes y excluir el código que no es propio.
2. **Identificar las capas de la intención.** Partir de lo que el equipo decidió —registros de decisiones, convenciones del marco de trabajo, el conocimiento del líder técnico—, no de la estructura actual, que puede contener ya la deriva.
3. **Asignar los paquetes** a cada capa mediante prefijos.
4. **Declarar la matriz empezando por la más estricta** y relajarla solo ante una decisión explícita.
5. **Fijar el umbral de concentración** con un procedimiento y no a juicio: el percentil 90 del acoplamiento eferente de los paquetes no exentos en la versión que el equipo acepta como línea base (Alves et al., 2010). El procedimiento y su calibración sobre sistemas reales se describen en la sección 4.9; lo que evita es elegir el valor justo por encima de un paquete concreto para dejar de señalarlo.
6. **Declarar las exenciones** solo con una justificación que quede escrita junto a ellas.
7. **Validar y versionar.** El mecanismo rechaza un catálogo inconsistente —capas sin paquetes, dependencias hacia capas inexistentes, exenciones fuera de la raíz, umbrales no numéricos o invertidos—; una vez aceptado, se guarda junto al código y se actualiza cuando cambia la intención, no cuando cambia el código.

Ese archivo no contradice el propósito de no depender de documentación externa. Lo que el mecanismo evita es depender de la documentación arquitectónica tradicional —diagramas y descripciones que se escriben aparte, envejecen sin que nada lo advierta y que las PyMES rara vez mantienen—. El catálogo es otra cosa: una declaración mínima que vive en el repositorio y se verifica en cada ejecución, de modo que no puede quedar desactualizada en silencio. Las tres reglas, además, no dependen de él en la misma medida: FF2 no necesita ningún parámetro, FF3 solo un umbral con valor por defecto, y únicamente FF1 exige declarar capas y dependencias, que son precisamente la intención que el equipo conoce aunque no la haya escrito. Por eso, sin catálogo, el mecanismo aplica una **configuración mínima** —infiere la raíz como el prefijo común de los paquetes y evalúa FF2 y FF3— y advierte que la estratificación no se verificó. Para reducir el esfuerzo de la declaración, el prototipo ofrece además un asistente que propone un catálogo a partir de la estructura del código y que la persona revisa antes de adoptarlo (sección 3.15).

3.4. Extracción del grafo de dependencias

El segundo término de la comparación, la arquitectura implementada, se obtiene del código mediante análisis estático: sin ejecutar el sistema, se recorre su código compilado y se registra qué clases dependen de cuáles. El resultado se agrega a nivel de paquete y se representa como un grafo dirigido $G = (N, E)$, en el que los nodos N son paquetes y cada arista $(p, q) \in E$ indica que alguna clase de p depende de alguna clase de q.

Por qué el código compilado. Se prefirió el *bytecode* al código fuente porque materializa todas las dependencias que el compilador resolvió —incluidas las referencias implícitas que el código fuente no escribe de forma literal—, en un formato uniforme y sin necesidad de reconstruir el entorno de compilación (TNG Technology Consulting, 2026); es también la opción de herramientas como *JDepend* y *Arcan* (Clark, 2015; Arcelli Fontana et al., 2016) y de la mayoría de las revisadas por Knodel and Popescu (2007). No altera lo que el segundo objetivo específico pide —extraer el grafo del código del sistema y de los archivos de configuración necesarios—: el *bytecode* es ese mismo código fuente una vez compilado, y el único archivo de configuración que la extracción necesita es el catálogo, del que toma la raíz y las exclusiones. La lectura de las clases se delega en la biblioteca *ArchUnit* (TNG Technology Consulting, 2026). Conviene declararlo como lo que es: una **precisión** del término «código fuente» del anteproyecto, no un cambio de alcance. El mecanismo sigue analizando el código del propio sistema y sin documentación externa, pero lo hace sobre su forma compilada, y hereda de ella los puntos ciegos que se enumeran al final de esta sección. La fidelidad del grafo resultante no se dio por supuesta: se contrastó con el que extrae una herramienta independiente sobre los mismos binarios (sección 4.10).

Por qué el paquete. El grafo se construye a nivel de paquete y no de clase. La decisión sacrifica precisión a cambio de dos beneficios: reduce el número de aristas hasta una escala legible y comprensible, y descarta las dependencias entre clases de un mismo paquete, que carecen de relevancia arquitectónica. Es, además, la granularidad en que se declaran las capas.

Reglas de extracción. El grafo se construye aplicando siete reglas:

R1. **Universo de clases.** Se importan todas las clases del código compilado, salvo las de prueba, que se reconocen por las convenciones de directorio de las herramientas de construcción.

R2. **Qué es una dependencia.** Una clase depende de otra cuando su *bytecode* la referencia en alguna de las formas de la tabla 3.4. Las formas que el compilador elimina —una constante que copia en su lugar de uso, una anotación de solo código fuente, una importación o variable que no se usa— no dejan rastro y no producen dependencia.

R3. **Agregación.** Existe la arista (p, q) si al menos una clase de p depende de al menos una clase de q, con $p \neq q$; las dependencias dentro de un mismo paquete se descartan.

R4. **Delimitación.** Solo se conservan los paquetes bajo la raíz declarada; las dependencias hacia el JDK o hacia bibliotecas externas se descartan, y los paquetes excluidos se retiran como origen y como destino.

R5. **Evidencia.** Cada arista conserva las referencias concretas entre clases que la originan, con el miembro y la línea, de modo que todo hallazgo pueda rastrearse hasta el código.

R6. **Determinismo.** Las clases y sus dependencias se recorren en orden alfabético, para que la misma entrada produzca siempre el mismo grafo y la misma evidencia.

R7. **Validez.** Si el grafo resulta vacío, o si el código está compilado para una versión de Java que la biblioteca no reconoce —*ArchUnit* 1.3.0 lee hasta Java 23 y omite sin aviso las clases posteriores—, la extracción se detiene con un error en lugar de producir un diagnóstico: un grafo vacío o incompleto obtendría una conformidad alta sin haber examinado el sistema.

Tabla 3.4: Formas de referencia entre clases y su efecto en el grafo, verificadas por la batería de pruebas sobre código compilado.

Producen dependencia	**No producen dependencia**
Herencia de clase e implementación de interfaz; tipo de un campo, de un parámetro, de retorno y de una excepción declarada; llamada a un método, creación de una instancia y acceso a un campo; anotación conservada en el *bytecode*; argumento de tipo genérico; literal de clase (`X.class`); comprobación `instanceof`; tipo de los elementos de un arreglo.	Constante de tiempo de compilación, que el compilador copia en su lugar de uso; anotación de solo código fuente; importación o variable local sin uso; conversión de tipo (*cast*), que la versión empleada de la biblioteca no registra.

Del grafo se derivan las dos métricas de acoplamiento que emplea FF3 (Martin, 2003): el acoplamiento eferente $Ce(p) = |\{q : (p, q) \in E\}|$, número de paquetes de los que p depende, y el aferente $Ca(p) = |\{q : (q, p) \in E\}|$, número de paquetes que dependen de p.

Lo que el grafo no ve. Las reglas anteriores delimitan también lo que la extracción no puede capturar, y que conviene tener presente al interpretar cualquier resultado: las dependencias que solo existen en tiempo de ejecución —carga dinámica de clases, reflexión a partir de nombres, configuración externa—, las que el compilador elimina (tabla 3.4) y las que cruzan los límites del sistema delimitado por la raíz.

3.5. Evaluación: cómo se obtiene la deriva

La evaluación combina las tres entidades anteriores: el grafo aporta los hechos, el catálogo la intención y las *Fitness Functions* el criterio con que se comparan. Para expresarlo con precisión, sea $\mathcal{C}$ el conjunto de capas declaradas, $\lambda : N \to \mathcal{C} \cup \{\bot\}$ la asignación de cada paquete a su capa —o a ninguna, $\bot$— según las reglas de la sección 3.3, $P \subseteq \mathcal{C} \times \mathcal{C}$ la matriz de dependencias permitidas, $X \subseteq N$ los paquetes con rol compartido y u el umbral de concentración.

FF1, integridad de capas. Verifica cada arista que une dos capas declaradas distintas,

$$V_1 = \{(p, q) \in E : \lambda(p) \neq \bot,\ \lambda(q) \neq \bot,\ \lambda(p) \neq \lambda(q)\},$$

y registra un incumplimiento por cada una cuyo par de capas no está permitido, $(\lambda(p), \lambda(q)) \notin P$. Las aristas dentro de una capa y las que tocan un paquete sin capa no se verifican.

FF2, ausencia de ciclos. Retira del grafo los paquetes con rol compartido, $G' = G \setminus X$, y calcula sus componentes fuertemente conexos con el algoritmo de Tarjan (1972). Verifica cada paquete de G' —de modo que $|V_2| = |N \setminus X|$— y registra un incumplimiento por cada paquete que pertenece a un componente de más de un elemento. Cada componente da lugar a un único hallazgo, cuya evidencia incluye una ruta concreta que cierra el ciclo.

FF3, concentración de dependencias. Verifica cada paquete sin rol compartido, $|V_3| = |N \setminus X|$, y registra un incumplimiento por cada uno cuyo acoplamiento eferente, medido sobre el grafo completo, supera el umbral: $Ce(p) > u$.

Un ejemplo completo. El escenario combinado del caso de estudio (C4, sección 3.14) tiene cinco paquetes y once dependencias, y se evalúa con el catálogo de la sección 3.3. La tabla 3.5 recorre el cálculo. FF1 encuentra cuatro aristas entre capas distintas: las dos descendentes que la matriz permite y las dos que la persistencia dirige hacia el negocio y la presentación, que no lo están. FF2 verifica los cinco paquetes —no hay exentos— y encuentra un componente que reúne a los tres de capa: tres incumplimientos, un hallazgo. FF3 verifica los cinco y encuentra que `util` depende de cuatro paquetes, por encima del umbral de tres. La deriva de C4 son, por tanto, cuatro hallazgos —dos de capas, un ciclo y una concentración— que suman seis incumplimientos en catorce verificaciones.

Tabla 3.5: Evaluación del escenario C4, paso a paso.

Regla	**Verif.**	**Incumpl.**	**Hallazgos**	**Detalle**
FF1 Capas	4	2	2	`persistencia`→`negocio` y `persistencia`→`presentacion`, no permitidas.
FF2 Ciclos	5	3	1	Un componente con `presentacion`, `negocio` y `persistencia`.
FF3 Concentración	5	1	1	`util`: $Ce = 4 > u = 3$.
Total	**14**	**6**	**4**	$CR = 100 \times (14 - 6)/14 = 57{,}14\,\%$; severidad máxima: crítica.

3.6. Unidades de medida y agregación

La evaluación produce hallazgos; la agregación los convierte en magnitudes que permitan comparar escenarios, seguir la evolución de un sistema y decidir. La tabla 3.6 resume las unidades en que se expresa cada resultado.

Tabla 3.6: Unidades en que se expresa la deriva.

Magnitud	**Unidad o escala**	**Qué expresa**
Verificación	Conteo (aristas o paquetes)	Cada comprobación atómica realizada; es el denominador de la CR.
Incumplimiento	Conteo	Verificaciones no superadas.
Hallazgo	Conteo, con ubicación y evidencia	Cada desviación que debe atenderse; un ciclo es un hallazgo con varios incumplimientos.
Conformidad local	Porcentaje, por regla	Qué dimensión se deteriora.
Tasa de Conformidad (CR)	Porcentaje, de 0 a 100	Proporción de verificaciones superadas en el sistema.
Severidad	Ordinal: mayor, crítica	Gravedad del peor hallazgo.
Estado	Ordinal: aceptable, en observación, no conforme	Veredicto que resume el diagnóstico.
Tendencia	Puntos porcentuales	Variación de la CR respecto del análisis anterior del mismo sistema.
Ce, Ca	Número de paquetes	Acoplamiento de cada paquete; sustenta FF3.

La Tasa de Conformidad. Convierte un conjunto de hallazgos dispersos en una señal única. El anteproyecto la definió como la proporción de reglas cumplidas frente al total de reglas evaluadas. Con un catálogo de tres reglas, esa formulación admite únicamente cuatro valores posibles —0 %, 33 %, 67 % y 100 %—, lo que la vuelve incapaz de distinguir entre un sistema con una violación de capas y otro con veinte. Se conservó la definición como cociente, pero desplazando la unidad de conteo desde la regla hasta la **verificación atómica** definida en la sección 3.5:

$$CR = 100 \times \frac{V - I}{V}, \tag{3.1}$$

donde $V = |V_1| + |V_2| + |V_3|$ es el número total de verificaciones e I el de incumplimientos. Cuando $V = 0$ la CR se define como 100 % por convención, aunque la regla R7 impide que ese caso llegue a producirse sobre un sistema real. Este refinamiento es una precisión de la definición aprobada, no una sustitución: la CR sigue siendo una proporción explícita de cumplimiento, se interpreta con los mismos umbrales del 85 % y el 70 % y es reconstruible a partir de los hallazgos individuales. Lo que gana es resolución.

Para que la precisión sea verificable y no una afirmación, la formulación del anteproyecto no se abandonó: el mecanismo la calcula y la reporta siempre, en la consola, en los archivos CSV y JSON y en la interfaz, como **CR por regla**,

$$CR_{\text{regla}} = 100 \times \frac{|\{\, r \in R : I_r = 0 \,\}|}{|R|}, \tag{3.2}$$

donde R es el conjunto de reglas que realizaron al menos una verificación e I_r el número de incumplimientos de la regla r. No interviene en la clasificación, pero permite contrastar las dos lecturas sobre la misma evidencia; la sección 4.3.1 muestra el resultado de ese contraste.

La fórmula arrastra tres limitaciones que condicionan cómo debe leerse cualquier valor:

- **Las unidades son heterogéneas.** FF1 cuenta aristas y FF2 y FF3 cuentan paquetes, de modo que el peso relativo de cada regla en la CR depende de la topología del sistema.
- **El denominador no es constante entre versiones.** Un paquete nuevo aumenta V aunque no introduzca ninguna violación; la CR de dos versiones solo es comparable acompañada del conteo de verificaciones, que el reporte declara siempre.
- **La proporción diluye las violaciones localizadas.** A medida que crece V, una violación aislada pesa cada vez menos. No es un defecto de implementación sino una consecuencia de toda métrica proporcional, y su alcance se midió durante la evaluación.

La tendencia. Si el valor absoluto de la CR se diluye con el tamaño, su **variación** no lo hace: un descenso de cuatro puntos entre dos análisis es igual de significativo en un sistema de cinco paquetes que en uno de quinientos, porque se mide contra el propio sistema y con un denominador casi idéntico. El mecanismo registra cada análisis y, cuando dispone de uno anterior del mismo sistema, acompaña la cifra con su variación:

```
Tasa de Conformidad     : 86.41%
Tendencia               : +4.10 pp respecto de la corrida
                          anterior (82.31%) - mejora
```

La clasificación. El estado se obtiene de dos lecturas. La lectura **por CR** aplica los umbrales del anteproyecto: aceptable si $CR \geq 85\,\%$, en observación si $70\,\% \leq CR < 85\,\%$ y no conforme si $CR < 70\,\%$. La lectura **por severidad** atiende al peor hallazgo: no conforme si hay alguno crítico, en observación si el peor es mayor y aceptable si no hay hallazgos. El criterio combinado adopta la más restrictiva de las dos, de modo que una violación aislada no quede diluida por la proporción; su motivación empírica se expone en el capítulo de evaluación, y el algoritmo 1 formaliza el procedimiento completo.

3.7. Interpretación de los resultados

El diagnóstico se lee de lo general a lo particular. Primero el **estado**, y si la severidad lo endureció respecto de lo que la CR sola habría dicho. Después la **conformidad local**, que indica qué dimensión se deteriora. Luego los **hallazgos**, por severidad, con su evidencia, que señalan dónde actuar. Por último la **tendencia**, que dice si la situación empeora, y la **declaración de alcance**, que recuerda qué no se examinó. La tabla 3.7 recoge cómo interpretar cada situación y qué decisiones admite.

Tan importante como lo que el diagnóstico afirma es lo que no autoriza a concluir. Un veredicto aceptable significa que no se hallaron incumplimientos de estas tres reglas en la estructura de paquetes, no que el sistema carezca de problemas: rendimiento, seguridad, corrección funcional y comportamiento en ejecución quedan fuera, y el reporte lo declara. La CR de dos sistemas distintos no es comparable, porque su denominador depende de la topología de cada uno; tiene sentido

Algorithm 1 Cálculo de la Tasa de Conformidad y clasificación del estado.

$V \leftarrow 0$; $I \leftarrow 0$; $s_{max} \leftarrow \varnothing$
for cada regla r del catálogo **do**
  $(v, i, H) \leftarrow$ evaluar(r, grafo) {una sola pasada por regla}
  $V \leftarrow V + v$; $I \leftarrow I + i$
  for cada hallazgo $h \in H$ **do**
    registrar(h); $s_{max} \leftarrow$ masGrave(s_{max}, severidad(h))
$CR \leftarrow 100 \times (V - I)/V$ {100 si $V = 0$}
$e_{CR} \leftarrow$ Aceptable si $CR \geq 85$; En observación si $CR \geq 70$; No conforme en otro caso
$e_{sev} \leftarrow$ No conforme si s_{max} es crítica; En observación si es mayor; Aceptable si no hubo hallazgos
if modo $=$ *cr* **then**
  estado $\leftarrow e_{CR}$
else
  estado $\leftarrow$ masRestrictivo(e_{CR}, e_{sev})
return Resultado(CR, e_{CR}, s_{max}, e_{sev}, estado, hallazgos)

comparar un sistema consigo mismo a lo largo del tiempo. Y los hallazgos se atribuyen a paquetes, no a personas: el diagnóstico es un insumo para la decisión técnica, no una medida del desempeño del equipo, en coherencia con los principios del capítulo 6.

Parte II. La instrumentación del proceso

El proceso anterior se instrumentó en un prototipo, *Arquitecta*, que automatiza los pasos P2 a P5 y apoya los pasos P1 y P6. Esta parte describe sus requisitos, su arquitectura, su implementación y su verificación, y presenta los escenarios del caso de estudio sobre los que se evaluó.

3.8. Requisitos

La construcción del prototipo se apoyó en un conjunto de requisitos funcionales y no funcionales.
Requisitos funcionales:

- **RF1.** Construir un grafo de dependencias entre paquetes a partir del código compilado, descartando las clases de prueba y las dependencias externas al sistema analizado.
- **RF2.** Verificar la integridad de capas: toda dependencia entre capas debe respetar las adyacencias permitidas que se declaren en la configuración.
- **RF3.** Detectar dependencias cíclicas entre paquetes mediante componentes fuertemente conexos.

Tabla 3.7: Guía de interpretación del diagnóstico.

Resultado	Qué significa	Decisiones posibles
Aceptable, sin hallazgos	El código respeta la intención declarada y no presenta ciclos ni concentraciones en las tres dimensiones evaluadas.	Ninguna; no implica ausencia de otros problemas de calidad.
Hallazgo de FF1	Una dependencia que la arquitectura prevista prohíbe.	Si es deriva, revertirla o encapsularla tras la capa permitida; si la intención cambió deliberadamente, actualizar el catálogo y registrar la decisión. Nunca relajar la matriz sin registrar por qué.
Hallazgo de FF2	Paquetes que ya no pueden cambiar por separado.	Romper el ciclo invirtiendo la dependencia, moviendo la clase que lo cierra o extrayendo una abstracción; la ruta indica dónde actuar. Es la prioridad más alta.
Hallazgo de FF3	Un paquete que acumula responsabilidades de muchos otros.	Repartir sus responsabilidades; si su papel lo justifica —raíz de composición, fachada—, declararlo como exención justificada.
CR alta y estado endurecido por la severidad	La proporción oculta un hallazgo grave, típicamente un ciclo, en un sistema grande.	Atender el hallazgo aunque la cifra parezca buena.
Descenso de la CR entre análisis	Se introdujo deriva desde el análisis anterior.	Revisar los hallazgos nuevos antes de que se consoliden.
CR baja y estable	Deriva heredada, no reciente.	Priorizar por severidad; vigilar que no crezca.
Sin capas declaradas	FF1 no se evaluó.	No concluir nada sobre la estratificación; declarar el catálogo para hacerlo.

- **RF4.** Medir la concentración de dependencias salientes de cada paquete y reportar como violación aquella que supere un umbral configurable.
- **RF5.** Agregar los resultados de las verificaciones en una Tasa de Conformidad y clasificar el estado arquitectónico del sistema, emitiendo un reporte legible.
- **RF6.** Acompañar cada hallazgo de la evidencia estructural que lo origina, hasta el nivel de la referencia concreta entre clases, y declarar en el reporte el alcance del diagnóstico emitido.
- **RF7.** Representar gráficamente el grafo de dependencias, ordenado según la estratificación declarada y con los hallazgos resaltados, en un formato que pueda consultarse sin instalar herramientas adicionales.
- **RF8.** Ofrecer una interfaz gráfica desde la cual seleccionar el proyecto a analizar —la carpeta tal como la deja su herramienta de construcción, o su archivo empaquetado— y consultar el diagnóstico, el grafo y la evidencia sin recurrir a la línea de comandos.

Requisitos no funcionales:

- **RNF1.** Configurabilidad: las capas, las adyacencias permitidas, las exenciones, los umbrales y el criterio de clasificación deben poder declararse en un archivo externo sin modificar el código, y toda exención aplicada debe quedar registrada en el reporte.
- **RNF2.** Ligereza: el mecanismo debe poder ejecutarse en un entorno de desarrollo convencional, sin infraestructura adicional ni dependencias de servicios externos.
- **RNF3.** Interpretabilidad: el reporte resultante debe ser comprensible para un profesional sin formación específica en verificación arquitectónica.
- **RNF4.** Reproducibilidad: ante la misma entrada, el mecanismo debe producir el mismo diagnóstico, y debe poder emitirlo en un formato verificable por otras herramientas.

El requisito RF6 no figuraba en la formulación inicial: se incorporó durante la evaluación, al constatar que la capacidad explicativa era el único indicador con margen de mejora, lo que ilustra el carácter iterativo que la metodología de diseño prescribe (Peffers et al., 2007). Tampoco figuraban RF7 y RF8, que responden a una misma constatación posterior: un diagnóstico correcto aporta poco si quien debe leerlo necesita, para obtenerlo, conocer la línea de comandos y la disposición interna de su herramienta de construcción. Ambos sirven a RNF3 sin alterar el proceso.

3.9. Arquitectura del prototipo

El prototipo se organizó en módulos con responsabilidades separadas, de modo que cada paso del proceso pudiera desarrollarse y verificarse de forma independiente. La figura 3.2 muestra los cinco módulos del núcleo —uno por paso del proceso—, los que lo ponen al alcance del usuario —un servicio de análisis, una interfaz gráfica y un asistente opcional—, la raíz de composición que los ensambla y el sentido en que fluyen sus dependencias.

El módulo de *configuración* carga y valida el catálogo (P1). El de *grafo* construye la estructura de dependencias, calcula los componentes fuertemente conexos y reconstruye la ruta de cada ciclo (P2). El de *reglas* asigna los paquetes a capas y evalúa las tres *Fitness Functions* (P3 y P4). El *motor* agrega y clasifica (P5). El de *reporte* traduce el diagnóstico a texto legible, a formatos verificables y a una representación gráfica. Por encima del núcleo, el *servicio* encadena los pasos, de modo que la línea de comandos y la interfaz gráfica comparten un único camino de análisis; la *interfaz* presenta el resultado de forma interactiva, y el *asistente*, opcional, apoya la declaración del catálogo.

3.10. Implementación

La implementación se desarrolló en Java, con nivel de lenguaje 17, como un proyecto Maven bajo el identificador `co.javeriana.msw:arquitecta`. La lectura del *bytecode* se delegó en *ArchUnit* 1.3.0 (TNG Technology Consulting, 2026); el resto de la lógica —la detección de ciclos, el motor, los exportadores, la interfaz— se implementó sin dependencias externas, en coherencia con RNF2.

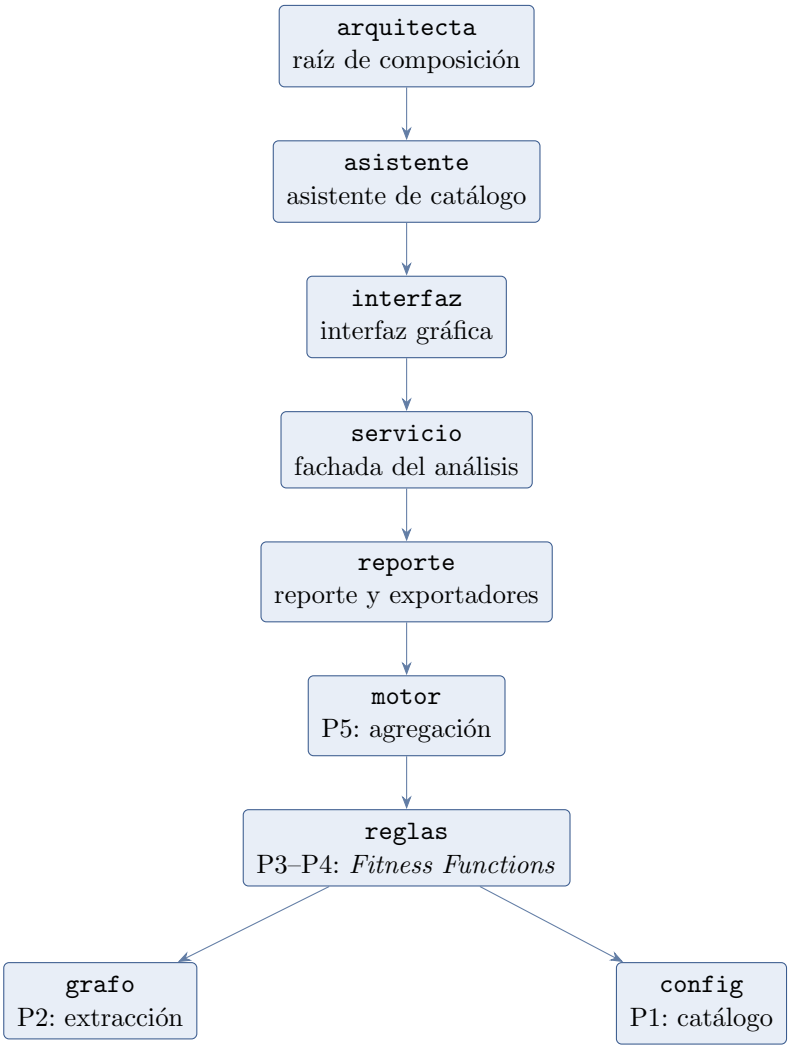


Figura 3.2: Módulos del prototipo, el paso del proceso que instrumenta cada uno y el sentido de sus dependencias. Se omiten, por legibilidad, las aristas que saltan niveles; ningún módulo depende de otro situado por encima de él, de modo que el grafo interno es acíclico.

Cada paso del proceso tiene una contraparte directa en el código. `CargadorConfiguracion` interpreta y valida el catálogo antes de que el análisis comience. `ConstructorGrafo` aplica las reglas de extracción R1 a R6 y puebla `GrafoDependencias`, que conserva la evidencia de cada arista; `ResolutorProyecto` localiza el código compilado del proyecto —reconoce las convenciones de Maven, Gradle, IntelliJ IDEA y Eclipse, o un `.jar`— y aplica la regla R7. `ReglaIntegridadCapas`, `ReglaCiclos` —apoyada en `Tarjan` y `BuscadorCiclos`— y `ReglaConcentracion` implementan las tres *Fitness Functions*; cada hallazgo es una `Violacion` que distingue la *ubicación* —los paquetes implicados, que responden a «dónde»— de la *evidencia* —las referencias y valores medidos, que

responden a «por qué»—. `MotorConformidad` aplica la ecuación 3.1 y el algoritmo 1, y `Resultado` conserva por separado el estado deducido de la CR y el de la severidad, para que la procedencia de la clasificación sea auditable.

Dos decisiones de implementación protegen la correspondencia entre el proceso y su resultado. La primera es el contrato de las reglas: cada una resuelve sus verificaciones, incumplimientos y hallazgos en una única pasada y los devuelve en un mismo objeto, cuya construcción rechaza combinaciones imposibles; así, la cifra que alimenta la CR y la evidencia que la explica no pueden proceder de recorridos distintos. La segunda es que cada regla se autodescribe —propiedad, racionalidad, unidad de verificación, criterio y severidad— y el prototipo imprime el catálogo de la tabla 3.3 a partir de su propia implementación, de modo que el catálogo publicado y el aplicado no pueden divergir, que es justamente la clase de deriva que el proyecto combate.

3.11. Representación gráfica del grafo

Un diagnóstico de conformidad se comprende mucho mejor cuando puede verse. Una lista de violaciones dice qué falla; un dibujo del grafo muestra además dónde está cada fallo respecto de la arquitectura que se pretendía, y permite advertir patrones —un paquete del que todo depende, una capa que concentra los ciclos— que ninguna tabla hace evidentes. El prototipo dibuja por ello el grafo que extrae, y lo hace sin delegar en herramientas externas.

La decisión de no delegar tiene una razón que va más allá de la ligereza. Los algoritmos genéricos de dibujo de grafos dirigidos, como el que implementa Graphviz (Gansner et al., 1993), colocan los nodos según el sentido de las aristas: un nodo se sitúa por encima de aquellos de los que depende. Aplicado a una arquitectura con deriva, ese criterio produce un resultado engañoso, porque precisamente las violaciones —dependencias que remontan la estratificación— empujan a las capas inferiores hacia arriba y pueden invertir el orden que el dibujo debería mostrar. En un grafo de conformidad, el orden vertical no debe deducirse de las aristas sino de la arquitectura declarada; y cualquier arista que lo contradiga debe dibujarse como lo que es.

El procedimiento sigue el esquema clásico del dibujo de grafos jerárquicos (Sugiyama et al., 1981), adaptado al dominio en cuatro pasos:

1. **Asignación de bandas.** Cada capa declarada ocupa una banda horizontal. Su posición se deduce de la matriz de dependencias permitidas y no del orden en que las capas se escribieron en la configuración: una capa que puede depender de otras se sitúa por encima de ellas.

2. **Asignación de filas.** Dentro de cada banda, los paquetes se distribuyen en filas según el camino más largo sobre la condensación del subgrafo de la banda, de modo que también las dependencias internas apunten hacia abajo. Los paquetes de un mismo ciclo comparten fila, y las filas demasiado anchas se parten para conservar la legibilidad.

3. **Reducción de cruces.** Cada fila se reordena según el baricentro horizontal de sus vecinos, alternando barridos descendentes y ascendentes.

4. **Columna lateral.** Los paquetes sin capa no pertenecen a la estratificación y se colocan en una columna aparte, a la altura media de sus vecinos.

El trazado de las aristas codifica su significado. Las que descienden van del borde inferior del origen al superior del destino; las que remontan la estratificación rodean los nodos por fuera; las que unen paquetes de una misma fila se dibujan como arcos. Los paquetes de los que dependen muchos otros se ensanchan en proporción, de modo que el tamaño mismo del recuadro delata a los concentradores. El color completa la lectura: rojo continuo para las violaciones de capas, rojo discontinuo para las aristas de un ciclo, ámbar para la concentración excesiva y gris atenuado para lo que concierne a un núcleo compartido exento. La misma disposición se emite en SVG, que cualquier navegador abre; en TikZ, lista para incluirse en un documento LaTeX; y en DOT, por interoperabilidad con Graphviz. Las figuras de grafos de este documento no se dibujaron a mano: las generó el propio prototipo a partir del grafo que extrajo, y son por construcción una representación fiel de lo que calculó.

3.12. Interfaz gráfica

Para que el diagnóstico pueda obtenerlo el propio líder técnico, sin componer invocaciones desde la consola, el prototipo incorpora una interfaz gráfica de escritorio (figura 3.3), en respuesta a RF8. Sus tres pasos siguen el proceso: la persona elige la **carpeta del proyecto** —la aplicación localiza el código compilado según las convenciones de la herramienta de construcción y, si solo encuentra fuentes, pide compilar—; fija el **catálogo**, que se carga solo si el proyecto lo trae y que, si falta, se sustituye por la configuración mínima con una advertencia explícita; y elige el **criterio de clasificación**.

El resultado se presenta en el orden de lectura de la sección 3.7: arriba, el estado —indicando si la severidad lo endureció—, la CR con su tendencia, el tamaño del sistema, los hallazgos y la conformidad local de cada regla; debajo, el grafo interactivo, la tabla de hallazgos con su evidencia y el reporte textual, idéntico al de la línea de comandos. La interfaz no reimplementa ninguna parte del análisis: pide el diagnóstico al servicio y dibuja lo que este le devuelve, de modo que un mismo sistema obtiene el mismo diagnóstico por cualquier vía. Se construyó con Swing, la biblioteca gráfica incluida en el JDK, de modo que RNF2 se mantiene, y su pantalla de inicio ofrece los cinco escenarios del caso de estudio para reproducir con un clic los diagnósticos del capítulo siguiente.

3.13. Verificación del prototipo

La construcción se acompañó de dos actividades de verificación que conviene distinguir de la evaluación experimental: aquí no se juzga si el mecanismo es útil, sino si el prototipo ejecuta correctamente el proceso descrito.

La primera fue una batería de **272 comprobaciones**, todas superadas, construida sin marcos de prueba externos en coherencia con RNF2. La mayoría se ejecuta sobre grafos construidos a

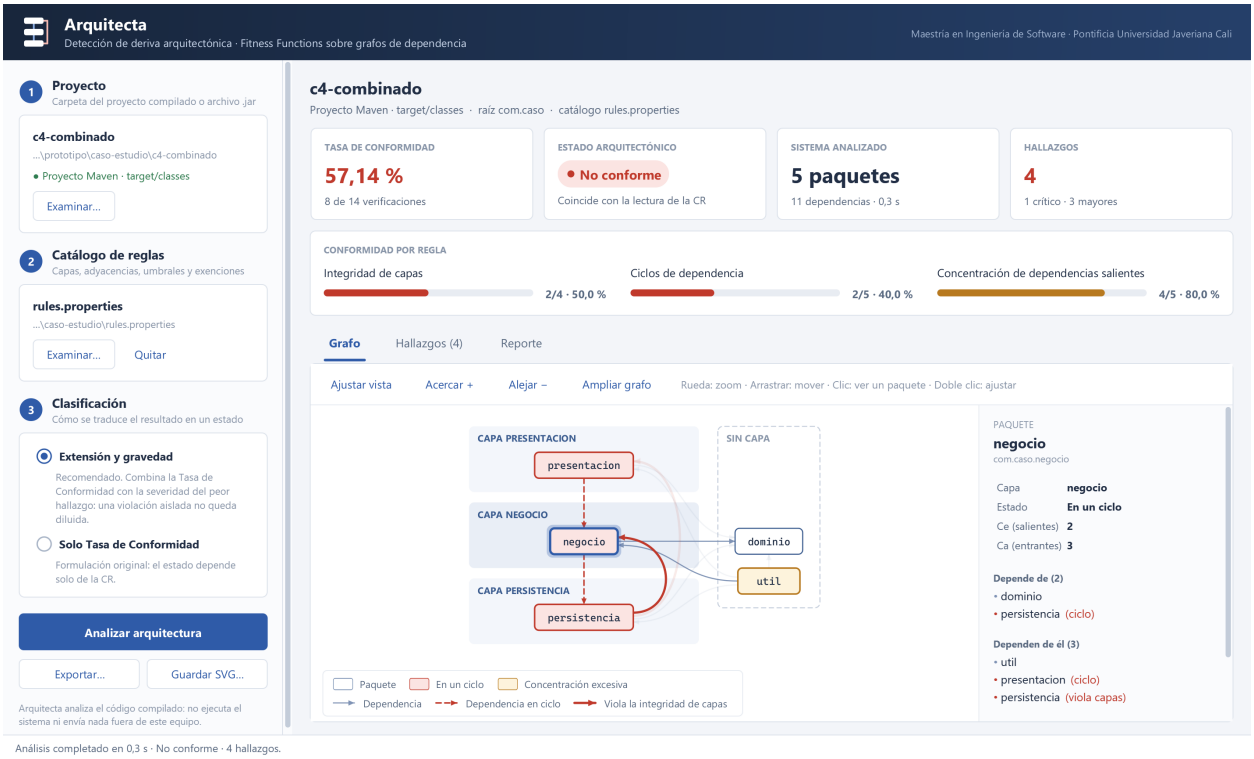


Figura 3.3: Interfaz gráfica del prototipo analizando el escenario combinado (C4). La barra lateral guía los tres pasos —proyecto, catálogo y criterio de clasificación—; el panel principal muestra los indicadores, la conformidad por regla y el grafo, en el que se ha seleccionado el paquete `negocio`: lo que no le concierne se atenúa y la ficha de la derecha detalla su capa, sus acoplamientos, sus vecinos y los hallazgos en que interviene.

mano, independientes de la extracción, lo que permite ejercitar casos límite que un caso real difícilmente ofrece: grafos vacíos, umbrales en su frontera exacta, autobucles, configuraciones inválidas y resultados internamente incoherentes. Otras recorren la extracción real: las que documentan las reglas de extracción compilan un código en que un paquete referencia a otros diecinueve de formas distintas y comprueban cuáles producen arista —la tabla 3.4 procede de ellas—, y las del servicio verifican el reconocimiento de proyectos Maven y Gradle, el rechazo de un proyecto sin compilar o compilado para una versión no soportada, y la paridad entre las distintas formas de indicar un mismo sistema.

La segunda consistió en aplicar el proceso sobre el propio prototipo. Si este predica una arquitectura estratificada y acíclica, resulta razonable exigirle que la practique. Con un catálogo que declara sus nueve paquetes como capas y prohíbe toda dependencia ascendente, el análisis arrojó **40 verificaciones, cero incumplimientos y una CR del** 100 %. El ejercicio produjo, además, un hallazgo que dio origen a las exenciones: evaluado sin ellas y con un umbral estricto, el instrumento

señalaba los módulos que ensamblan el núcleo —la raíz de composición y el servicio de análisis— y el de reporte, que concentran dependencias por el papel que desempeñan y no por deterioro. Elevar el umbral para todo el sistema habría ocultado también las concentraciones reales de cualquier otro paquete; declarar esos papeles y calibrar el umbral en cuatro resolvió el caso dejando la excepción a la vista en el reporte. El episodio muestra que FF3, al operar únicamente sobre la topología, es ciega al rol de cada paquete, y que es la declaración de la persona la que lo suple.

3.14. Escenarios de evaluación

Para evaluar el proceso se construyeron cinco versiones de un mismo sistema sintético bajo el paquete raíz `com.caso`: un pequeño dominio de ventas organizado en las capas `presentacion`, `negocio` y `persistencia`, con un paquete de dominio compartido (`com.caso.dominio`) que no pertenece a ninguna capa.

La versión **C0** constituye la implementación de referencia y respeta la arquitectura declarada: la presentación depende del negocio, el negocio de la persistencia, y no existen dependencias en sentido contrario ni ciclos. Su grafo se muestra en la figura 3.4.

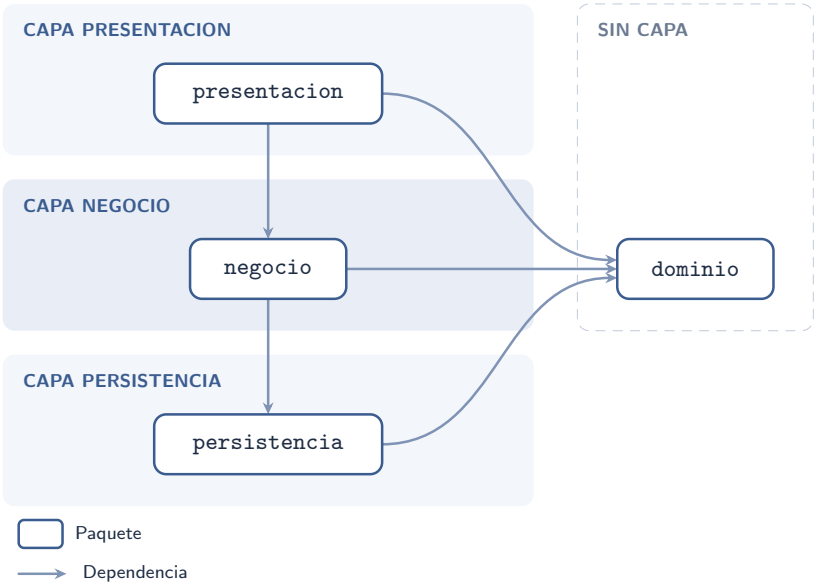


Figura 3.4: Grafo de dependencias de la implementación de referencia (C0), dibujado por el propio mecanismo: 4 paquetes y 5 dependencias, sin violaciones. Cada banda corresponde a una capa declarada; el dominio compartido, que no pertenece a ninguna, ocupa la columna lateral.

Las cuatro versiones restantes introducen derivas deliberadas. Las tres primeras las introducen **de forma aislada**, de modo que cada una active una sola *Fitness Function* y sea posible medir la sensibilidad de cada regla por separado; la cuarta las combina para observar su efecto acumulado.

- **C1 — Ruptura de capas.** El controlador de presentación accede directamente al repositorio de persistencia, saltándose la capa de negocio. La dependencia `presentacion` → `persistencia` no figura en la matriz de adyacencias y, al no invertir el sentido del flujo, no cierra ningún ciclo.
- **C2 — Ciclo de dependencia.** Un subpaquete de cálculo dentro de la capa de negocio y su paquete padre se referencian mutuamente. Como ambos pertenecen a la misma capa, la estratificación permanece intacta y solo se activa FF2. Su grafo se muestra en la figura 3.5.
- **C3 — Concentración excesiva.** Se agrega un paquete `com.caso.util` que depende de los cuatro paquetes restantes. Al no estar asignado a ninguna capa no vulnera FF1, y al no ser destino de ninguna dependencia tampoco cierra ciclos.
- **C4 — Escenario combinado.** Reúne las tres derivas: la persistencia pasa a depender del negocio y de la presentación, se forma un ciclo entre los tres paquetes de capa y reaparece el componente dios. Su evaluación paso a paso se mostró en la tabla 3.5, y su grafo en la figura 3.6.

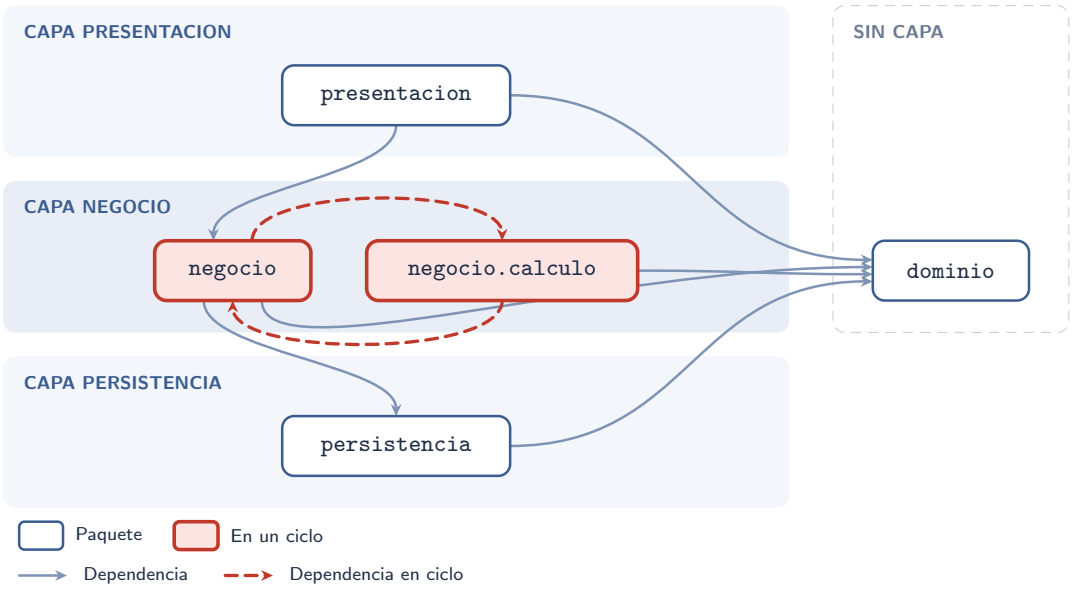


Figura 3.5: Grafo del escenario C2, dibujado por el propio mecanismo. El ciclo queda contenido en la capa de negocio: ambos paquetes comparten fila y su dependencia mutua se traza como dos arcos, uno por encima y otro por debajo. Ninguna arista remonta la estratificación.

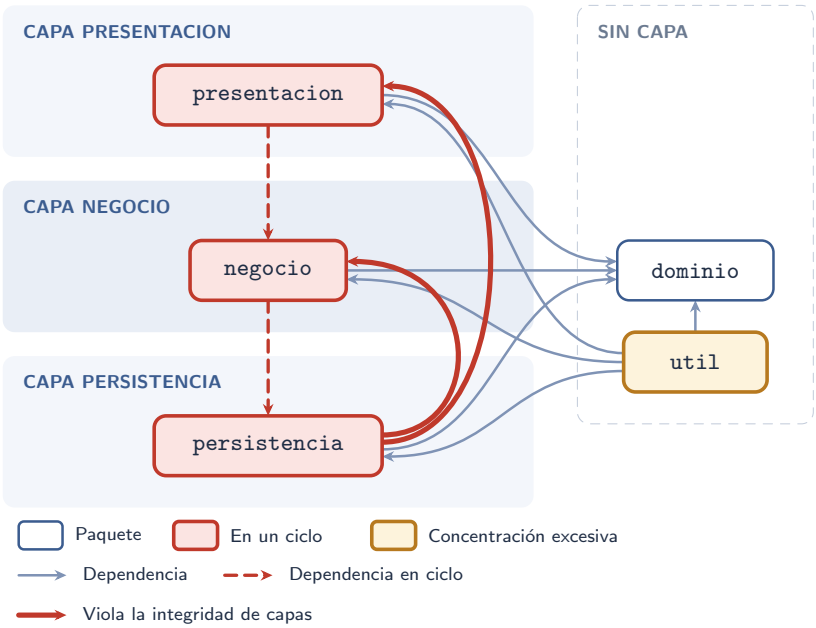


Figura 3.6: Grafo del escenario combinado (C4), dibujado por el propio mecanismo: 5 paquetes y 11 dependencias. Las dos dependencias que la persistencia dirige hacia capas superiores violan la estratificación y rodean los nodos por fuera; el ciclo que cierran se marca en los tres paquetes de capa, y el paquete que concentra dependencias salientes aparece en ámbar en la columna lateral.

Los tres escenarios aislados constituyen la principal diferencia respecto del diseño experimental inicial, que contemplaba una única versión alterada. La razón es metodológica: con las tres derivas juntas es imposible saber cuánto aporta cada regla al descenso de la CR, ni si el mecanismo sería capaz de advertir una desviación que llegara sola, que es precisamente el escenario que una PyME encontraría en la práctica.

3.15. Usos y escenarios de aplicación

El prototipo pone el proceso al alcance de un equipo por dos vías equivalentes: la línea de comandos, adecuada para automatizar el análisis y registrar su tendencia, y la interfaz gráfica, adecuada para explorar un sistema, discutir sus hallazgos y obtener un primer diagnóstico sin preparar nada. Ambas ofrecen las mismas capacidades: la extracción automática del grafo, la verificación de las tres reglas, la agregación en una CR reconstruible y un reporte que asocia cada hallazgo a su regla, su evidencia y su severidad, y que declara qué quedó fuera del diagnóstico.

La principal barrera de adopción no es ejecutar el proceso sino completar su primer paso: las *Fitness Functions* las aporta el mecanismo, pero la arquitectura prevista solo la conoce el equipo, y traducirla a prefijos de paquete y a una matriz de dependencias exige un esfuerzo inicial. Para reducirlo, el prototipo incluye un asistente opcional que, con un modelo de lenguaje —de Anthropic, OpenAI o DeepSeek, o uno que se ejecute en el propio equipo con Ollama o LM Studio—, propone

el catálogo a partir de la estructura del proyecto. El asistente no crea ni modifica reglas: solo traduce la intención del equipo al formato del catálogo, justificando cada decisión, preguntando lo que no puede saber y señalando como deriva probable, en lugar de permitirla, cualquier dependencia actual que contradiga la estratificación. Al modelo solo se le envía el perfil estructural —paquetes, número de clases, algunos nombres de clase y dependencias—, nunca el código fuente; mientras la persona no elija expresamente un servicio externo, el asistente usa un modelo local, de modo que por defecto ningún dato sale del equipo. La propuesta se valida con el mismo cargador que un catálogo escrito a mano y se previsualiza sobre el código antes de que la persona decida adoptarla. Se trata de una ayuda a la adopción, ajena al proceso evaluado: el análisis no distingue un catálogo escrito a mano de uno propuesto con ayuda. Una exploración con modelos reales, remotos y locales, documentada en el anexo B, mostró que las propuestas de los modelos remotos declaran la estratificación prevista sin legitimar las derivas de capas ni los ciclos, pero que el umbral de concentración es inestable y puede llegar a ocultar la deriva de concentración, y que el modelo local es menos fiable: oculta la concentración declarando exento al paquete que concentra y llegó a legitimar una deriva de capas. Por eso la herramienta comprueba cada propuesta con reglas deterministas y advierte cuando el umbral deja sin señalar al paquete que más concentra o cuando una exención recae sobre él.

En cuanto a los usos, el proceso se presta a la auditoría puntual de conformidad, a la señal temprana por iteración —si se ejecuta al cierre de cada ciclo, la desviación se advierte cuando todavía es barata de revertir (Klein et al., 2022)—, a la priorización de la refactorización por severidad y dimensión (Hamed et al., 2023) y al contraste de los *Architecture Decision Records* con lo efectivamente construido (AOE Technology Radar, 2026). Está pensado para el perfil que motivó su diseño: una PyME que desarrolla en Java, con una arquitectura en capas y sin documentación arquitectónica actualizada. Es más adecuado para preservar una arquitectura que todavía se respeta que para diagnosticar una que ya se perdió: un sistema legado produce en su primera ejecución un volumen de hallazgos difícil de accionar (Anthony et al., 2024), y atenderlo exige fijar el estado actual como línea de partida, una extensión que se plantea en el capítulo de conclusiones.

Con la misma claridad conviene delimitar lo que el prototipo no hace, en estricta correspondencia con el alcance aprobado. No corrige las derivas que detecta: diagnostica, clasifica y aporta evidencia para la decisión humana. No está integrado como producto de producción ni como etapa de una tubería CI/CD organizacional; la opción de terminar con un código de salida condicionado al veredicto es una afordancia para una adopción futura. No soporta varios ecosistemas en una misma ejecución ni realiza análisis dinámico (Modawski and Wolniak, 2025).

3.16. Trazabilidad entre requisitos e implementación

La tabla 3.8 vincula cada requisito con el componente que lo materializa y con la evidencia que permite comprobar su satisfacción.

Tabla 3.8: Trazabilidad entre requisitos, implementación y evidencia de cumplimiento.

Req.	Componente que lo materializa	Evidencia de cumplimiento
RF1	`ConstructorGrafo`, `GrafoDependencias`	Reglas de extracción verificadas sobre código compilado (tabla 3.4); conteo de paquetes y dependencias en cada reporte.
RF2	`ReglaIntegridadCapas`	Detección de la deriva en C1 y en C4; ausencia de hallazgos en C0, C2 y C3.
RF3	`ReglaCiclos`, `Tarjan`, `BuscadorCiclos`	Detección del ciclo en C2 y en C4, con la ruta cíclica concreta en la evidencia.
RF4	`ReglaConcentracion`	Detección del componente dios en C3 y en C4, con los valores de Ce y Ca.
RF5	`MotorConformidad`, `GeneradorReporte`	Tabla comparativa de los cinco escenarios; reportes legibles con CR y estado.
RF6	`Violacion`, evidencia por arista, sección de alcance del reporte	Hallazgos con método y línea de origen; declaración de reglas, umbrales y exclusiones en todo reporte.
RF7	`DisposicionGrafo`, `ExportadorSvg`, `ExportadorTikz`	Figuras 3.4, 3.5 y 3.6, generadas por el mecanismo.
RF8	`VentanaPrincipal`, `ResolutorProyecto`, `Analisis`	Figura 3.3; pruebas de paridad entre la carpeta del proyecto, el directorio de clases y el `.jar`.
RNF1	`CargadorConfiguracion`, `Exenciones`	El caso de estudio, el autoanálisis y los sistemas externos se analizan con el mismo binario y catálogos distintos; el reporte enumera las exenciones.
RNF2	Ausencia de dependencias más allá de *ArchUnit*	Compilación y ejecución con un JDK, sin Maven ni red; batería de pruebas sin marco externo.
RNF3	`GeneradorReporte`, interfaz gráfica	Conformidad local por regla, evidencia por hallazgo, declaración de alcance y guía de lectura (tabla 3.7).
RNF4	Exportadores JSON, CSV, SVG, TikZ y DOT; regla R6	Ejecuciones repetidas con resultados idénticos byte a byte.

3.17. Resumen del capítulo

Este capítulo describió el proceso de detección de deriva arquitectónica y su instrumentación. La deriva se obtiene comparando dos representaciones de un mismo sistema —la arquitectura prevista, que la persona declara en un catálogo, y la implementada, que se extrae de su código compilado como un grafo de dependencias entre paquetes con reglas explícitas— mediante tres *Fitness Functions* seleccionadas a partir de la literatura sobre verificación de conformidad y *smells* arquitectónicos: una prescriptiva, la integridad de capas, y dos intrínsecas, la ausencia de ciclos y la concentración acotada. Cada regla produce verificaciones atómicas y hallazgos con su evidencia; la

Tasa de Conformidad los agrega como proporción de verificaciones superadas, la severidad corrige su dilución y la tendencia permite leerla en el tiempo; y una guía de interpretación traduce cada resultado en decisiones posibles. El proceso se instrumentó en un prototipo de cinco módulos, uno por paso, verificado mediante 272 comprobaciones y mediante su aplicación sobre sí mismo. El capítulo siguiente evalúa el proceso sobre los escenarios del caso de estudio.

Capítulo 4

Evaluación

Este capítulo presenta la evaluación del mecanismo, que constituye la quinta fase de la metodología: la ejecución del experimento controlado y el análisis de sus resultados. Se describe primero el diseño del experimento y la matriz de validación que fija de antemano lo que debía observarse; se exponen después los resultados obtenidos sobre los cinco escenarios; se analiza a continuación un hallazgo que obligó a refinar el criterio de clasificación y se contrastan los resultados con los indicadores comprometidos en la metodología. La segunda mitad del capítulo somete esos resultados a pruebas que el caso de estudio no puede ofrecer: la aplicación a sistemas reales con derivas inyectadas, la observación de deriva real a lo largo de versiones publicadas, la calibración estadística del umbral, el contraste de la extracción con una herramienta independiente y la comparación con la verificación escrita directamente en *ArchUnit*. El capítulo cierra contrastando la hipótesis de trabajo componente por componente y examinando las amenazas a la validez.

4.1. Diseño de la evaluación

La evaluación se planteó como un experimento controlado de caso único, la modalidad que la investigación en diseño reserva para demostrar la viabilidad de un artefacto antes de someterlo a validación en campo (Hevner et al., 2004; Peffers et al., 2007). Su propósito fue determinar si el mecanismo construido detecta las derivas arquitectónicas que se le presentan y si clasifica de forma coherente el estado de conformidad del sistema evaluado.

La hipótesis operativa del experimento, que traduce a observables los dos primeros componentes de la hipótesis de trabajo (sección 1.2.3), fue que, al aplicar el mecanismo sobre una implementación de referencia y sobre versiones deliberadamente alteradas, la primera obtendría una conformidad plena sin hallazgos, mientras que cada versión alterada activaría la *Fitness Function* correspondiente a la deriva inducida, vería descender su Tasa de Conformidad y recibiría un estado distinto de *aceptable*. La hipótesis completa, que incluye además la utilidad de la evidencia para decidir, se contrasta al final del capítulo (sección 4.12).

Como variable independiente se manipuló la presencia y el tipo de deriva en el código, según los cinco escenarios descritos en el capítulo anterior. Las variables dependientes observadas fueron cuatro: los hallazgos reportados por cada *Fitness Function*, la Tasa de Conformidad resultante, el estado arquitectónico asignado y la evidencia estructural asociada a cada hallazgo. Todas las demás condiciones se mantuvieron constantes —el catálogo de reglas de la sección 3.2, los umbrales, la versión del prototipo y el entorno de ejecución—, de modo que las diferencias observadas puedan atribuirse a la variable manipulada y no a variaciones del procedimiento (Wohlin et al., 2012).

La incorporación de los tres escenarios de deriva aislada (C1, C2 y C3) constituye la decisión de diseño más relevante de esta evaluación. Un único escenario que acumulase las tres derivas permitiría comprobar que el mecanismo reacciona, pero no cuánto aporta cada regla a esa reacción ni si el instrumento advertiría una desviación que llegara sola, que es el caso que una PyME encontraría en la práctica. Aislar las derivas convierte el experimento en una prueba de sensibilidad por dimensión y no en una simple comprobación de funcionamiento.

4.1.1. Matriz de validación

Antes de ejecutar el experimento se fijó la matriz de validación de la tabla 4.1, que declara para cada escenario qué reglas debían activarse y cuáles debían permanecer en silencio. Establecerla de antemano cumple una función metodológica precisa: impide que el resultado observado se reinterprete después como el esperado, y convierte los falsos negativos y los falsos positivos en desviaciones verificables y no en apreciaciones.

Tabla 4.1: Matriz de validación: comportamiento esperado de cada *Fitness Function* por escenario.

Escenario	**FF1**	**FF2**	**FF3**	**Justificación del comportamiento esperado**
C0 · base	—	—	—	La implementación respeta la matriz de adyacencias, es acíclica y ningún paquete supera el umbral.
C1 · capas	✓	—	—	El salto `presentacion` → `persistencia` no está autorizado, pero no invierte el flujo ni cierra ciclos.
C2 · ciclo	—	✓	—	El ciclo queda contenido dentro de la capa de negocio, de modo que la estratificación no se vulnera.
C3 · concentración	—	—	✓	El paquete `util` no pertenece a ninguna capa y no es destino de ninguna dependencia.
C4 · combinado	✓	✓	✓	Las tres derivas actúan simultáneamente sobre el mismo sistema.

4.1.2. Criterios de éxito y procedimiento

Para juzgar el objetivo específico OBJ4 se estableció un criterio cuantitativo fijado en el anteproyecto: el mecanismo debía detectar al menos el 80 % de las derivas inducidas. Con seis derivas repartidas entre los cuatro escenarios alterados, el criterio exigía detectar al menos cinco. Se añadió una segunda condición, de signo contrario y de igual importancia: la implementación de referencia no debía producir ningún hallazgo, pues un instrumento que alarma sin motivo resulta tan inútil como uno que calla ante el problema.

El procedimiento se ejecutó íntegramente en un entorno de desarrollo local, sin dependencias de servicios externos, en coherencia con RNF2. Cada ejecución recibió los mismos dos insumos —el directorio de clases compiladas y el archivo de configuración— y se automatizó en un guion único que recorre los cinco escenarios y deposita la evidencia en archivos verificables: el reporte legible, la evidencia estructurada en JSON, el grafo en formato DOT y los hallazgos tabulados en CSV. Las ejecuciones se repitieron para comprobar que el resultado fuese idéntico, verificando

así el requisito de reproducibilidad. El anexo A detalla los pasos necesarios para reproducir el experimento completo.

4.2. Resultados de la evaluación

La tabla 4.2 resume el resultado agregado sobre los cinco escenarios, con los valores de la Tasa de Conformidad calculados según la ecuación 3.1 y el estado arquitectónico deducido de sus umbrales.

Tabla 4.2: Resultados agregados del mecanismo sobre los cinco escenarios.

Escenario	Paq.	Dep.	Verif.	Incum.	CR	CR por regla	Estado según CR
C0 · base	4	5	10	0	100,00 %	100,00 %	Aceptable
C1 · capas	4	6	11	1	90,91 %	66,67 %	Aceptable
C2 · ciclo	5	8	12	2	83,33 %	66,67 %	En observación
C3 · concentración	5	9	12	1	91,67 %	66,67 %	Aceptable
C4 · combinado	5	11	14	6	57,14 %	0,00 %	No conforme

4.2.1. Contraste con la matriz de validación

La tabla 4.3 contrasta el comportamiento esperado con el efectivamente observado. La coincidencia es completa: en los cinco escenarios se activaron exactamente las reglas previstas, sin omisiones ni activaciones espurias.

Tabla 4.3: Contraste entre el comportamiento esperado y el observado. Cada celda indica el número de hallazgos reportados por la regla.

	Esperado			Observado				
Escenario	FF1	FF2	FF3	FF1	FF2	FF3	**Total**	**Veredicto**
C0 · base	—	—	—	0	0	0	0	Coincide
C1 · capas	✓	—	—	1	0	0	1	Coincide
C2 · ciclo	—	✓	—	0	1	0	1	Coincide
C3 · concentración	—	—	✓	0	0	1	1	Coincide
C4 · combinado	✓	✓	✓	2	1	1	4	Coincide

En C4, la *Fitness Function* de integridad de capas reportó dos hallazgos —las dependencias indebidas de `persistencia` hacia `negocio` y hacia `presentacion`— porque el escenario introduce dos aristas prohibidas y cada una constituye una violación independiente. La tabla 4.4 detalla los hallazgos de ese escenario.

Tabla 4.4: Hallazgos detectados en el escenario combinado (C4).

Fitness Function	**Hallazgo**	**Severidad**
Integridad de capas	Dependencia no permitida `persistencia` → `negocio`	Mayor
Integridad de capas	Dependencia no permitida `persistencia` → `presentacion`	Mayor
Ausencia de ciclos	Ciclo entre `presentacion`, `negocio` y `persistencia`	Crítica
Concentración	`com.caso.util` concentra cuatro dependencias salientes (umbral: 3)	Mayor

Conviene aclarar una asimetría que la tabla podría ocultar: C4 reporta cuatro hallazgos pero seis incumplimientos. La diferencia procede de FF2, que cuenta un incumplimiento por cada paquete atrapado en el ciclo —tres, en este caso— aunque los agrupe en un único hallazgo legible. La decisión es deliberada: la penalización debe crecer con la extensión del enredo, mientras que el reporte debe presentar el ciclo como el problema único que en realidad es.

4.2.2. Sensibilidad por dimensión

Los escenarios aislados permiten observar el comportamiento de cada regla por separado. La tabla 4.5 muestra la conformidad local de cada *Fitness Function*, es decir, la proporción de sus propias verificaciones que el sistema superó.

Tabla 4.5: Conformidad local por *Fitness Function* en cada escenario (verificaciones superadas sobre verificaciones realizadas).

Escenario	**FF1 — Capas**	**FF2 — Ciclos**	**FF3 — Concentración**
C0 · base	2/2 (100 %)	4/4 (100 %)	4/4 (100 %)
C1 · capas	2/3 (66,7 %)	4/4 (100 %)	4/4 (100 %)
C2 · ciclo	2/2 (100 %)	3/5 (60,0 %)	5/5 (100 %)
C3 · concentración	2/2 (100 %)	5/5 (100 %)	4/5 (80,0 %)
C4 · combinado	2/4 (50,0 %)	2/5 (40,0 %)	4/5 (80,0 %)

La lectura por filas confirma el aislamiento buscado: en cada escenario de deriva única solo desciende la conformidad de la regla correspondiente, mientras las otras dos permanecen en el 100 %. La lectura por columnas revela algo más sutil: la conformidad local reacciona con claridad —FF2 cae al 60 % en C2, FF1 al 66,7 % en C1—, mucho más de lo que reacciona la CR global de esos mismos escenarios. Esa discrepancia entre la señal local y la global es la que motiva el análisis de la sección siguiente.

4.2.3. Trazabilidad del cálculo

El valor del 57,14 % obtenido en C4 no es una puntuación opaca: corresponde al cociente entre las ocho verificaciones que el sistema superó y las catorce que las tres reglas ejecutaron. Cada incumplimiento resta directamente a ese cociente, de modo que la cifra final puede reconstruirse a

mano a partir de la tabla 4.5. Esta propiedad de trazabilidad es, en sí misma, un resultado relevante: un equipo de desarrollo puede explicar con precisión por qué su proyecto obtuvo una determinada tasa, lo que convierte el diagnóstico en un insumo negociable y no en un veredicto inapelable.

La evidencia acompaña a cada hallazgo hasta su origen. En C1, por ejemplo, el reporte no se limita a señalar la arista infractora: identifica el campo, el constructor y el método del controlador que la producen, con el archivo y la línea correspondientes. Esa precisión, incorporada durante la evaluación como requisito RF6, transforma el diagnóstico en una lista de intervenciones concretas.

4.2.4. Interpretación de los resultados por escenario

Las cifras anteriores adquieren sentido al leerlas como lo haría el equipo responsable del sistema, siguiendo la guía de la sección 3.7.

C0 · línea base. Conformidad plena y ningún hallazgo: el código respeta la estratificación declarada y no presenta ciclos ni concentraciones. Para el equipo, el diagnóstico no exige ninguna acción, pero tiene un valor que conviene no pasar por alto: fija el punto de partida contra el que se medirá la tendencia de los análisis siguientes. Que la línea base no produzca hallazgos es, además, la condición para que el equipo confíe en los que aparezcan después.

C1 · ruptura de capas. Un único hallazgo de FF1: la presentación depende de la persistencia, saltándose el negocio. La evidencia lo localiza en el controlador —un campo del tipo del repositorio, su construcción en el constructor y una llamada a su método `guardar`, con la línea correspondiente—, de modo que la intervención es inmediata: llevar ese acceso al servicio de negocio. Cabe también la otra salida que la guía prevé: si el equipo decidiera que la presentación puede consultar la persistencia directamente, lo correcto sería declarar esa adyacencia en el catálogo y registrar la decisión, no ignorar el hallazgo. Lo instructivo del escenario es el contraste entre señales: la conformidad local de FF1 cae al 66,7 %, pero la CR global queda en el 90,91 %, por encima del umbral de aceptación. Leída sola, la cifra diría que no hay nada que atender.

C2 · ciclo. Un hallazgo de FF2 que agrupa dos paquetes: el servicio de negocio y su subpaquete de cálculo se necesitan mutuamente, de modo que ninguno puede probarse ni modificarse sin el otro. La ruta del ciclo indica dónde romperlo —lo habitual es que el subpaquete defina la abstracción que necesita y el servicio la implemente—, y si ambos fueran en realidad una sola unidad, la decisión correcta sería fusionarlos de forma explícita. Es el hallazgo de mayor severidad del catálogo, y aun así la CR solo lo clasifica *en observación*.

C3 · concentración. El paquete `util` depende de los cuatro paquetes restantes, por encima del umbral de tres. Es una utilidad que conoce a todo el sistema, de modo que cualquier cambio en cualquier capa puede afectarla: la intervención consiste en devolver cada función a la capa a la que pertenece. Como en C1, la CR del 91,67 % no delata el problema; la conformidad local de FF3, del 80 %, sí.

C4 · combinado. Cuatro hallazgos y una CR del 57,14 %, *no conforme* por cualquiera de los dos criterios. La guía ordena atender primero el ciclo, que es crítico, y la evidencia permite advertir algo que la lista de hallazgos no dice por sí sola: el ciclo lo cierran precisamente las dos dependencias que la persistencia dirige hacia el negocio y la presentación. Si se eliminan esas dos aristas, el grafo deja de tener ciclos: corregir las dos violaciones de capas resuelve tres de los cuatro hallazgos. La concentración de `util` es independiente y debe tratarse por separado. Los hallazgos, por tanto, no son una lista de tareas independientes, y leer su evidencia en conjunto permite encontrar la causa común.

Las cinco lecturas comparten una conclusión: la regla que se activa y su evidencia indican qué hacer, mientras que la CR global, en los escenarios de una sola deriva, puede indicar que no hay nada que hacer. Es la discrepancia que se analiza a continuación.

4.3. Un hallazgo sobre la métrica: la dilución de las violaciones localizadas

Los escenarios aislados produjeron un resultado que el diseño experimental original no habría revelado. En C1 y en C3 el mecanismo detectó correctamente la deriva inducida —FF1 y FF3 reportaron su hallazgo con la evidencia correspondiente—, pero la Tasa de Conformidad se mantuvo en 90,91 % y 91,67 % respectivamente, por encima del umbral del 85 %. En consecuencia, el estado arquitectónico asignado fue *aceptable* en ambos casos, pese a existir una violación real y documentada.

No se trata de un defecto de implementación sino de una consecuencia matemática de la formulación proporcional anticipada en la sección 3.6: una violación aislada representa una fracción pequeña del total de verificaciones, y esa fracción se reduce conforme el sistema crece. En los escenarios evaluados, con once o doce verificaciones, un incumplimiento apenas mueve la métrica nueve puntos. En un sistema de doscientos paquetes, el mismo incumplimiento la movería menos de un punto. La CR mide con fidelidad la **extensión** del deterioro, pero es estructuralmente ciega a su **gravedad**.

La consecuencia práctica es seria para el propósito del proyecto. Un instrumento pensado como señal temprana debería alertar precisamente cuando la primera desviación aparece, que es cuando corregirla resulta barata; sin embargo, el criterio basado solo en la CR guarda silencio hasta que las desviaciones se acumulan. El riesgo se agrava porque el veredicto *aceptable* podría interpretarse como ausencia de problemas, y no como lo que realmente significa.

Merece subrayarse que este riesgo no era desconocido: el anteproyecto lo había anticipado al identificar la sobreinterpretación de la CR como uno de los riesgos éticos del proyecto y al advertir que la métrica «no debe reemplazar el juicio técnico». Lo que la evaluación aporta no es la intuición del riesgo, sino la medida de su magnitud y una respuesta verificable.

4.3.1. La formulación del anteproyecto: la CR por regla

La dilución procede de haber tomado la verificación, y no la regla, como unidad de conteo (sección 3.6). Cabe entonces preguntarse si la formulación literal del anteproyecto —reglas cumplidas sobre reglas evaluadas, ecuación 3.2— habría evitado el problema. La tabla 4.2 lo responde: la CR por regla no se diluye, pero tampoco gradúa. Cualquier escenario con una sola deriva queda en 66,67 %, por debajo del umbral de observación y, por tanto, *no conforme*, sea la deriva una arista de capas o un ciclo; el combinado cae a 0 %. Con tres reglas, la métrica solo puede tomar cuatro valores, y los umbrales del 85 % y el 70 % dejan de discriminar: todo sistema con algún hallazgo es no conforme.

Los sistemas reales de las secciones siguientes muestran el costo de esa rigidez con más claridad que el caso de estudio. La CR por regla de Jackson Databind vale 0 % en las trece versiones analizadas, mientras la CR por verificación desciende del 65,38 % al 57,38 % a medida que aparece deriva real (sección 4.8): la primera no puede mostrar la tendencia que la segunda sí registra. En DDD Sample, con ocho hallazgos y ciento una verificaciones, la CR por regla es también del 0 % y la CR por verificación del 89,11 %; ninguna de las dos, por sí sola, describe el sistema. La conclusión es que ambas lecturas responden a preguntas distintas —qué reglas se incumplen y qué parte del sistema las incumple— y que ninguna resuelve la gravedad. Por eso se conservaron las dos en el reporte y se resolvió la gravedad con un criterio propio, que es el refinamiento que se describe a continuación.

4.4. Refinamiento del artefacto

La investigación en diseño prescribe que los resultados de la evaluación retroalimenten el artefacto en un nuevo ciclo, en lugar de limitarse a describir sus carencias (Hevner et al., 2004; Peffers et al., 2007). Siguiendo ese principio se introdujo un segundo criterio de clasificación, complementario y no sustitutivo del primero.

El razonamiento es el siguiente. La Tasa de Conformidad responde a la pregunta «¿qué proporción del sistema incumple?»; la severidad del peor hallazgo responde a «¿qué tan grave es lo que incumple?». Ambas preguntas son legítimas y ninguna subsume a la otra: un sistema puede tener muchas violaciones leves o una sola crítica, y esos dos estados requieren respuestas distintas. El refinamiento consiste en evaluar las dos y adoptar como veredicto el más restrictivo, según la correspondencia siguiente:

- una violación **crítica** —un ciclo de dependencia— exige el estado *no conforme*;
- una violación **mayor** —una ruptura de capas o una concentración excesiva— exige, al menos, el estado *en observación*;
- la ausencia de hallazgos permite el estado *aceptable*.

Dos precisiones delimitan el alcance de este cambio. La primera es que **la Tasa de Conformidad no se modifica**: se sigue calculando igual, se sigue interpretando con los umbrales del 85 %

y el 70 %, y se sigue reportando siempre, de modo que la formulación aprobada en el anteproyecto permanece intacta y verificable. La segunda es que el criterio es **configurable**: el modo `cr` reproduce el comportamiento original y el modo `cr+severidad` aplica el refinamiento, lo que permite comparar ambos sobre la misma evidencia en lugar de sustituir uno por otro sin posibilidad de contraste. El reporte, además, advierte de forma explícita cuando la severidad ha endurecido el veredicto que la CR habría emitido por sí sola.

La tabla 4.6 muestra el resultado de reejecutar el experimento completo bajo ambos criterios.

Tabla 4.6: Clasificación de los cinco escenarios bajo ambos criterios. La columna «esperado» recoge el estado que el escenario merece según las derivas efectivamente inducidas.

Escenario	CR	Modo `cr`	Modo `cr+severidad`	Esperado
C0 · base	100,00 %	Aceptable	Aceptable	Aceptable
C1 · capas	90,91 %	Aceptable	En observación	En observación o peor
C2 · ciclo	83,33 %	En observación	No conforme	No conforme
C3 · concentración	91,67 %	Aceptable	En observación	En observación o peor
C4 · combinado	57,14 %	No conforme	No conforme	No conforme

El criterio original clasifica correctamente tres de los cinco escenarios; el refinado, los cinco. La mejora se concentra, como era de esperar, en los escenarios de deriva aislada, que son justamente aquellos en los que un mecanismo de detección temprana debe demostrar su valor. Obsérvese, además, que el refinamiento solo puede endurecer el veredicto y nunca relajarlo, propiedad que la batería de pruebas verifica de forma explícita: incorporar la severidad no puede hacer que un sistema problemático parezca sano.

Queda pendiente una discusión que el refinamiento abre y no cierra. Que un único ciclo baste para declarar un sistema *no conforme* es una posición defendible —los ciclos son la estructura más dañina de las tres— pero también exigente para un sistema legado que arrastre varios. Por esa razón el criterio se dejó configurable y no impuesto: cada organización debe decidir cuánta severidad tolera, del mismo modo que decide sus umbrales. Lo que el mecanismo garantiza es que esa decisión quede declarada y sea auditable, no que sea la misma para todos.

4.5. Contraste con los indicadores de la metodología

La metodología aprobada en el anteproyecto definió cuatro indicadores para juzgar el desempeño del mecanismo. Contrastar los resultados con cada uno de ellos es lo que convierte un experimento en una evaluación con criterios explícitos, en lugar de una demostración de que el prototipo «funciona». La tabla 4.7 resume esa correspondencia.

Tabla 4.7: Contraste entre los indicadores metodológicos y los resultados observados.

Indicador	Definición	Resultado observado
Cobertura de detección	Proporción de derivas inducidas que el mecanismo identifica.	Detectó las seis derivas repartidas entre los cuatro escenarios alterados: cobertura del 100 %, superior al criterio de éxito del 80 % fijado para OBJ4.
Falsos positivos en la línea base	Verificación de que el mecanismo no marque como anómalo un estado estructural esperado.	C0 obtuvo conformidad plena y cero hallazgos. Ninguna regla se activó fuera de los escenarios previstos por la matriz de validación, de modo que tampoco hubo falsos positivos en los escenarios alterados.
Variación de la Tasa de Conformidad	Comprobación de que la métrica sea sensible a la degradación introducida.	La CR descendió en los cuatro escenarios alterados, de forma monótona respecto del número de incumplimientos (100 % → 91,67 % → 90,91 % → 83,33 % → 57,14 %). Es sensible, pero su magnitud resultó insuficiente para clasificar correctamente las derivas aisladas, lo que motivó el refinamiento de la sección anterior.
Capacidad explicativa del reporte	Presencia de la regla violada, su ubicación y evidencia estructural suficiente para interpretación humana.	Cada hallazgo asocia la regla, los paquetes implicados, la severidad y la evidencia hasta el método y la línea que originan la dependencia; los ciclos incluyen su ruta concreta. El reporte declara además reglas, umbrales, paquetes sin capa y exclusiones del diagnóstico.

Este contraste cumple una doble función. Por un lado, demuestra que el experimento se juzgó contra los mismos criterios que el anteproyecto había comprometido, sin introducir criterios a posteriori. Por otro, muestra que el trabajo no se limitó a confirmar lo que ya se esperaba: el tercer indicador reveló una limitación real de la métrica, y esa limitación se atendió dentro del alcance aprobado en lugar de posponerse como trabajo futuro.

4.5.1. Criterios de cierre

El anteproyecto fijó, además, cuatro criterios para considerar que la metodología había producido resultados suficientes. Los cuatro se cumplen. Existe un **catálogo acotado de reglas ejecutables**: las tres *Fitness Functions* de la tabla 3.3, que el prototipo imprime a partir de su propia implementación. Hay un **prototipo funcional sobre un único ecosistema**, Java, cuya verificación se describe en la sección 3.13. Se generan **reportes de conformidad a partir del código** de cada escenario, en formato legible y en formatos verificables por otras herramientas. Y hay **evidencia de que el mecanismo identifica las derivas introducidas** en un entorno controlado: las seis derivas inducidas, sin hallazgos sobre la línea base.

4.6. Aplicación a un sistema externo

Todo lo anterior se obtuvo sobre escenarios construidos para el experimento. Cabe entonces una pregunta que el diseño controlado no puede responder por sí mismo: ¿qué ocurre cuando el mecanismo se enfrenta a código que nadie preparó para él? Para responderla se aplicó el instrumento sobre *ArchUnit* en su versión 1.3.0, una biblioteca de uso extendido cuyo código no fue escrito por el autor ni con este trabajo en mente. El análisis se realizó directamente sobre el archivo `.jar` con que la biblioteca se distribuye, sin extraerlo, y las configuraciones empleadas se conservan en el repositorio para que el ejercicio pueda reproducirse.

Conviene delimitar el estatus de este ejercicio antes de exponer sus resultados. **No constituye un segundo caso de estudio ni forma parte del experimento controlado**, cuyo alcance —un único sistema— permanece tal como el anteproyecto lo aprobó. Es una demostración de aplicabilidad, del mismo tipo que el autoanálisis del capítulo anterior: muestra que el mecanismo opera sobre código real y a qué se enfrenta al hacerlo, sin pretender que de ello se siga una validación externa.

Una segunda precisión es de justicia hacia el sistema analizado. La estratificación evaluada —`base` ← `core` ← `lang` ← `library`— es una **hipótesis** formulada a partir de los nombres de sus módulos, no una arquitectura que el proyecto declare. Cualquier desviación que se reporte lo es respecto de esa suposición, y no necesariamente un defecto de *ArchUnit*.

El sistema aporta 923 clases propias repartidas en 22 paquetes. El análisis completo tarda **1,2 segundos**, magnitud que lo hace compatible con una ejecución en cada integración. La tabla 4.8 contrasta el diagnóstico obtenido sin exenciones con el obtenido tras declararlas.

Tabla 4.8: Diagnóstico de *ArchUnit* 1.3.0 antes y después de declarar las exenciones.

	Sin exenciones	**Con exenciones**
Paquetes en el grafo	43	22
Dependencias extraídas	222	107
FF1 — Integridad de capas	61/61	61/61
FF2 — Ciclos (incumplimientos)	12	9
tamaño del mayor ciclo	7 paquetes	4 paquetes
FF3 — Concentración (hallazgos)	14	5
Tasa de Conformidad	82,31 %	86,41 %
Estado según CR	En observación	Aceptable
Estado final (con severidad)	No conforme	No conforme

De estos números se desprenden tres lecturas, y las tres importan por razones distintas.

La regla de capas resultó exacta. FF1 realizó 61 verificaciones y las superó todas, en ambas configuraciones y sin un solo falso positivo. Dos conclusiones se siguen de ello: que la hipótesis sobre la arquitectura de *ArchUnit* era correcta —su estratificación se respeta de forma estricta— y que

la regla es capaz de validar una arquitectura real sin ruido. De las tres dimensiones del catálogo, esta es la que se muestra lista para uso industrial.

Sin exenciones, casi la mitad del grafo era código ajeno. *ArchUnit* empaqueta Guava y ASM bajo su propio prefijo, de modo que la restricción a la raíz —pensada para dejar fuera a las bibliotecas de terceros— no los excluye. Peor aún, esos paquetes entran en el grafo incluso cuando sus clases no se importan, porque basta con que alguien las referencie para que aparezca la arista. El efecto es cuantificable: declarar las exenciones elimina **9 de los 14** hallazgos de concentración, es decir, cerca de dos tercios eran ruido, y reduce el ciclo mayor de siete paquetes a cuatro, porque el paquete de anotaciones al que todo el sistema referencia actuaba como puente entre módulos que entre sí no se alcanzan. La Tasa de Conformidad asciende en consecuencia 4,10 puntos porcentuales y cruza el umbral de aceptación.

Los hallazgos reales sobreviven. Los tres ciclos permanecen tras las exenciones, y el veredicto final sigue siendo *no conforme* por su severidad. Esta es, probablemente, la observación más importante del ejercicio: el mecanismo de exenciones retira ruido sin silenciar hallazgos, que es la única forma en que una excepción puede ser legítima. Si las exenciones hubieran hecho desaparecer también los ciclos, habría motivos para sospechar que el instrumento se había convertido en un complaciente.

La figura 4.1 muestra el grafo resultante tal como lo dibujó el mecanismo. Ofrece una lectura que los números no transmiten: las cuatro capas aparecen en el orden correcto aunque la configuración las declaró de abajo arriba, los tres ciclos se concentran en las capas `core` y `lang` —no en la de reglas predefinidas—, y el paquete `base`, del que dependen casi todos los demás, se reconoce de inmediato por su anchura. El dibujo es denso porque el sistema lo es: ciento siete dependencias entre veintidós paquetes no admiten una representación despejada, y la figura no lo disimula.

Queda por señalar qué *no* establece este ejercicio. Se trata de un solo sistema, analizado por el propio autor, bajo una arquitectura supuesta y sin contraste con el criterio de quienes lo desarrollaron. Sirve para mostrar que el mecanismo opera sobre código real, a qué velocidad y con qué clase de dificultades; no sustituye la validación externa sobre repositorios industriales que el capítulo de conclusiones plantea como línea prioritaria de trabajo.

4.7. Validación de aplicabilidad en entorno controlado sobre sistemas reales

El caso de estudio permite saber con exactitud qué derivas contiene cada escenario, pero es pequeño y fue construido para el experimento; la aplicación a *ArchUnit* enfrenta el mecanismo a código real, pero sin saber qué derivas contiene. La validación que se describe a continuación combina ambas virtudes: toma sistemas reales y les **inyecta derivas conocidas**, de modo que se sabe exactamente qué debería detectarse. Como la aplicación anterior, **no forma parte del experimento controlado de OBJ4**, cuyo alcance —un único sistema— se mantiene; es una

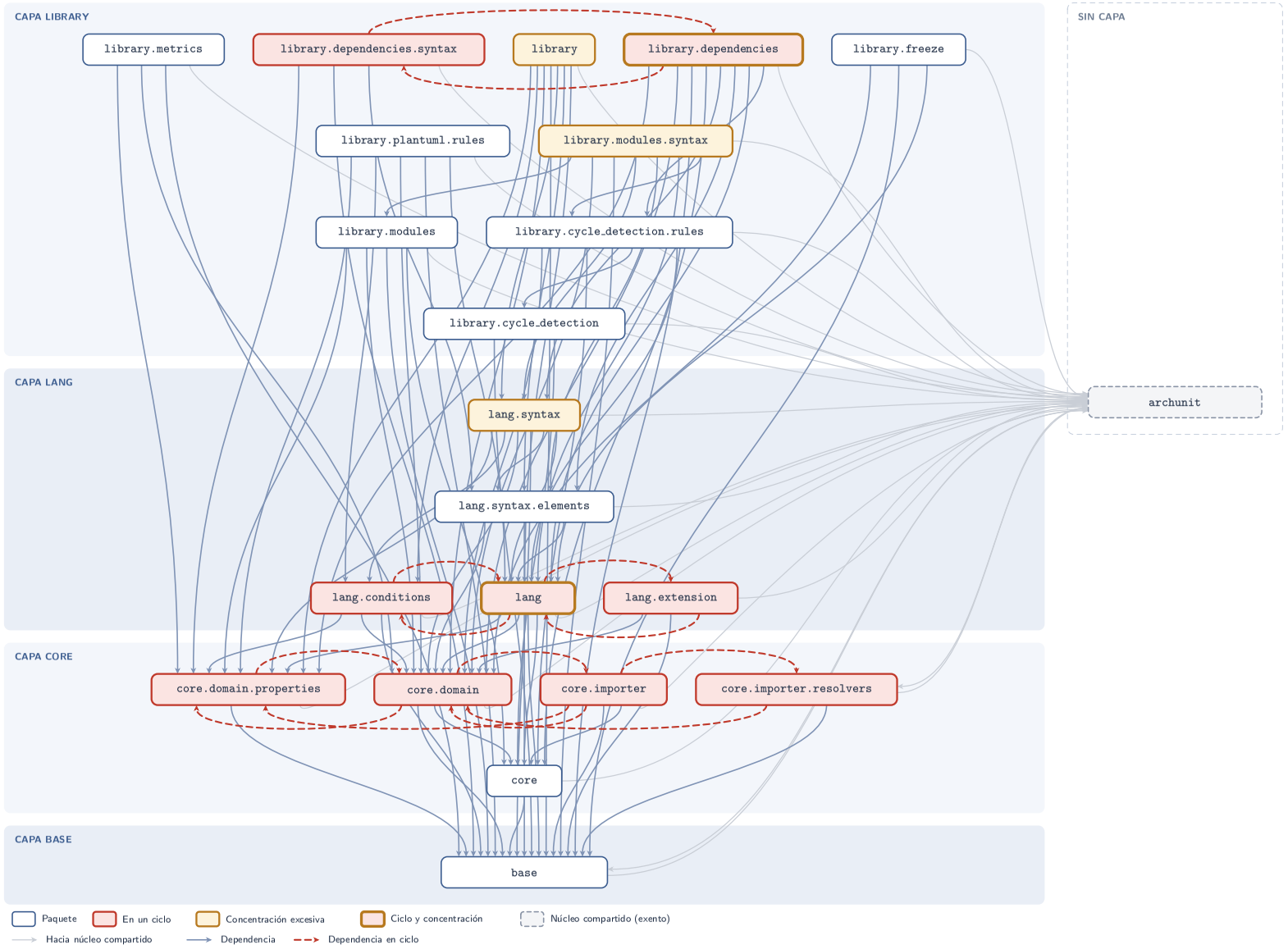


Figura 4.1: Grafo de dependencias de *ArchUnit* 1.3.0 con las exenciones declaradas, dibujado por el propio mecanismo a partir del archivo `.jar`. Las bandas siguen la estratificación hipotética `library` → `lang` → `core` → `base`; el núcleo de anotaciones exento ocupa la columna lateral y sus dependencias se dibujan atenuadas.

demostración complementaria de que lo observado en el caso de estudio se sostiene fuera de él.

Sistemas. Se emplearon siete sistemas Java de código abierto en dos grupos. El primero reúne cuatro bibliotecas de uso extendido y tamaño creciente, analizadas directamente sobre el `.jar` que publican en Maven Central y cuya integridad se verificó con la suma SHA-1 del repositorio: Apache Commons Lang 3.14.0, Jackson Databind 2.17.2, *ArchUnit* 1.3.0 y Guava 33.2.1. Ninguna declara formalmente su arquitectura: para *ArchUnit* se conservó la estratificación hipotética de la sección anterior, y para las otras tres se adoptó, sin modificarla, la propuesta del asistente de catálogo (sección 3.15), que reproduce la situación de una organización que incorpora la herramienta sin haber declarado nunca su arquitectura.

El segundo grupo responde a la objeción de que una biblioteca no es el perfil de una PyME. Lo forman tres aplicaciones en capas, fijadas en un *commit* concreto y compiladas a partir de su código: **Spring Framework PetClinic**, que su documentación describe como una arquitectura de tres capas —presentación, servicio y repositorio— sobre un modelo de dominio compartido; **MyBatis JPetStore 6**, «una aplicación MVC típica con tres capas: presentación, lógica y acceso a datos»; y **DDD Sample**, la aplicación de referencia de Domain Language y Citerus que materializa la arquitectura en capas del diseño guiado por el dominio —interfaces de usuario, aplicación, dominio e infraestructura— (Evans, 2003). Para estas tres el catálogo **no lo propuso el asistente**: transcribe la arquitectura que cada proyecto documenta, de modo que sus hallazgos sí pueden leerse como desviaciones respecto de una intención declarada por sus autores. Su umbral de concentración se fijó con el procedimiento de la sección 4.9, sin ajustarlo después.

Procedimiento. A cada sistema se le inyectaron derivas del mismo tipo que las de C1 a C4: una dependencia entre capas que la matriz no permite, una que cierra un ciclo, y un paquete nuevo con más dependencias salientes que el umbral, primero por separado y después las tres juntas. Cada deriva es una clase mínima, generada directamente en *bytecode* e incorporada al `.jar`, que declara un campo del tipo de una clase del paquete destino: añade exactamente una dependencia entre paquetes y nada más. Su ubicación se eligió de forma determinista para que activara solo la regla correspondiente —la de capas no cierra ningún ciclo ni excede el umbral; la de ciclo no viola las capas—. La única excepción es JPetStore, cuya estratificación es estricta: cada capa es un solo paquete y no hay dependencias dentro de una misma capa, de modo que toda arista que cierra un ciclo apunta necesariamente hacia arriba y viola también las capas. Allí la deriva de ciclo activa las dos reglas, y el hallazgo de capas que produce se cuenta como explicado por la inyección. Como cada sistema real trae sus propios hallazgos, lo que se mide es la **diferencia** entre la versión alterada y la original: si aparece el hallazgo de la deriva inyectada, si aparece alguno que no se inyectó y cuánto varía la CR. En total se analizaron 34 versiones, con 31 derivas inyectadas.

Tabla 4.9: Inyección controlada de derivas en siete sistemas reales. La variación de la CR corresponde a las versiones con una sola deriva.

Sistema	Clases	Paq.	CR orig.	Detectadas	No inyect.	ΔCR (pp)	Tiempo
Commons Lang 3.14.0*	403	18	82,35 %	4/4	0	−1,8 a −5,9	0,2 s
Jackson Databind 2.17.2	784	23	60,00 %	6/6	0	−0,3 a −3,3	0,3 s
ArchUnit 1.3.0	923	22	86,41 %	6/6	0	−0,7 a −1,9	0,6 s
Guava 33.2.1	2 017	18	100,00 %	6/6	0	−1,2 a −2,4	0,4 s
Spring Framework PetClinic	49	9	100,00 %	3/3	0	−3,9 a −8,3	< 0,1 s
MyBatis JPetStore 6	26	5	100,00 %	3/3	0	−5,9 a −18,8	< 0,1 s
DDD Sample	94	26	89,11 %	3/3	0	−0,8 a −2,0	< 0,1 s
Total				**31/31**	**0**		

* El catálogo propuesto no declara capas, de modo que la deriva de capas no aplica.

Los resultados de la tabla 4.9 permiten cuatro lecturas.

La detección se sostiene en sistemas reales. El mecanismo detectó las 31 derivas inyectadas y no produjo ningún hallazgo que no se hubiera inyectado: la cobertura de detección y la ausencia de falsos positivos observadas en el caso de estudio se reproducen sobre sistemas de hasta dos mil clases, en bibliotecas y en aplicaciones en capas, con un tiempo de análisis inferior a un segundo.

La dilución de la CR crece con el número de verificaciones. Una sola deriva hace descender la CR entre 0,3 y 5,9 puntos según el sistema. Lo que determina el descenso no es el número de clases sino el de verificaciones, porque la CR es una proporción sobre ellas: un ciclo resta 5,9 puntos en Commons Lang, con 34 verificaciones, y 1,9 en *ArchUnit*, con 103; Guava, con más clases que *ArchUnit* pero 83 verificaciones, queda entre ambos. Guava muestra además la consecuencia con nitidez: partiendo de una conformidad del 100 %, cualquiera de las derivas aisladas la deja por encima del 97 %, y la CR por sí sola seguiría clasificando el sistema como *aceptable*. El criterio combinado, en cambio, lo clasifica *en observación* ante una violación de capas o una concentración, y *no conforme* ante un ciclo. Es la limitación de la sección 4.3 reproducida en un sistema real y con unas siete veces más verificaciones que los escenarios del caso de estudio, y confirma que el refinamiento no era un ajuste al caso de estudio.

En un sistema muy acoplado, una deriva nueva puede no ser un hallazgo nuevo. Jackson Databind ya contiene un ciclo que abarca 19 de sus 23 paquetes. El ciclo inyectado no creó un componente nuevo: incorporó dos paquetes más a ese ciclo, que pasó a reunir 21. El mecanismo lo reportó correctamente, pero la novedad solo se advierte al comparar con el análisis anterior. Es un argumento empírico a favor de la lectura por tendencia de la sección 3.6 y de la línea base que el capítulo de conclusiones plantea como trabajo futuro.

Un catálogo deducido del código detecta la deriva futura, no la pasada. El catálogo que el asistente propuso para Guava declara quince capas cuya matriz reproduce las dependencias existentes, y por eso el sistema original obtiene una CR del 100 %: describe el estado actual en lugar de cuestionarlo. Aun así, detectó todas las derivas inyectadas después. Ese comportamiento es, en la práctica, el de una línea base, y delimita lo que puede esperarse de un catálogo propuesto sin intervención humana: protege contra la deriva que se introduzca a partir de su adopción, pero no revela la que ya existía. La propuesta para Guava, además, excluyó por su nombre un paquete propio de la biblioteca (`thirdparty.publicsuffix`), y la de Commons Lang activó la revisión determinista del umbral, dos recordatorios de que la propuesta debe revisarse antes de adoptarse.

Las aplicaciones en capas: lo que el mecanismo encuentra cuando la intención está escrita. PetClinic y JPetStore cumplen su arquitectura documentada sin un solo hallazgo, lo que es en sí un resultado: la regla de capas validó dos estratificaciones declaradas por sus autores sin falsos positivos. DDD Sample, en cambio, presenta ocho hallazgos reales, que nadie inyectó, y su lectura siguiendo la guía de la sección 3.7 ilustra el tipo de decisión que el reporte permite tomar:

- **Un tipo de la capa de interfaces usado por la de aplicación.** El servicio `ApplicationEvents` recibe como parámetro `HandlingEventRegistrationAttempt`, una clase definida en `interfaces.handling`. Es una inversión de dependencia en sentido estricto: la capa de aplicación conoce un tipo de la capa superior. El mismo tipo cierra, además, el ciclo crítico entre `interfaces.handling`, `interfaces.handling.ws` y `application`, de modo que **mover esa clase a la capa de aplicación resolvería a la vez un hallazgo de capas, el ciclo y el hallazgo de infraestructura** que también depende de ella. Es el mismo patrón de causa común que se observó en C4, ahora en un sistema real.
- **Un ciclo entre dos agregados del dominio.** Los paquetes `handling` y `cargo` del modelo de dominio se referencian mutuamente: un evento de manipulación conoce su carga, y la carga deriva su estado de los eventos. No viola ninguna capa —ambos paquetes pertenecen al dominio—, pero acopla dos agregados que el diseño guiado por el dominio procura mantener independientes. Es una decisión de diseño discutible más que un error, y el reporte la pone sobre la mesa con su evidencia en lugar de ocultarla.
- **Consumidores de mensajería que invocan servicios de aplicación.** Las clases de `infrastructure.messaging.jms` reciben mensajes y llaman a servicios de aplicación. Según la estratificación declarada, la infraestructura solo puede depender del dominio, y la llamada es una violación; pero esas clases actúan como adaptadores de entrada, un papel que en la arquitectura de Evans corresponde a la capa de interfaces. El hallazgo admite, por tanto, dos lecturas: deriva —los adaptadores están en el paquete equivocado— o un catálogo impreciso —la infraestructura de entrada debería poder invocar a la aplicación—. El mecanismo no puede decidir entre ellas, y no debe: es la discusión de diseño que el riesgo de formalismo vacío del anteproyecto pretendía provocar.

- **Dos concentraciones en la capa de interfaces**, la de su paquete raíz ($Ce = 10$) y la de la fachada interna de reservas ($Ce = 7$), por encima del umbral de 6 derivado de la propia distribución del sistema.

La lectura conjunta muestra que los ocho hallazgos se reducen a tres decisiones: reubicar un tipo, discutir el acoplamiento entre dos agregados y decidir dónde viven los adaptadores de entrada. Esa reducción es la que la CR, por sí sola, no ofrece: el sistema obtiene un 89,11 %, *aceptable* por la proporción, y *no conforme* por la severidad de sus dos ciclos.

Las limitaciones de esta validación son claras. Las aplicaciones en capas son aplicaciones de referencia y no sistemas en producción de una PyME; las derivas inyectadas son mínimas y sintéticas, aunque se insertan en código real; y los catálogos de las bibliotecas son hipótesis, no arquitecturas declaradas por quienes las mantienen. La validación muestra que el mecanismo detecta derivas conocidas en código real, a escala y sin ruido; no sustituye la validación externa con equipos industriales. La evidencia completa se conserva en `prototipo/ejemplos/sistemas/` y se reproduce con `arquitecta aplicaciones` seguido de `arquitecta sistemas`.

4.8. Deriva real a lo largo de la evolución

Toda deriva medida hasta aquí fue fabricada, en el caso de estudio o por inyección. Queda una objeción legítima: que el mecanismo detecte lo que se le pone delante no prueba que detecte la deriva que aparece sola, que es la que una PyME padece. Para responderla se aplicó el catálogo declarado para una versión de referencia —el mismo de la sección anterior, sin modificarlo— a las versiones publicadas antes y después de ella de tres bibliotecas: trece de Jackson Databind (de la 2.4.0 a la 2.22.0), doce de Guava (de la 18.0 a la 33.7.0) y diez de Commons Lang (de la 3.1 a la 3.21.0), treinta y cinco en total, todas descargadas de Maven Central y verificadas con su SHA-1. Las versiones posteriores a la referencia responden a la pregunta operativa del mecanismo: si el equipo hubiera declarado esa arquitectura en ese momento, ¿qué deriva habría aparecido después? Las anteriores muestran desde cuándo existía lo que hoy se señala. La tabla 4.10 resume el resultado.

Tabla 4.10: Evolución de la conformidad respecto del catálogo declarado para la versión de referencia. «Posteriores» resume los hallazgos que aparecieron después de ella.

Serie	Primera	Referencia	Última	Mayor ciclo	Posteriores a la referencia
Jackson Databind	65,38 %	60,00 %	57,38 %	16 → 19 paq.	2.19.0: `introspect` → `deser.impl` (capas) y concentración en `introspect` ($Ce = 13 > 12$)
Guava	100 %	100 %	100 %	sin ciclos	Ninguno en cinco versiones
Commons Lang	90,91 %	82,35 %	79,41 %	2 → 6 paq.	3.21.0: concentración en `builder` ($Ce = 6 > 5$)

El mecanismo detecta deriva que nadie inyectó. En Jackson Databind 2.19.0 aparece una dependencia de la introspección hacia la implementación de la deserialización: el método

`_addCreatorParams` de la clase `POJOPropertiesCollector` invoca a `creatorParamName`, de `UnwrappedPropertyHandler`. Es exactamente el tipo de atajo que describe la deriva —un componente de bajo nivel que pasa a conocer uno de alto nivel para resolver un caso particular—, y el mecanismo lo señala en la versión en que aparece, con el método que lo origina. En Commons Lang 3.21.0, el paquete `builder` incorpora una dependencia hacia `mutable` y supera el umbral. Ninguna de estas desviaciones se fabricó para el experimento.

La tendencia hace visible lo que el valor absoluto oculta. En Jackson Databind, la CR por verificación desciende del 65,38 % al 57,38 % a lo largo de las versiones, mientras el ciclo mayor crece de 16 a 19 paquetes; la CR por regla, en cambio, vale 0 % en todas y no registra ningún cambio (sección 4.3.1). En Commons Lang el ciclo pasa de dos a seis paquetes antes de la versión de referencia, y la CR desciende del 90,91 % al 82,35 %: la erosión fue gradual, como la describen las leyes de Lehman (Lehman, 1980), y la lectura por tendencia de la sección 3.6 la habría hecho visible versión a versión.

También detecta la ausencia de deriva. Guava conserva una conformidad del 100 % en las doce versiones, desde la 18.0 hasta la 33.7.0: su estratificación se mantuvo intacta mientras el número de clases crecía de 1 687 a más de 2 000. Un instrumento que solo encontrara problemas sería sospechoso; este distingue el sistema que se erosiona del que no.

Dos precisiones acotan estas lecturas. Los catálogos son hipótesis y no arquitecturas declaradas por los equipos, de modo que lo detectado es deriva respecto de ellos, no necesariamente un defecto reconocido por sus autores. Y se trata de tres series, no de una muestra representativa; lo que establecen es que el mecanismo detecta deriva real y no solo la inyectada, no con qué frecuencia la encontraría en otros sistemas. La evidencia se conserva en `prototipo/ejemplos/estudios/evolucion.md` y se reproduce con `arquitecta estudios evolucion`.

4.9. Calibración del umbral de concentración

FF3 es la única regla con un parámetro numérico, y un umbral es el parámetro más fácil de ajustar para dejar de ver una deriva: la evaluación del asistente lo mostró (anexo B). El umbral del caso de estudio, $Ce > 3$, se fijó con el catálogo y no tenía otro respaldo que el juicio del autor. Para sustituir el juicio por un procedimiento se aplicó el método de derivación de umbrales a partir de datos de referencia de Alves et al. (2010): se reunió la distribución del acoplamiento eferente de los paquetes que FF3 verifica en un **conjunto de calibración** —las cuatro bibliotecas, 78 paquetes— y se contrastó después en un **conjunto de validación** que no intervino en la calibración —las tres aplicaciones en capas y el caso de estudio—. La tabla 4.11 resume las distribuciones.

Tabla 4.11: Distribución del acoplamiento eferente de los paquetes verificados por FF3 (percentiles por rango más cercano).

Sistema	**Conjunto**	**Paq.**	**p50**	**p70**	**p80**	**p90**	**Máx.**
Commons Lang 3.14.0	calibración	17	2	2	2	4	5
Jackson Databind 2.17.2	calibración	22	6	10	11	12	14
ArchUnit 1.3.0	calibración	21	5	6	7	8	10
Guava 33.2.1	calibración	18	3	5	6	7	7
Conjunto de calibración		78	4	6	7	10	14
Spring Framework PetClinic	validación	8	2	2	2	3	3
MyBatis JPetStore 6	validación	5	1	2	2	2	2
DDD Sample	validación	24	4	4	5	6	10
Caso de estudio (C0)	validación	4	1	2	2	2	2

Un umbral absoluto común no sirve. El percentil 90 del conjunto de calibración es 10. Aplicado a las aplicaciones en capas, no señalaría nunca nada: su máximo observado es 2 o 3. Aplicado a Jackson Databind, señalaría seis paquetes. La razón es estructural: el acoplamiento eferente está acotado por el número de paquetes del sistema, de modo que un mismo valor absoluto significa cosas distintas en un sistema de cinco paquetes y en uno de veinticinco. Es la misma advertencia que Lanza and Marinescu (2006) hacen sobre trasladar umbrales entre contextos.

El procedimiento adoptado: el percentil 90 de la línea base. Lo que sí se traslada es el procedimiento. El umbral se fija en el percentil 90 del Ce de los paquetes no exentos de la versión que el equipo acepta como línea base, la frontera de riesgo «muy alto» de Alves et al. (2010). Con él, cada sistema señala solo los paquetes que destacan sobre su propia distribución —uno en Commons Lang y en *ArchUnit*, dos en Jackson Databind y en DDD Sample, ninguno en Guava ni en las aplicaciones más pequeñas—, y el catálogo congela esa distribución para señalar todo paquete que la exceda después. Las tres aplicaciones de la sección 4.7 usaron este procedimiento sin ajustes.

La clasificación es robusta al umbral. En el caso de estudio, el percentil 90 de C0 es 2, no 3. Se reejecutaron los cinco escenarios con umbral 2: C1 y C2 suman un hallazgo de concentración —la arista inyectada eleva también el Ce de su paquete de origen hasta 3—, pero **ninguno de los cinco veredictos cambia**. El experimento conservó el umbral de 3, fijado antes de ejecutarlo, y el resultado de OBJ4 no depende de esa elección. La observación revela, además, una propiedad que conviene declarar: en sistemas muy pequeños el percentil 90 coincide con el máximo, de modo que cualquier dependencia nueva del paquete más acoplado se señala, y una misma arista puede activar dos reglas. Es el comportamiento de línea base que se busca, con el costo de hallazgos solapados que el reporte presenta por separado.

La evidencia se conserva en `prototipo/ejemplos/estudios/umbral.md`, junto con la sensibilidad completa —cuántos paquetes señala cada umbral en cada sistema—, y se reproduce con

`arquitecta estudios umbral`.

4.10. Contraste con un oráculo independiente

Todos los resultados anteriores dependen de que el grafo extraído sea correcto, y hasta aquí esa corrección solo la verificaban pruebas escritas por el propio autor. Para obtener un contraste independiente se ejecutó JDepend (Clark, 2015), una herramienta que calcula las métricas de paquetes de Martin (2003) con su propio lector de *bytecode*, sobre los mismos binarios de los ocho sistemas analizados, y se compararon arista por arista las dependencias entre paquetes del sistema. JDepend se ejecutó como un proceso aparte y no forma parte de la herramienta.

Tabla 4.12: Dependencias entre paquetes extraídas por el mecanismo y por JDepend sobre los mismos binarios.

Sistema	Mecanismo	JDepend	Comunes	Solo mec.	Solo JD	Paq. en ciclo
Commons Lang 3.14.0	43	43	43	0	0	12 / 12
Jackson Databind 2.17.2	166	163	163	3	0	21 / 21
ArchUnit 1.3.0	107	105	105	2	0	12 / 10
Guava 33.2.1	63	48	48	15	0	0 / 0
Spring Framework PetClinic	13	13	13	0	0	0 / 0
MyBatis JPetStore 6	5	5	5	0	0	0 / 0
DDD Sample	93	94	93	0	1	5 / 5
Caso de estudio (C0)	5	5	5	0	0	0 / 0
Total	495	476	**475**	20	1	

Ambas herramientas reconocen exactamente los mismos paquetes en los ocho sistemas, y coinciden en 475 de las 496 aristas distintas que entre las dos detectan. Lo importante es que **las 21 diferencias se explican sin excepción por las reglas de extracción documentadas** en la sección 3.4, y en ninguna dirección revelan un error:

- **Cinco aristas proceden de argumentos de tipos genéricos** —por ejemplo, un método de *ArchUnit* que devuelve `Optional<JavaClass>`—. El mecanismo las registra (regla R2, forma «genérico»); JDepend no lee la firma genérica de las clases.
- **Quince proceden de anotaciones con retención** `CLASS`: los paquetes de Guava anotados con `@GwtCompatible` o `@Beta` dependen del paquete de anotaciones. El mecanismo las registra (forma «anotación retenida»); JDepend no lee los atributos de anotaciones invisibles en tiempo de ejecución.
- **Una, solo de JDepend, procede de un tipo que únicamente aparece en una variable local** de DDD Sample, a través de la firma de un método invocado. El mecanismo, por la regla R2, no registra los tipos de variables locales.

La única discrepancia en los ciclos, la de *ArchUnit*, es consecuencia de las aristas genéricas, que incorporan dos paquetes más a un componente. El contraste confirma así dos cosas: que el grafo coincide con el de una herramienta independiente en todo lo que ambas pueden ver, y que el catálogo de formas de dependencia de la tabla 3.4 describe con exactitud lo que el mecanismo ve y lo que no. Durante el contraste se observó también que JDepend 2.9.5 aborta el análisis de todo un archivo al encontrar el descriptor de módulo de Java 9 (`module-info.class`); se le entregaron las clases sin él. La evidencia, arista por arista, se conserva en `prototipo/ejemplos/estudios/oraculo-jdepend.md`.

4.11. Aporte frente a la verificación con *ArchUnit*

El mecanismo se apoya en *ArchUnit* para leer el código compilado, lo que obliga a responder con datos qué aporta frente a escribir las mismas reglas directamente con esa biblioteca, que es como hoy se expresan en Java las *Fitness Functions* estructurales. Para responderlo se escribieron las tres reglas del caso de estudio con *ArchUnit*: la estratificación con su API de arquitectura en capas, los ciclos con su regla de *slices* libres de ciclos y, como la biblioteca no ofrece una regla de acoplamiento eferente, la concentración con código propio. La tabla 4.13 compara ambas formas sobre los cinco escenarios.

Tabla 4.13: Las reglas del caso de estudio escritas con *ArchUnit* y declaradas en el catálogo del mecanismo.

	ArchUnit	**Catálogo del mecanismo**
Detección en C0–C4	Idéntica regla por regla en los cinco escenarios	Idéntica regla por regla en los cinco escenarios
Especificación	33 líneas de Java, con código propio para Ce	14 líneas de propiedades
Quién puede escribirla	Quien programe en Java y conozca la API	Quien conozca la arquitectura prevista
Resultado	Pasa o falla por regla, con la lista de accesos infractores	CR, estado con severidad, evidencia por hallazgo, alcance del diagnóstico y tendencia

La detección es la misma, como debe ser: ambas formas leen el mismo *bytecode* con el mismo importador. El aporte no está en el motor de reglas, sino en lo que el mecanismo hace con su resultado. Reduce a menos de la mitad la especificación y la saca del lenguaje de programación, y sobre todo convierte tres veredictos binarios en un diagnóstico: una medida de la extensión, un estado que pondera la gravedad, la declaración de lo que quedó fuera y la comparación con el análisis anterior. Es la agregación que, según la revisión del capítulo 2, no ofrecía ninguna de las propuestas basadas en *Fitness Functions*. La comparación se reproduce con `arquitecta estudios archunit`.

4.12. Contraste de la hipótesis de trabajo

La hipótesis del anteproyecto (sección 1.2.3) tiene tres componentes, y no todos pueden contrastarse con la misma clase de evidencia. La tabla 4.14 los separa.

Tabla 4.14: Contraste de los componentes de la hipótesis de trabajo.

Componente	**Evidencia**	**Resultado**
(i) Detectar tempranamente derivas relevantes	Seis de seis derivas inducidas en el caso de estudio, sin falsos positivos; 31 de 31 inyectadas en siete sistemas reales; deriva real detectada en la versión en que aparece (Jackson 2.19.0, Commons Lang 3.21.0); grafo concordante con una herramienta independiente.	Confirmado
(ii) Diferenciar objetivamente un estado aceptable de uno problemático	Con el criterio aprobado (solo CR), dos de cinco escenarios con deriva real se clasificaron como aceptables; con el refinamiento, los cinco correctamente, y en los tres sistemas reales que partían de un estado aceptable toda deriva aislada inyectada cambió el veredicto final, aunque la CR apenas variara. El criterio es objetivo —un cociente y una regla de severidad declarados— y robusto al umbral de concentración.	Confirmado tras el refinamiento
(iii) Producir evidencia suficiente para apoyar decisiones de mantenimiento	Cada hallazgo identifica regla, paquetes, severidad y la referencia hasta el método y la línea. En C4 y en DDD Sample, la lectura conjunta redujo varios hallazgos a una causa común y a decisiones concretas. No se evaluó con equipos reales si el reporte les resulta útil para decidir.	Confirmado parcialmente
Sin depender de documentación externa	El mecanismo solo requiere el código compilado y un catálogo de pocas líneas versionado con él; sin catálogo evalúa FF2 y FF3.	Confirmado, con la precisión de la sección 3.3

El tercer componente merece una declaración explícita, porque es el único que el trabajo no pudo demostrar por completo. Lo que se demuestra es la **suficiencia de la evidencia**: el reporte contiene la información que una decisión de mantenimiento requiere, y la interpretación de los sistemas reales muestra decisiones concretas que de ella se derivan. Lo que no se demuestra es su **utilidad percibida**: que equipos de PyMES, al recibir el reporte, lo consideren útil y lo usen para priorizar. Esa demostración exige evaluadores ajenos al autor, y durante la ejecución no hubo acceso a equipos de desarrollo ni a profesionales dispuestos a evaluarlo; juzgarlo el propio autor no constituiría evidencia. Se trata, por tanto, de un impedimento de acceso y no de diseño, y su superación es directa: un estudio breve en el que entre cinco y diez desarrolladores lean el reporte de un sistema que conozcan y respondan, con una escala de Likert, si identifican la causa del hallazgo, si sabrían cómo corregirla y si la priorizarían, del tipo de la validación con dieciséis profesionales que emplearon Sas and Avgeriou (2023). Se plantea como la primera línea de trabajo futuro.

4.13. Amenazas a la validez

Toda evaluación empírica debe examinar con honestidad los límites de la evidencia que produce. El análisis sigue la clasificación habitual en investigación empírica en ingeniería de software (Wohlin et al., 2012; Runeson and Höst, 2009).

Validez interna. La principal amenaza proviene de la naturaleza sintética de los escenarios: al haber sido las derivas introducidas por el propio investigador, existe el riesgo de que el mecanismo se haya ajustado, aun de forma no intencional, a las particularidades de esos casos. Se aplicaron tres contenciones. Primera, las reglas se mantuvieron genéricas y se declararon por configuración, de modo que no codifican conocimiento del caso alterado; el mismo binario evaluó los cinco escenarios y el propio prototipo, con configuraciones distintas. Segunda, la matriz de validación se fijó antes de ejecutar el experimento, lo que impide reinterpretar a posteriori lo observado. Tercera, el comportamiento esperado se codificó además en la batería de pruebas, que verifica los valores de la CR sobre grafos construidos a mano, con independencia de la extracción por *bytecode*: si la herramienta de análisis estático produjera un grafo distinto del previsto, la discrepancia se manifestaría en lugar de pasar inadvertida. Una amenaza que no se logra contener por completo es que el refinamiento de la clasificación se diseñó *después* de observar el comportamiento de C1 y C3; aunque su formulación es independiente de esos escenarios —no menciona capas, ciclos ni concentración, sino solo severidades—, su validación sobre los mismos casos que lo motivaron no constituye evidencia independiente. Esta amenaza se atenuó después con datos que no intervinieron en el diseño: las tres aplicaciones en capas, cuyos catálogos transcriben la documentación de sus autores y cuyo umbral se fijó con un procedimiento definido antes de analizarlas, y las treinta y cinco versiones históricas, en las que el refinamiento siguió distinguiendo los sistemas con ciclos de los que no los tienen. Atenuada no significa eliminada: el autor eligió esos sistemas.

Validez externa. La generalización está limitada por el tamaño y la artificialidad de los escenarios. Cinco paquetes y once dependencias distan mucho de un sistema industrial, y los resultados demuestran la viabilidad del enfoque sin autorizar a extrapolar su desempeño a proyectos con arquitecturas heterogéneas, múltiples lenguajes o volúmenes de código muy superiores. El autoanálisis del prototipo —seis paquetes, catorce dependencias, veinticuatro verificaciones— aporta un segundo sistema real, aunque igualmente pequeño y desarrollado por el mismo autor. La aplicación a *ArchUnit* descrita en la sección 4.6 añade un tercero que sí es ajeno y de un orden de magnitud mayor, y con ello despeja una duda concreta: el mecanismo opera sobre código industrial sin degradarse, en poco más de un segundo. Pero no resuelve la amenaza de fondo, por tres razones que conviene no disimular: sigue siendo un único sistema, la arquitectura contra la que se le evaluó fue supuesta por el propio autor en lugar de declarada por sus desarrolladores, y nadie con conocimiento del sistema confirmó si los hallazgos corresponden a deriva real o a decisiones deliberadas. Hay, además, un efecto que el tamaño reducido de los escenarios oculta: la dilución descrita en la sección 4.3 se agrava con la escala, de modo que en sistemas grandes cabe esperar que el criterio basado solo en la CR sea aún menos discriminante que lo observado aquí. Las validaciones de las

secciones 4.7 a 4.10 amplían de forma considerable esa base —siete sistemas reales de entre 26 y 2 017 clases, tres de ellos aplicaciones en capas con arquitectura documentada, y treinta y cinco versiones históricas—, pero siguen siendo sistemas de código abierto elegidos por el autor y analizados sin la participación de sus equipos. La generalización exige una validación sobre repositorios industriales con la participación de quienes los mantienen, que se plantea como trabajo futuro.

Validez de constructo. La pregunta es si las variables observadas representan adecuadamente el fenómeno que se pretende medir. Operacionalizar la deriva arquitectónica en solo tres dimensiones es, necesariamente, una aproximación parcial de un fenómeno complejo: un sistema puede obtener una CR del 100 % y padecer derivas que el catálogo no cubre, razón por la cual el reporte declara explícitamente qué quedó fuera del diagnóstico. La propia extracción tiene puntos ciegos conocidos (sección 3.4): las dependencias que solo existen en tiempo de ejecución —reflexión, carga dinámica, configuración externa— y las que el compilador elimina no llegan al grafo, de modo que la arquitectura implementada que se compara es la que el código compilado declara, no necesariamente toda la que el sistema ejecuta. El contraste con JDepend (sección 4.10) acota esta amenaza en lo que el análisis estático sí puede ver: sobre 496 aristas, todas las discrepancias se explican por las reglas de extracción declaradas. A ello se suma una limitación específica de FF3 que el autoanálisis puso de manifiesto: al operar solo sobre la topología, la regla no distingue un componente dios de un concentrador legítimo, como una raíz de composición o una fachada, y por tanto su umbral no es un valor universal sino un parámetro que cada organización debe calibrar.

Las exenciones descritas en la sección 3.3 atenúan esa ceguera, pero no la eliminan, y al introducirlas aparece una amenaza nueva que debe declararse con franqueza. El mecanismo sigue sin ser capaz de reconocer por sí mismo el papel de un paquete: se limita a aplicar el que alguien le declaró. Esto traslada el juicio a quien configura, y con ello abre la posibilidad de que una exención mal concedida oculte una concentración o un ciclo perfectamente reales. La evidencia recogida sobre *ArchUnit* sugiere que el riesgo no se materializó allí —los tres ciclos sobrevivieron a las exenciones—, pero un solo caso favorable no es garantía. Las salvaguardas disponibles son de procedimiento y no técnicas: que la exención se declare en la configuración y no en el código, que el reporte la enumere siempre, y que su concesión quede sujeta a la misma revisión que cualquier otra decisión arquitectónica.

Por último, la asignación de paquetes a capas se resuelve por prefijo y en el orden en que las capas se declaran, lo que exige cuidado cuando una capa es prefijo de otra.

Validez de conclusión. Las conclusiones se apoyan en la transparencia del cálculo: al ser la CR un cociente explícito y reproducible, cualquier afirmación puede auditarse reconstruyendo el diagnóstico paso a paso, y la evidencia se conserva en formatos verificables por terceros. No obstante, el carácter determinista del experimento —cinco escenarios, una configuración, un sistema— impide todo tratamiento estadístico: el mecanismo produce siempre el mismo resultado sobre el mismo código, de modo que repetir la ejecución no aporta variabilidad que estimar. La única fuente de variabilidad del trabajo es el asistente, cuyas corridas se repitieron y se reportan como frecuencias (anexo B). Los resultados deben leerse como evidencia de viabilidad y de sensibilidad por dimensión,

no como una estimación de desempeño generalizable.

4.14. Resumen del capítulo

Este capítulo presentó el diseño de la evaluación, concebido como un experimento controlado con una matriz de validación fijada de antemano y con criterios de éxito explícitos. El mecanismo detectó las seis derivas inducidas en los cuatro escenarios alterados, sin producir un solo hallazgo sobre la implementación de referencia, superando así el criterio del 80 % comprometido para OBJ4. Como complemento, una validación de aplicabilidad en entorno controlado sobre siete sistemas reales —cuatro bibliotecas y tres aplicaciones en capas— detectó las 31 derivas inyectadas sin hallazgos espurios y confirmó, fuera del caso de estudio, la dilución de la CR que motivó el refinamiento; sobre treinta y cinco versiones publicadas, el mecanismo encontró además deriva real que nadie inyectó. La formulación de la CR del anteproyecto, calculada en paralelo, no se diluye pero satura y no muestra tendencia; el umbral de concentración se sustituyó por un procedimiento estadístico ante el que la clasificación resultó robusta; el grafo coincidió con el de una herramienta independiente salvo en diferencias explicadas por las reglas de extracción; y las mismas reglas escritas con *ArchUnit* detectaron lo mismo con más del doble de especificación y sin veredicto agregado. Los componentes de detección y de diferenciación de la hipótesis quedaron confirmados; el de utilidad para decidir, solo en cuanto a la suficiencia de la evidencia, por falta de acceso a evaluadores. Los escenarios de deriva aislada revelaron, además, que la Tasa de Conformidad —al ser una proporción no ponderada— diluye las violaciones localizadas y puede clasificar como aceptable un sistema con una desviación real. Ese hallazgo motivó un refinamiento del criterio de clasificación que incorpora la severidad del peor hallazgo sin alterar el cálculo de la métrica, y que clasifica correctamente los cinco escenarios. Al margen del experimento, el mecanismo se aplicó sobre una biblioteca de terceros para comprobar su comportamiento ante código real: la regla de integridad de capas resultó exacta, el análisis completo tardó poco más de un segundo y las exenciones declaradas eliminaron cerca de dos tercios de los hallazgos de concentración sin silenciar los ciclos genuinos. El capítulo cerró examinando las amenazas a la validez de la evidencia producida, entre las que destacan la artificialidad de los escenarios, el hecho de que el refinamiento se validara sobre los mismos casos que lo motivaron y el riesgo que introducen las propias exenciones al trasladar a quien configura un juicio que el mecanismo no sabe emitir.

Capítulo 5

Conclusiones

5.1. Conclusiones

Este trabajo se propuso desarrollar y evaluar un mecanismo automatizado que detectara la deriva arquitectónica combinando grafos de dependencia y *Fitness Functions*, con el fin de clasificar el estado de conformidad estructural de proyectos de PyMES sin depender de documentación externa. Los resultados obtenidos permiten afirmar que el objetivo general se cumplió: el mecanismo fue construido, evaluado y demostró ser capaz de discriminar entre una implementación conforme y otra que acumula desviaciones, así como de detectar cada una de las tres familias de deriva cuando se presentan de forma aislada, tanto en el caso de estudio como en siete sistemas reales a los que se inyectaron derivas conocidas, y de encontrar deriva real —no inyectada— en la evolución de sistemas publicados. Contrastada componente por componente (sección 4.12), la hipótesis de trabajo queda confirmada en la detección y en la diferenciación objetiva de estados, y solo parcialmente en la utilidad de la evidencia para decidir: se demostró que el reporte contiene lo que una decisión necesita, pero no que equipos reales lo juzguen útil, porque no hubo acceso a evaluadores.

El cumplimiento de los objetivos específicos respalda esa afirmación. Respecto a OBJ1, se formuló, a partir de la literatura sobre verificación de conformidad y *smells* arquitectónicos y con criterios de selección explícitos, un catálogo acotado de tres *Fitness Functions* —integridad de capas, ausencia de ciclos y control de concentración de dependencias— en el que cada regla declara su propiedad protegida, su racionalidad, su unidad de verificación, su criterio de incumplimiento y su severidad; el catálogo no reside solo en el documento, sino que el propio prototipo lo deriva de la implementación vigente y puede imprimirlo bajo demanda. En cuanto a OBJ2, se implementó un componente de análisis estático que construye un grafo de dependencias entre paquetes a partir del código compilado, con reglas de extracción explícitas y verificadas, prescindiendo de la documentación arquitectónica y conservando, junto a cada arista, la referencia entre clases que la origina. En relación con OBJ3, se construyó un motor de validación que agrega las verificaciones en una Tasa de Conformidad y clasifica el estado del sistema en tres niveles interpretables, empleando los umbrales del 85 % y el 70 %; la formulación de la CR del anteproyecto se calcula y se reporta en paralelo, de modo que la precisión introducida es verificable sobre la misma evidencia. Por último, OBJ4 se satisfizo mediante un experimento controlado sobre cinco escenarios en el que el mecanismo detectó el cien por ciento de las derivas inducidas sin producir un solo hallazgo sobre la implementación de referencia, superando el criterio de éxito del ochenta por ciento.

La tabla 5.1 recoge esa correspondencia frente a los criterios de éxito que el anteproyecto fijó para cada objetivo, junto con la evidencia concreta que permite verificarla.

Tabla 5.1: Cumplimiento de los criterios de éxito comprometidos en el anteproyecto.

Obj.	**Criterio de éxito comprometido**	**Evidencia de cumplimiento**
OBJ1	Catálogo con al menos una regla ejecutable por familia de deriva, con propósito, evidencia esperada y criterio de incumplimiento.	Tres *Fitness Functions* documentadas en la tabla 3.3, derivadas de la implementación vigente e imprimibles desde el propio prototipo.
OBJ2	Grafo navegable que represente de forma consistente los componentes y dependencias necesarios para evaluar las reglas.	Grafo construido desde el *bytecode* y dibujado por el propio mecanismo, ordenado según la estratificación y con los hallazgos resaltados; las figuras 3.4, 3.5, 3.6 y 4.1 las generó él. Coincide con el de una herramienta independiente en 475 de 496 aristas, con todas las diferencias explicadas (tabla 4.12).
OBJ3	Cálculo automático de la CR, aplicación de los umbrales y clasificación coherente con la matriz de validación.	Ecuación 3.1 y algoritmo 1; clasificación coincidente con la matriz esperada en los cinco escenarios (tabla 4.3).
OBJ4	Detección de al menos el 80 % de las derivas inducidas, línea base sin hallazgos inesperados y reportes comprensibles.	Detección del 100 % de las seis derivas, cero hallazgos sobre la línea base y reportes con evidencia hasta la línea de código (tabla 4.7). Como complemento, 31 de 31 derivas inyectadas en siete sistemas reales.

El resultado más instructivo, sin embargo, no fue la confirmación de lo esperado sino el descubrimiento de una limitación. Al someter el mecanismo a escenarios con una única deriva —un diseño que el trabajo incorporó precisamente para medir la sensibilidad de cada regla por separado—, se constató que la Tasa de Conformidad, por ser una proporción no ponderada, diluye las violaciones localizadas hasta el punto de clasificar como aceptable un sistema que contiene una desviación real y documentada. La métrica mide con fidelidad la extensión del deterioro, pero es ciega a su gravedad, y esa ceguera se agrava conforme el sistema crece. La conclusión que de ello se desprende trasciende al prototipo: cualquier indicador de conformidad construido como proporción heredará esta propiedad, de modo que la agregación de hallazgos arquitectónicos no puede descansar únicamente en el conteo. La alternativa literal del anteproyecto, contar reglas en lugar de verificaciones, no la resuelve: no se diluye, pero satura —toda deriva aislada deja el sistema en 66,67 %— y no registra la tendencia que, en Jackson Databind, sí registró la CR por verificación a lo largo de trece versiones.

Los hallazgos dialogan con la literatura existente y la complementan. Frente a los enfoques basados en modelos, que exigen una especificación de referencia detallada (Murphy et al., 2001; San Martín et al., 2024; Knodel and Popescu, 2007), la propuesta demuestra que basta con una declaración mínima de la intención —unas pocas decenas de líneas que viven junto al código— para verificar la conformidad, y que la estructura implementada puede obtenerse directamente del código. Frente a los trabajos centrados en *smells* (Sas et al., 2019; Sas and Avgeriou, 2023), la contribución consiste en traducir la detección en un veredicto agregado que sigue siendo reconstruible por quien lo recibe. Y frente a las propuestas de *Fitness Functions* como práctica (Ford et al., 2017, 2022; Paul

and Wang, 2019; TNG Technology Consulting, 2026), este trabajo aporta un mecanismo que integra la extracción, la evaluación y la clasificación en un único instrumento ligero; escritas directamente con *ArchUnit*, las mismas reglas detectan lo mismo, pero exigen más del doble de especificación en código y no producen un diagnóstico agregado (sección 4.11). En conjunto, la investigación confirma que la gobernanza arquitectónica continua no es un privilegio exclusivo de las grandes organizaciones, sino una capacidad que puede ponerse al alcance de las PyMES (Modawski and Wolniak, 2025).

5.2. Contribuciones del trabajo

Más allá del cumplimiento puntual de los objetivos, el trabajo aporta cuatro contribuciones que conviene deslindar.

La primera es **conceptual**: la propuesta articula en un único instrumento dos cuerpos de conocimiento que hasta ahora avanzaban por separado —el análisis estático de dependencias y las *Fitness Functions* como práctica de gobernanza— y demuestra que su integración es viable y produce un diagnóstico accionable.

La segunda es **empírica** y no estaba prevista. Comprende dos hallazgos sobre los límites de este tipo de instrumentos, ambos medidos y no supuestos. El primero es que una tasa de conformidad formulada como proporción de verificaciones diluye las violaciones aisladas, y que esa ceguera se corrige combinando la extensión del incumplimiento con la gravedad del peor hallazgo; la corrección es deliberadamente simple —adoptar el más restrictivo de dos veredictos— y por ello sigue siendo explicable ante un equipo, que es la condición que el contexto de las PyMES impone a cualquier instrumento de gobernanza. El segundo es que una regla puramente topológica produce, sobre código real, una proporción de ruido que la vuelve inutilizable si no se le permite conocer el papel de cada paquete: sobre *ArchUnit*, dos tercios de los hallazgos de concentración procedían de dependencias vendorizadas o de un núcleo de anotaciones, y desaparecieron al declarar las exenciones correspondientes sin que los ciclos genuinos se vieran afectados. El tercero es que un umbral absoluto de acoplamiento no se traslada entre sistemas de distinto tamaño, mientras que un procedimiento —el percentil 90 de la propia línea base— sí, y que la clasificación resultante es robusta a esa elección. La lección que se desprende de ambos es la misma y va en contra de la intuición inicial del proyecto: lo que hace útil a un mecanismo de conformidad no es la cantidad de reglas que evalúa, sino la política con que interpreta sus resultados.

La tercera es **metodológica**: el documento ofrece un proceso explícito y reproducible para detectar la deriva sin depender de documentación externa —qué se declara, cómo se extrae el grafo, cómo se evalúa, en qué unidades se mide y cómo se interpreta—, junto con una matriz de validación fijada antes de la ejecución, una estrategia de verificación por etapas una técnica de inyección controlada de derivas que permite evaluar el mecanismo sobre sistemas reales sabiendo de antemano qué debería detectar, y un esquema de validación en tres niveles —derivas inyectadas, deriva real en la evolución publicada y contraste de la extracción con un oráculo independiente— que puede reutilizarse para evaluar cualquier herramienta de conformidad. Las decisiones que lo configuraron, con sus alternativas y su respaldo, quedan registradas como *Architecture Decision*

Records (anexo C).

La cuarta es **práctica**: el prototipo constituye una base funcional que una PyME puede adoptar, adaptar y calibrar según su contexto —basta con elegir la carpeta del proyecto en su interfaz gráfica para obtener un primer diagnóstico—, cuyo código está estructurado para favorecer la extensión y cuya evidencia se emite en formatos verificables por terceros.

Estas contribuciones dialogan con el estado del arte sin pretender superarlo en todos los frentes. La propuesta no compite con los enfoques basados en modelos en poder expresivo, ni con los enfoques de detección de *smells* en profundidad analítica; su valor reside en el punto de equilibrio que logra entre costo, autonomía e interpretabilidad, precisamente el punto que los trabajos previos dejaban desatendido.

Esta lectura se alinea, sin desviarse, con la brecha que el anteproyecto identificó como espacio de contribución. Allí se constató que ningún antecedente reunía de forma simultánea cinco elementos: el análisis estático exclusivo del código, la combinación de grafo de dependencias con *Fitness Functions*, la clasificación mediante una Tasa de Conformidad, un experimento controlado con criterios explícitos y una orientación específica a las PyMES sin documentación arquitectónica externa. El presente trabajo no añadió objetivos nuevos ni amplió el alcance: se limitó a ocupar, de manera verificable, ese espacio delimitado, y a documentar con honestidad dónde el instrumento resultó insuficiente y cómo se corrigió.

5.3. Aplicación en la industria

La evidencia reunida permite precisar cómo se usaría el mecanismo en una organización real y qué cabe esperar de él. Conviene empezar por lo que la sostiene: el instrumento detectó todas las derivas inducidas en el caso de estudio y todas las inyectadas en siete sistemas reales de hasta dos mil clases —entre ellos tres aplicaciones en capas—, sin un solo hallazgo espurio; encontró deriva real en la evolución de dos bibliotecas publicadas, y analiza cualquiera de ellos en menos de un segundo, directamente sobre su código compilado o su `.jar`. Con esa base, su uso en la industria se organiza en torno a cuatro momentos.

El primero es la **adopción**. Una PyME rara vez tiene su arquitectura escrita, y ese es el obstáculo real, no la ejecución. El camino que el prototipo ofrece es el que se ensayó con las bibliotecas de la sección 4.7: abrir la carpeta del proyecto, obtener un primer diagnóstico de ciclos y concentración con la configuración mínima y, si se desea verificar la estratificación, pedir al asistente una propuesta de catálogo, revisarla y adoptarla. Lo que el equipo aporta es lo único que el código no contiene: la intención.

El segundo es la **vigilancia continua**. Una vez adoptado el catálogo, el análisis puede ejecutarse en cada integración y detener la entrega cuando el veredicto alcanza la gravedad elegida. Aquí la validación sobre sistemas reales aporta una pauta de uso concreta: en un sistema de cierto tamaño, una deriva nueva apenas mueve la CR, de modo que la señal útil no es su valor absoluto sino su variación entre corridas y la severidad del hallazgo nuevo, que es lo que el criterio combinado y el registro de tendencia ya ofrecen. Del mismo modo, un catálogo adoptado sobre un sistema que ya arrastra deriva funciona en la práctica como una línea base: no revela la deriva heredada, pero

impide que crezca.

El tercero es el **apoyo a la decisión**. El reporte y el grafo sirven para revisar un cambio antes de integrarlo, para priorizar una refactorización por severidad y dimensión —el ciclo de 19 paquetes de Jackson Databind es el tipo de hallazgo que justifica un plan de trabajo, y los ocho hallazgos de DDD Sample, reducidos a tres decisiones de diseño, muestran cómo leerlos en conjunto—, para incorporar a un miembro nuevo con una fotografía verificable de la estructura y para contrastar los *Architecture Decision Records* con lo efectivamente construido.

El cuarto es la **auditoría**. Como el análisis no ejecuta el sistema ni requiere su código fuente, puede aplicarse a un componente que se va a adquirir o a una entrega de un proveedor, con una evidencia que un tercero puede reproducir.

Esta lectura debe acompañarse de sus condiciones. El mecanismo responde a las tres dimensiones del catálogo y a nivel de paquete; no evalúa atributos de calidad ni el comportamiento en ejecución, y un veredicto *aceptable* significa que no se hallaron incumplimientos de esas reglas, no que el sistema carezca de problemas. Y su utilidad depende de que el catálogo exprese una intención real: uno deducido del código sin revisión protege contra la deriva futura, pero no diagnostica la pasada.

5.4. Trabajos futuros

La evaluación realizada abre varias líneas de trabajo que conviene explorar, ordenadas aquí por la urgencia que la propia evidencia les confiere.

La más apremiante es la **validación con personas**, que es lo único que el trabajo no pudo hacer y que el tercer componente de la hipótesis requiere. Un estudio breve en el que entre cinco y diez desarrolladores lean el reporte de un sistema que conozcan y valoren si identifican la causa, si sabrían corregirla y si la priorizarían (sección 4.12) convertiría la suficiencia de la evidencia, ya demostrada, en utilidad demostrada. Inmediatamente después viene la **validación externa** sobre proyectos reales de la industria. La aplicación a sistemas reales (secciones 4.6 a 4.10) despejó una parte del problema —el mecanismo detecta derivas conocidas y reales en código ajeno, a escala, sin ruido y en menos de un segundo—, pero dejó intacta la otra: nadie con conocimiento de esos sistemas confirmó si sus hallazgos correspondían a deriva o a decisiones deliberadas, y las aplicaciones en capas son aplicaciones de referencia y no sistemas en producción de una PyME. Un estudio con equipos industriales, sobre sus propias aplicaciones y con su propio catálogo, es el paso que convertiría esta evidencia en validación externa.

La segunda línea, y la que más acercaría el instrumento a una adopción real, es la incorporación de una **línea base**. El obstáculo que hoy impide aplicarlo al diagnóstico de un sistema legado no es técnico sino de utilidad: una primera ejecución sobre código con deriva acumulada produce un volumen de hallazgos que ningún equipo puede accionar, y el desenlace habitual de esa avalancha es que el instrumento se desactive. La solución conocida consiste en fijar el estado actual como punto de partida y reportar en adelante únicamente las violaciones *nuevas*, de modo que la conformidad se exija sobre el código que se escribe y no sobre el que se hereda. Conviene señalar que esta extensión no añadiría una cuarta *Fitness Function*: el catálogo permanecería en tres, y lo que se incorporaría es una capa de política sobre el motor existente.

La tercera línea es la **reducción del costo de la primera configuración**. Para una organización pequeña, la barrera de entrada no es ejecutar el mecanismo sino redactar la declaración inicial de su arquitectura. Que la estratificación de *ArchUnit* pudiera deducirse de la estructura de sus paquetes y resultara correcta en las sesenta y una verificaciones sugiere que esa declaración es, al menos parcialmente, inferible: un modo que proponga capas candidatas a partir del árbol de paquetes y del sentido de las dependencias, y que fije el umbral de concentración con el procedimiento de percentiles de la sección 4.9, convertiría un ejercicio de análisis en un punto de partida editable. El asistente de catálogo que incluye el prototipo (sección 3.15) da un primer paso en esa dirección, apoyándose en un modelo de lenguaje en lugar de una heurística fija; una exploración preliminar con modelos reales, remotos y locales (anexo B), indica que las propuestas de los modelos remotos no legitiman las derivas de capas ni los ciclos, que las de un modelo local son bastante menos fiables, y que tanto el umbral de concentración como las exenciones exigen salvaguardas deterministas. Queda pendiente evaluar empíricamente la calidad de sus propuestas, por ejemplo contrastándolas con los catálogos que declaren los arquitectos de sistemas reales y midiendo cuántas de sus decisiones se conservan tras la revisión.

A ellas se suman tres extensiones de alcance más acotado: ampliar el catálogo de *Fitness Functions* a otras manifestaciones de la deriva, como la violación de patrones de diseño; refinar la granularidad del análisis para emitir diagnósticos a nivel de clase y no solo de paquete; e integrar el mecanismo en una tubería de integración continua, para lo cual el prototipo ya ofrece la afordancia de terminar con un código de salida condicionado al veredicto, si bien esa integración solo resulta sostenible una vez incorporada la línea base, pues de otro modo bloquearía toda entrega desde el primer día. Por último, la extensión a otros ecosistemas mediante herramientas análogas a *ArchUnit* en otras plataformas (Panneerselvam, 2023) ampliaría el alcance de la propuesta más allá del ecosistema Java.

Dos líneas más se desprenden de la validación sobre sistemas reales. La primera es llevar el **análisis de la evolución** del nivel de versión publicada (sección 4.8) al de cada cambio del historial de un repositorio, para saber en qué *commit* apareció cada desviación y quién estaba en condiciones de evitarla. La segunda es una **métrica que no se diluya ni sature**: entre los dos extremos medidos —la CR por verificación, que se diluye, y la CR por regla, que satura—, el criterio combinado corrige la clasificación, pero la CR sigue perdiendo sensibilidad a medida que crecen las verificaciones; ponderar los incumplimientos por regla o por severidad, o reportar la conformidad local de cada regla como señal principal, son alternativas que conviene evaluar con la misma inyección controlada empleada en este trabajo. A ello se suma completar FF3 con el componente aferente del *smell* de dependencia concentradora, que hoy queda fuera por fidelidad a la dimensión aprobada.

5.5. Lecciones aprendidas

El proceso dejó enseñanzas que trascienden el resultado puntual.

La primera, y la más valiosa, se refiere al diseño experimental: **un único escenario que acumula todas las derivas comprueba que el instrumento reacciona, pero oculta cómo**

reacciona. Fue la decisión de construir escenarios con una sola deriva la que reveló la limitación de la métrica; con un solo caso alterado, el trabajo habría concluido que el mecanismo funcionaba correctamente y esa conclusión, aun siendo cierta en sus términos, habría sido engañosa. Aislar las variables no es un refinamiento estético del experimento: es lo que lo convierte en una fuente de conocimiento.

La segunda tiene que ver con la agregación de métricas. Reducir un diagnóstico a una sola cifra facilita la comunicación, pero toda agregación pierde información, y conviene saber cuál. En este caso, la proporción perdía la gravedad de los hallazgos, y recuperarla exigió añadir una segunda dimensión en lugar de complicar la fórmula de la primera. Mantener las dos lecturas separadas y visibles resultó preferible a fundirlas en un índice ponderado que nadie habría podido reconstruir.

La tercera se refiere a la configurabilidad: externalizar las reglas en un archivo de propiedades permitió reutilizar el mismo motor sobre trece sistemas distintos —los cinco escenarios, el propio prototipo, cuatro bibliotecas y tres aplicaciones reales— y sobre treinta y cinco versiones históricas sin modificar una sola línea de código, una decisión de diseño que resultó decisiva.

La cuarta concierne al contrato de las reglas. Una primera versión permitía que los conteos que alimentan la métrica y la evidencia que la explica procedieran de recorridos distintos; obligar a que ambos surjan de una única evaluación eliminó por construcción una clase entera de inconsistencias posibles. En un instrumento cuya credibilidad descansa en que el número y su justificación coincidan, esa garantía no podía quedar en manos de la disciplina del programador.

La quinta se refiere a la validación por etapas —primero el núcleo lógico sobre grafos construidos a mano, después el componente que depende de la herramienta externa—, que redujo el riesgo de confundir un defecto propio con un comportamiento ajeno y permitió ejercitar casos límite que ningún caso de estudio ofrece. Aplicar además el instrumento sobre sí mismo resultó ser una comprobación barata y reveladora: confirmó que la arquitectura descrita era la implementada y, de paso, expuso una limitación de la regla de concentración que ningún escenario diseñado a propósito había mostrado.

La sexta es que **la evidencia propia necesita un contraste ajeno**. Las pruebas que el autor escribe verifican lo que el autor espera; solo el contraste con una herramienta independiente mostró que el grafo coincide con el de otra implementación y que las diferencias son exactamente las que las reglas de extracción predicen. Ese contraste costó una tarde y es, probablemente, la pieza de evidencia que más reduce la desconfianza razonable de un lector externo.

Finalmente, una lección sobre el alcance. La disciplina de acotar el catálogo a tres dimensiones, en lugar de perseguir una cobertura exhaustiva, resultó esencial para entregar un instrumento funcional y evaluable. En investigación aplicada, la tentación de ampliar el alcance suele conspirar contra la posibilidad de validar; en este trabajo, la decisión inversa —acotar para poder verificar— fue la que dejó margen para investigar a fondo lo poco que se abarcó, y ese margen fue justamente donde apareció el hallazgo más interesante.

Capítulo 6

Riesgos éticos

6.1. Riesgos éticos

Este capítulo aborda las consideraciones éticas que rodearon el desarrollo y la evaluación del mecanismo. Aunque el proyecto no involucró sujetos humanos ni datos personales, la naturaleza de su artefacto —un instrumento que emite juicios sobre la calidad estructural del código— obligó a examinar con cuidado los usos posibles de sus resultados y las responsabilidades que de ellos se derivan. La exposición conserva la estructura comprometida en el anteproyecto: primero los principios que orientaron el diseño, después los cinco riesgos identificados con el tratamiento asignado a cada uno, y por último el estado en que quedaron tras la ejecución del proyecto.

6.2. Principios orientadores

El desarrollo y la evaluación del mecanismo se rigieron por cuatro principios.

El primero fue la **privacidad por diseño**: el mecanismo opera sobre el código compilado y de manera local, sin transmitir, almacenar ni exponer información fuera del entorno de quien lo ejecuta; la única excepción es el asistente de catálogo, que es opcional, se activa a petición expresa, envía solo la estructura de paquetes —nunca el código— y, mientras la persona no elija expresamente un servicio externo, usa un modelo local, de modo que por defecto ningún dato sale del equipo. El segundo fue la **transparencia del diagnóstico**: toda clasificación emitida se acompaña de la evidencia concreta que la sustenta, de modo que el resultado sea auditable. El tercero fue la **supervisión humana**: el instrumento se concibió como un apoyo a la decisión, nunca como un reemplazo del juicio de los profesionales. El cuarto fue el **uso proporcional de las métricas**: las mediciones se limitaron a propiedades estructurales del código y se evitaron inferencias sobre el desempeño de personas o equipos.

6.3. Riesgos identificados y tratamiento aplicado

Siguiendo la lógica de gestión de riesgos adoptada en el anteproyecto, cada situación recibió uno de cuatro tratamientos posibles —evitarla, mitigarla, transferirla o aceptarla de forma controlada—, y a continuación se describe cómo se materializó ese tratamiento durante la ejecución.

- **Riesgo sobre información sensible. Tratamiento: evitarlo.** El análisis estático requiere acceso al código, por lo que la respuesta adecuada consistió en impedir desde el inicio todo escenario innecesario de exposición. El prototipo se ejecuta íntegramente en local, sin servicios

externos ni telemetría, y los artefactos que genera contienen únicamente relaciones estructurales y métricas agregadas. La evaluación se realizó sobre sistemas sintéticos construidos para el proyecto, sobre el propio prototipo y sobre una biblioteca de código abierto de acceso público, de modo que en ningún momento se procesó información sujeta a confidencialidad.

- **Riesgo de sesgo algorítmico y opacidad del diagnóstico. Tratamiento: mitigarlo.** Para evitar decisiones de refactorización basadas en umbrales arbitrarios o interpretaciones erróneas, las reglas se mantuvieron explícitas, versionadas y trazables: residen en un archivo de configuración legible, y cada hallazgo declara la regla activada, la evidencia estructural encontrada —hasta el método y la línea que originan la dependencia— y la severidad asignada. La Tasa de Conformidad se calcula como un cociente explícito que cualquier miembro del equipo puede rehacer a mano, y no mediante un modelo cuyo resultado no pudiera reconstruirse. La validación se apoyó, además, en una matriz de comportamiento esperado fijada antes de la ejecución y sometida a revisión humana.

- **Riesgo de impacto sociotécnico en los equipos. Tratamiento: mitigarlo.** Para reducir la posibilidad de que la gobernanza automatizada se perciba como vigilancia punitiva, el mecanismo mide exclusivamente propiedades de paquetes y no atribuye hallazgos a personas: ni el modelo de datos ni el reporte contemplan la noción de autor. El instrumento se propone como soporte para conversaciones técnicas, retrospectivas arquitectónicas y priorización de refactorización, no como insumo de evaluación individual.

- **Riesgo de formalismo vacío. Tratamiento: aceptarlo de forma controlada.** Siempre existirá la posibilidad de que una métrica de conformidad se use de manera mecánica y sin reflexión contextual. Ese riesgo no puede eliminarse, por lo que se aceptó de forma controlada y se redujo su impacto por dos vías. La primera es que el reporte declara siempre su propio alcance —qué reglas se evaluaron, qué umbrales se aplicaron, qué paquetes quedaron fuera de la estratificación y qué aspectos no cubre el diagnóstico—, advirtiendo de forma expresa que un veredicto *aceptable* significa que no se hallaron incumplimientos de esas reglas y no que el sistema carezca de problemas estructurales. La segunda es la configurabilidad de las reglas y los umbrales, que obliga a cada organización a decidir explícitamente qué considera admisible en lugar de heredar un criterio ajeno.

- **Riesgo de sobreinterpretación de la Tasa de Conformidad. Tratamiento: transferirlo parcialmente a la decisión humana responsable.** La CR representa únicamente el cumplimiento de las reglas configuradas dentro del alcance del prototipo. La evaluación aportó evidencia concreta de por qué esta advertencia no era retórica: en dos de los escenarios evaluados, la métrica clasificó como aceptable un sistema que contenía una violación real, precisamente porque una proporción diluye los incumplimientos localizados. El hallazgo confirma que la cifra no puede leerse como veredicto absoluto y que la decisión final sobre priorización de refactorización o aceptación del riesgo corresponde al responsable técnico y no a la herramienta. En respuesta, el mecanismo incorporó la severidad del peor hallazgo como criterio complementario y advierte de forma explícita en el reporte cuando ese criterio ha

endurecido el veredicto que la métrica habría emitido por sí sola, de modo que la discrepancia entre ambas lecturas quede a la vista de quien decide. Por la misma razón el reporte incluye también la CR por regla del anteproyecto: mostrar dos lecturas de la misma evidencia, con sus límites declarados, desalienta tomar cualquiera de ellas como veredicto único.

La tabla 6.1 sintetiza los cinco riesgos, su tratamiento y el estado en que quedaron al cierre del proyecto.

Tabla 6.1: Riesgos éticos, tratamiento aplicado y estado al cierre del proyecto.

Riesgo	**Tratamiento**	**Control materializado**	**Estado**
Información sensible	Evitar	Ejecución local sin servicios externos; evaluación sobre sistemas propios, sintéticos y de código abierto.	Controlado
Sesgo y opacidad del diagnóstico	Mitigar	Reglas explícitas y versionadas; evidencia hasta la línea de código; métrica reconstruible a mano.	Controlado
Impacto sociotécnico en los equipos	Mitigar	El modelo de datos no contempla la noción de autor; los hallazgos se atribuyen a paquetes.	Controlado
Formalismo vacío	Aceptar de forma controlada	Declaración de alcance en todo reporte; umbrales y reglas de decisión configurables.	Residual
Sobreinterpretación de la CR	Transferir a la decisión humana	Severidad como criterio complementario; aviso explícito cuando ambos criterios discrepan; declaración de exclusiones.	Residual

6.4. Riesgos residuales

Dos riesgos permanecen activos en un nivel residual y conviene declararlo sin atenuantes.

El primero es el formalismo vacío. Ningún control técnico impide que una organización convierta la Tasa de Conformidad en un objetivo por sí mismo y persiga el número en lugar de la propiedad que representa. Las salvaguardas incorporadas —la declaración de alcance y la configurabilidad— reducen la probabilidad de que ello ocurra por desconocimiento, pero no por decisión deliberada.

El segundo es la interpretación indebida de los resultados en contextos arquitectónicos distintos del evaluado. El autoanálisis del prototipo mostró un caso concreto: la regla de concentración señala como problemáticos a componentes que concentran dependencias por diseño, como una raíz de composición o una fachada. En un sistema real, un hallazgo así podría desencadenar una refactorización innecesaria si se acatara sin criterio. La mitigación disponible es la que el proyecto ya adopta —transparencia del diagnóstico y recomendación explícita de supervisión humana—, y la solución de fondo, el registro de excepciones justificadas, queda planteada como trabajo futuro.

Ambos riesgos se consideran aceptables en el estado actual del proyecto, dado que el artefacto se entrega como prototipo de investigación y no como producto de adopción industrial. No se identificaron conflictos de interés ni implicaciones éticas no previstas durante la ejecución.

Bibliografía

Alves, T. L., Ypma, C., and Visser, J. (2010). Deriving metric thresholds from benchmark data. In *Proceedings of the 26th IEEE International Conference on Software Maintenance (ICSM)*, pages 1–10. IEEE.

Anthony, E., Berntsson, A., Santilli, T., and Wohlrab, R. (2024). We're drifting apart: Architectural drift from the developers' perspective. In *Proceedings of the IEEE 21st International Conference on Software Architecture (ICSA 2024)*, pages 101–111.

AOE Technology Radar (2026). Architecture decision records (ADR) — methods & patterns. Consultado en 2026.

Arcelli Fontana, F., Pigazzini, I., Roveda, R., and Zanoni, M. (2016). Automatic detection of instability architectural smells. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 433–437.

Bass, L., Clements, P., and Kazman, R. (2012). *Software Architecture in Practice*. Addison-Wesley Professional, Boston, MA, USA, 3rd edition.

Besker, T., Martini, A., and Bosch, J. (2019). Software developer productivity loss due to technical debt—a replication and extension study examining developers' development work. *Journal of Systems and Software*, 156:41–61.

Clark, M. (2015). JDepend: métricas de calidad de diseño por paquete para Java. Herramienta de código abierto; se usó la versión 2.9.5 de guru.nidi.

codecentric AG (2026). Archunit in practice: Keep your architecture clean. Consultado en 2026.

Cunningham, W. (1992). The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, pages 29–30. ACM.

de Silva, L. and Balasubramaniam, D. (2012). Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1):132–151.

Diaz-Pace, J. A., Soria, Á., Rodríguez, G., and Campo, M. R. (2011). Taking ArchSync to the real world: An analysis of three case-studies. In *Simposio Argentino de Ingeniería de Software (ASSE), 40 JAIIO*, pages 25–36. SADIO.

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, Boston.

Ford, N., Parsons, R., and Kua, P. (2017). *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media, Sebastopol, CA, USA.

Ford, N., Parsons, R., Kua, P., and Sadalage, P. (2022). *Building Evolutionary Architectures: Automated Software Governance*. O'Reilly Media, Sebastopol, CA, 2 edition.

Gansner, E. R., Koutsofios, E., North, S. C., and Vo, K.-P. (1993). A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230.

Hamed, A. S. E., Elbakry, H. M., Riad, A. E., and Moawad, R. (2023). A proposed technical debt management approach applied on software projects in egypt. *Journal of Internet Services and Information Security (JISIS)*, 13(3):156–177.

Hevner, A. R., March, S. T., Park, J., and Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–105.

Jamil, J. (2026). Zero trust architecture for small/medium enterprises in hybrid cloud: A lightweight blueprint. *Scholars Journal of Engineering and Technology*, 14(1):48–56.

Jolak, R., Karlsson, S., and Dobslaw, F. (2025). An empirical investigation of the impact of architectural smells on software maintainability. *Journal of Systems and Software*, 225:112382.

Klein, J., Nord, R., and Shull, F. (2022). Why architecture conformance matters for software systems. Technical Report DM22-0924, Software Engineering Institute, Carnegie Mellon University.

Knodel, J. and Popescu, D. (2007). A comparison of static architecture compliance checking approaches. In *Sixth Working IEEE/IFIP Conference on Software Architecture (WICSA 2007)*, page 12. IEEE.

Lanza, M. and Marinescu, R. (2006). *Object-Oriented Metrics in Practice*. Springer, Berlin.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076.

Li, R., Liang, P., Soliman, M., and Avgeriou, P. (2022). Understanding software architecture erosion: A systematic mapping study. *Journal of Software: Evolution and Process*, 34(3):e2423.

Manglaras, O., Farkas, A., Woolford, T., Treude, C., and Wagner, M. (2026). Modaro: A modular approach to architecture reconstruction of distributed microservice codebases. *arXiv preprint arXiv:2602.08181*.

Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, Upper Saddle River, NJ, USA.

Modawski, A. and Wolniak, A. (2025). Lightweight AI governance (LAIG) framework for SMEs. In *Proceedings of the European Workshop on Trustworthy AI (TRUST-AI 2025), organized as part of the European Conference of Artificial Intelligence (ECAI 2025)*, Bologna, Italy. CEUR-WS.org.

Murphy, G. C., Notkin, D., and Sullivan, K. J. (2001). Software reflexion models: Bridging the gap between source and high-level models. *IEEE Transactions on Software Engineering*, 27(4):364–380.

Panneerselvam, S. (2023). Maintaining architectural integrity in .net projects with NetArchTest. Consultado en 2026.

Patrício, L. and Varela, L. (2025). Implementation of a low-cost digital transformation model for small- and medium-sized industrial enterprises in the context of industry 4.0. *Sci*, 7(4):187.

Paul, P. and Wang, R. (2019). Fitness function-driven development. *Thoughtworks*.

Peffers, K., Tuunanen, T., Rothenberger, M. A., and Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77.

Perry, D. E. and Wolf, A. L. (1992). Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52.

Pigazzini, I. (2021). *Evaluating and Detecting Architecture Erosion*. Phd thesis, Università degli Studi di Milano-Bicocca.

Rosik, J. (2007). *Maintaining Architecture Consistency*. PhD thesis, University of Limerick.

Rossini, S. (2022). Agile architecture part 7: Fitness functions & architectural guardrails. Consultado en 2026.

Runeson, P. and Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164.

San Martín, D., Angulo, G., and Vieira de Camargo, V. (2024). A MAPE-K-based method for architectural conformance checking in self-adaptive systems. Preprint, arXiv:2401.16382 (versión 3, 2025).

Sas, D. and Avgeriou, P. (2023). An architectural technical debt index based on machine learning and architectural smells. *IEEE Transactions on Software Engineering*, 49(8):4169–4195.

Sas, D., Avgeriou, P., and Fontana, F. A. (2019). Investigating instability architectural smells evolution: An exploratory case study. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 557–567. IEEE.

Sugiyama, K., Tagawa, S., and Toda, M. (1981). Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160.

TNG Technology Consulting (2026). Unit test your java architecture - archunit. Consultado en 2026.

Tyree, J. and Akerman, A. (2005). Architecture decisions: Demystifying architecture. *IEEE Software*, 22(2):19–27.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg.

Anexo A

Reproducción del experimento

Este anexo reúne la información necesaria para reproducir íntegramente el experimento descrito en el capítulo de evaluación. Se incluye porque una evaluación cuyos resultados no pueden rehacerse por un tercero aporta poca evidencia, con independencia de lo cuidadoso que haya sido su diseño.

A.1. Entorno y dependencias

El prototipo se implementó con nivel de lenguaje Java 17 y se verificó sobre JDK 17 y JDK 26. La única dependencia externa es *ArchUnit* en su versión 1.3.0, empleada exclusivamente para importar el *bytecode* y resolver las dependencias entre clases; el resto de la lógica —el algoritmo de detección de ciclos, el motor de conformidad y los exportadores— no depende de bibliotecas de terceros.

La biblioteca se distribuye junto al repositorio, de modo que la compilación y la ejecución no requieren Maven, repositorio de artefactos ni conexión a internet. Esta decisión responde al requisito de ligereza (RNF2) y, de paso, hace que el experimento siga siendo reproducible aunque los repositorios remotos cambien. Todas las operaciones se invocan mediante un único lanzador, `arquitecta.bat` en Windows o `./arquitecta.sh` en macOS y Linux, que compila el prototipo la primera vez y cada vez que su código cambia.

A.2. Estructura del repositorio

```
prototipo/
  arquitecta.bat / .sh        lanzador unico (aplicacion y tareas)
  src/main/java/              nucleo, servicio, interfaz y asistente
  src/test/java/              bateria de pruebas (272 comprobaciones)
  lib/                        ArchUnit 1.3.0 y slf4j-api
  caso-estudio/
    c0-base/                  escenario C0 (linea base)
    c1-capas/                 escenario C1 (ruptura de capas)
    c2-ciclo/                 escenario C2 (ciclo de dependencia)
    c3-concentracion/         escenario C3 (componente dios)
    c4-combinado/             escenario C4 (las tres derivas)
    rules.properties          catalogo de reglas del caso de estudio
  ejemplos/
    autoanalisis/             catalogo para el autoanalisis
```

```
    archunit/                configuraciones de la seccion 4.6
    sistemas/                catalogos y resultados de la seccion 4.7
    estudios/                evolucion, umbral y oraculo (4.8 a 4.10)
    asistente/               exploracion del asistente (anexo B)
    graficos/                grafos en SVG generados por la herramienta
    interfaz/                capturas de la interfaz grafica
  scripts/                   guiones que invoca el lanzador
decisiones/                  registro de decisiones (ADR, anexo C)
```

Cada escenario tiene la estructura de un proyecto Maven: su código está en `src/main/java` y la compilación deja sus clases en `target/classes`, de modo que la carpeta del escenario puede indicarse tal cual, desde la línea de comandos o desde la interfaz gráfica.

A.3. Pasos para reproducir el experimento

1. **Compilar.** Ejecutar `arquitecta compilar` (`./arquitecta.sh compilar` en macOS o Linux). El guion compila el núcleo y la batería de pruebas en `out/`, empaqueta la herramienta en `out/arquitecta.jar` y compila cada escenario en su `target/classes`. El paso es opcional: los siguientes compilan por su cuenta si hace falta.

2. **Verificar el núcleo.** Ejecutar `arquitecta pruebas`. La batería debe reportar 272 comprobaciones superadas y terminar con código de salida cero; cualquier fallo detiene la reproducción y señala el caso concreto que discrepa.

3. **Ejecutar el experimento.** Ejecutar `arquitecta experimento`. El guion imprime el catálogo de reglas, recorre los cinco escenarios bajo los dos modos de clasificación y deposita la evidencia en `resultados/`.

4. **Validación sobre sistemas reales (opcional).** Ejecutar primero `arquitecta aplicaciones`, que clona las tres aplicaciones en capas en el *commit* fijado y las compila —con Docker y la imagen oficial de Maven con JDK 17 si está disponible, o con el *Maven Wrapper* de cada proyecto—, y después `arquitecta sistemas`. Este segundo guion descarga de Maven Central los `.jar` de las bibliotecas, verifica su suma SHA-1, inyecta las derivas descritas en la sección 4.7 y escribe la tabla de resultados y la ubicación de cada deriva en `ejemplos/sistemas/`. Si alguna aplicación no está compilada, se omite con un aviso. Requiere conexión la primera vez.

5. **Estudios complementarios (opcional).** Ejecutar `arquitecta estudios`, que reproduce la evolución histórica (sección 4.8), la calibración del umbral (sección 4.9), el contraste con JDepend (sección 4.10) y la comparación con *ArchUnit* (sección 4.11); cada uno puede ejecutarse por separado con `arquitecta estudios evolucion`, `umbral`, `oraculo` o `archunit`. Descarga y verifica las treinta y cinco versiones históricas y JDepend la primera vez, y escribe la evidencia en `ejemplos/estudios/`.

6. **Contrastar.** Comparar el archivo `resultados/modo-cr/resumen.csv` con la tabla 4.2 y el archivo `resultados/modo-cr-severidad/resumen.csv` con la tabla 4.6.

Para reproducir el autoanálisis descrito en el capítulo de desarrollo basta con dirigir el mecanismo hacia su propio directorio de clases compiladas:

```
arquitecta analizar out\arquitecta-core ^
    ejemplos\autoanalisis\rules-arquitecta.properties
```

A.4. Figuras de grafos y sistema externo

Las figuras de grafos de este documento se regeneran con `arquitecta figuras`, que escribe las versiones TikZ en el directorio `figuras/` de la monografía y las SVG en `ejemplos/graficos/` del prototipo. Para obtener solo el dibujo de un sistema cualquiera basta con indicar las opciones `--svg` o `--tikz`:

```
arquitecta analizar lib\archunit-1.3.0.jar ^
    ejemplos\archunit\rules-archunit.properties --svg archunit.svg
```

El mismo análisis puede hacerse desde la interfaz gráfica, que se abre ejecutando el lanzador sin argumentos —o con doble clic sobre él— y permite elegir el `.jar` y el catálogo con un selector de archivos; su pantalla de inicio ofrece, además, los cinco escenarios del caso de estudio para analizarlos con un clic. El ejemplo reproduce además el análisis de la sección 4.6: el mecanismo admite directamente el archivo `.jar`, y el directorio `ejemplos/archunit/` contiene las configuraciones con y sin exenciones cuyos resultados recoge la tabla 4.8.

A.5. Evidencia generada

Cada ejecución con la opción `--salida` produce cuatro artefactos por escenario y una fila en el resumen comparativo:

- un **reporte legible** (`.txt`) con la Tasa de Conformidad, los estados deducidos por cada criterio, los hallazgos con su evidencia y la declaración de alcance del diagnóstico;
- la **evidencia estructurada** (`.json`), que incluye el grafo completo, los conteos por regla y la totalidad de las trazas de cada hallazgo, sin el recorte que el reporte legible aplica por brevedad;
- el **grafo de dependencias** dibujado en tres formatos con la misma disposición: una imagen `.svg` que se abre en cualquier navegador, una figura `.tikz` lista para incluirse en LaTeX y un archivo `.dot` para quien prefiera procesarlo con Graphviz;
- los **hallazgos tabulados** (`.csv`), una fila por hallazgo, para su tratamiento en una hoja de cálculo.

El archivo `resumen.csv` de cada modo agrega una fila por escenario con el número de paquetes, dependencias, verificaciones e incumplimientos, la Tasa de Conformidad, los tres estados calculados y la CR por regla. Es el artefacto del que proceden directamente las tablas del capítulo de evaluación.

A.6. Determinismo de los resultados

El mecanismo no emplea aleatoriedad, concurrencia ni marcas temporales en su cálculo, por lo que ante la misma entrada produce siempre el mismo diagnóstico. La construcción del grafo recorre las clases y sus dependencias en orden alfabético, y las estructuras que lo recorren conservan el orden de inserción, de modo que también el orden de los hallazgos y de su evidencia es estable entre ejecuciones y no depende de cómo se nombre la ruta analizada ni del orden en que la biblioteca de importación entregue las clases. Ello permite comparar dos corridas mediante una simple diferencia textual de sus archivos de evidencia, sin necesidad de normalización previa. Las capturas de la interfaz gráfica son la única excepción: dependen de las fuentes tipográficas del equipo y muestran el tiempo de análisis, por lo que se regeneran junto con las figuras pero no se comparan byte a byte.

Al ejecutar sobre JDK 26 aparecen dos avisos en la salida de error —la ausencia de un proveedor de *logging* para SLF4J y una advertencia sobre el uso de `sun.misc.Unsafe` emitida por una biblioteca embebida en ArchUnit— que no afectan al resultado y no se presentan con JDK 17.

Anexo B

Exploración del asistente de catálogo

Este anexo documenta una exploración del asistente de catálogo descrito en la sección 3.15 con modelos de lenguaje reales, uno remoto y uno que se ejecuta en el propio equipo. Se incluye como complemento y **no forma parte del experimento controlado**: el asistente es una ayuda para redactar el catálogo, ajena al mecanismo que el capítulo 4 evalúa, y la exploración no persigue ninguno de los objetivos específicos.

B.1. Pregunta y procedimiento

La pregunta que interesa no es si el catálogo propuesto coincide palabra por palabra con el de referencia, sino si es **seguro adoptarlo**. El riesgo propio de un asistente es que, al describir el código que ve, tome como prevista una deriva que ya existe y la legitime, de modo que la herramienta deje de detectarla. El caso de estudio permite medirlo, porque en los escenarios C1 a C4 las derivas inducidas están presentes en el código que el asistente recibe.

Para cada escenario, el asistente propuso un catálogo **sin ningún contexto**: solo con el perfil estructural del código, el escenario más desfavorable, en el que no cuenta con la intención del equipo. Cada propuesta se validó con el cargador de la herramienta y se previsualizó sobre el propio escenario, y se registró si los hallazgos seguían incluyendo las derivas inducidas de la matriz de validación (tabla 4.1). Se usó el proveedor DeepSeek con dos modelos, `deepseek-v4-pro` y `deepseek-flash`, con tres repeticiones por escenario y modelo, y con dos versiones de las instrucciones del asistente: las iniciales y una segunda que se redactó tras observar los resultados de la primera. En total se obtuvieron 60 propuestas, el 19 de octubre de 2026. Como el asistente usa por defecto un modelo local —para que ningún dato salga del equipo, en aplicación del principio de privacidad por diseño—, se repitió después la serie con las instrucciones de la segunda versión y `qwen2.5:14b` servido por Ollama, otras 15 propuestas. Esa serie se ejecutó solo en el procesador, porque con el contexto por defecto de Ollama (32 768 *tokens*) el modelo y su caché no cabían en los 12 GB de la GPU del equipo y la carga fallaba. Con un contexto de 8 192 *tokens* —suficiente, pues cada consulta del asistente ocupa unos 2 100— el modelo cabe por completo en la GPU, genera unas diez veces más rápido y cada propuesta tarda en torno a 11 s; una corrida de control con esa configuración reprodujo el mismo comportamiento que la serie en procesador.

B.2. Resultados

La tabla B.1 resume las cuatro series. Las 21 derivas inducidas de cada serie son las siete de la matriz de validación repetidas tres veces.

Tabla B.1: Exploración del asistente de catálogo: 5 escenarios × 3 repeticiones por serie, sin contexto. Las derivas que siguen detectándose se cuentan sobre los catálogos válidos.

	Instrucciones iniciales		**Segunda versión**		
	v4-pro	**flash**	**v4-pro**	**flash**	**qwen 14B local**
Catálogos válidos	15/15	15/15	15/15	15/15	11/15
Estratificación prevista bien declarada	15/15	15/15	15/15	15/15	10/11
Derivas inducidas que siguen detectándose	21/21	21/21	20/21	21/21	11/17
Corridas con hallazgos sobre la línea base (C0)	1/3	1/3	0/3	0/3	2/3
Corridas con algún hallazgo adicional	7/15	5/15	4/15	4/15	7/11
Umbral de concentración propuesto	2–6	2–5	2–6	2–5	3–6
Mediana de tiempo por propuesta	44 s	12 s	50 s	11 s	100 s*

* En procesador; en la GPU del equipo, con contexto de 8 192 *tokens*, unos 11 s.

De la tabla se desprenden seis observaciones.

La primera es la más importante: **ningún catálogo de los modelos remotos legitimó una deriva de capas ni un ciclo**. En las 60 propuestas de DeepSeek, el asistente declaró la estratificación prevista —presentación, negocio y persistencia en capas distintas, con las dependencias descendentes permitidas y las ascendentes prohibidas— y señaló como deriva probable las dependencias que la invertían, en lugar de permitirlas.

La segunda es que **el umbral de concentración es inestable**. El valor propuesto varió entre 2 y 6 para un mismo escenario, y en una corrida de la segunda serie de `deepseek-v4-pro` sobre C4 el modelo lo fijó en 6, por encima del acoplamiento del paquete `util` ($Ce = 4$), con lo que la deriva de concentración inducida dejó de detectarse. El mismo comportamiento había aparecido en una consulta exploratoria previa a la serie, y la segunda versión de las instrucciones, que pide expresamente no elevar el umbral para dejar de señalar un paquete, no lo evitó.

La tercera son los **hallazgos sobre la línea base**. En una de cada tres corridas de la primera serie, el modelo declaró `dominio` como capa sin permitir que la presentación lo usara, lo que produjo un hallazgo sobre un código sano. La segunda versión de las instrucciones, que pide tratar las entidades del dominio como un núcleo compartido, los eliminó; como efecto secundario, inclinó los umbrales hacia valores bajos que señalan la presentación de C1 ($Ce = 3$). Esos hallazgos adicionales no ocultan ninguna deriva, pero muestran que cada instrucción desplaza el comportamiento del modelo sin fijarlo.

La quinta procede del **modelo local**, que resultó claramente menos fiable. Cuatro de sus quince respuestas no produjeron un catálogo utilizable —dos sin el bloque estructurado y dos inválidas—, repitió en dos de tres corridas el error sobre la línea base que la segunda versión de las instrucciones había corregido en DeepSeek, y ocultó seis de las diecisiete derivas inducidas en sus catálogos válidos. Cinco de ellas fueron la concentración de `util`, y lo hizo por una vía que la revisión determinista

original no vigilaba: en lugar de elevar el umbral, declaró `util` como paquete de **rol compartido**, exento de FF2 y FF3. La sexta es más grave: en una corrida sobre C4 permitió la dependencia de la persistencia hacia el negocio, es decir, legitimó una deriva de capas. Privacidad y fiabilidad entran aquí en tensión: el modelo que garantiza que nada sale del equipo es también el que más revisión exige.

La sexta observación se sigue de las anteriores: **lo que no puede garantizarse con instrucciones debe comprobarse con reglas**. Por eso la herramienta añade a cada propuesta una revisión determinista: si el umbral deja sin señalar al paquete más acoplado y este destaca sobre el resto sin estar exento, lo advierte antes de que la persona adopte el catálogo. Aplicada a posteriori a las 60 propuestas, la revisión advirtió en la corrida que ocultaba la deriva y en otras 18 en que el paquete más acoplado superaba al siguiente en una sola dependencia. A la vista de la serie local, la revisión se amplió con una segunda regla: si una exención recae sobre el paquete que más dependencias salientes concentra y este destaca sobre el resto, lo advierte, porque una exención oculta la concentración igual que un umbral alto. Aplicada a posteriori a las 75 propuestas, la regla nueva no se activó en ninguna de las de DeepSeek y sí en las cinco corridas locales que ocultaban derivas, de modo que, entre las dos reglas, **toda propuesta que ocultaba una deriva habría llegado a la persona con una advertencia**. La revisión no cubre, en cambio, la legitimación de una deriva de capas: decidir si una dependencia entre capas es intención o desviación es precisamente el juicio que el catálogo debe expresar y que ninguna regla puede sustituir. Su función, por tanto, es llamar la atención sobre una decisión que conviene revisar, no emitir un veredicto, en coherencia con el principio de supervisión humana del capítulo 6.

B.3. Alcance de la exploración

Los resultados deben leerse con cautela. Proceden de un único caso, pequeño y sintético, cuyas derivas son evidentes en la estructura de paquetes; la segunda versión de las instrucciones se redactó a la vista de los resultados de la primera sobre ese mismo caso, de modo que su mejora no puede generalizarse; se emplearon solo dos familias de modelos, y el local en una única configuración; y los modelos no son deterministas, por lo que otra ejecución arrojará cifras distintas. Sobre todo, la exploración no mide lo que de verdad importa para la adopción: cuánto se parecen las propuestas a los catálogos que declararían los arquitectos de sistemas reales, que es la línea de trabajo futuro planteada en el capítulo de conclusiones.

La evidencia completa —el catálogo propuesto y la explicación del modelo en cada corrida, y la tabla de resultados— se conserva en `prototipo/ejemplos/asistente/`, y la exploración se repite con:

```
java -cp "out\arquitecta-core;out\arquitecta-test;lib\*" ^
     co.javeriana.msw.arquitecta.asistente.EvaluacionAsistente ^
     3 etiqueta [modelo] [proveedor]
```

donde el cuarto argumento, por ejemplo `ollama`, usa ese proveedor con su dirección por defecto sin tocar la configuración guardada. La revisión retroactiva se reproduce con la clase

`RevisionRetroactiva` del mismo paquete, que escribe `revision.csv` en cada carpeta.

Anexo C

Registro de decisiones

Las decisiones con consecuencias conceptuales —las que cambian el artefacto, el método de evaluación o la forma de interpretar los resultados— se registraron como *Architecture Decision Records* en la carpeta `decisiones/` del repositorio, siguiendo la práctica de documentar explícitamente las decisiones arquitectónicas y su razón de ser (Tyree and Akerman, 2005; AOE Technology Radar, 2026). Cada registro recoge el contexto que motivó la decisión, la decisión adoptada, las alternativas descartadas, sus consecuencias y su respaldo bibliográfico o empírico. Las decisiones anteriores a la creación de la carpeta se registraron de forma retrospectiva y se fechan con el cambio correspondiente del historial de versiones. Este anexo resume el registro; el texto completo de cada decisión está en el repositorio.

C.1. Desviaciones respecto del anteproyecto

El anteproyecto aprobado no se modificó. La tabla C.1 reúne las decisiones que se apartan de su letra y remite a la sección del documento que las justifica.

Tabla C.1: Decisiones que se apartan de la letra del anteproyecto.

Anteproyecto	**Trabajo realizado y motivo**	**Registro**
CR = reglas cumplidas / reglas totales	La clasificación usa la CR por verificación, que gradúa la extensión y registra la tendencia; la del anteproyecto, que satura, se calcula y reporta siempre como CR por regla (secciones 3.6 y 4.3.1).	ADR-0008
Clasificación por umbrales de la CR	Se añade la severidad del peor hallazgo porque la proporción diluye las violaciones aisladas; el modo original se conserva (sección 4.3).	ADR-0009, 0010
Análisis estático «del código fuente»	Se analiza el código compilado, derivado fiel del fuente, con puntos ciegos declarados y contrastados con un oráculo (secciones 3.4 y 4.10).	ADR-0003, 0019
Un único sistema en el estudio de caso	Se mantiene para OBJ4; se añaden validaciones complementarias en siete sistemas reales y treinta y cinco versiones publicadas (secciones 4.7 y 4.8).	ADR-0016 a 0018
Umbral de concentración fijado en el catálogo	Se sustituye el juicio por un procedimiento estadístico de percentiles, frente al que la clasificación es robusta (sección 4.9).	ADR-0011
Revisiones mensuales con el director	No se realizaron durante la ejecución; se declara (sección de metodología del capítulo 1).	ADR-0026
(No previsto) Asistente de catálogo	Aporte complementario fuera de OBJ1–OBJ4, con modelo local por defecto (anexo B).	ADR-0014, 0015

C.2. Índice del registro

Tabla C.2: Registro de decisiones del trabajo.

ADR	Decisión
0001	Registrar las decisiones de diseño e investigación como ADR.
0002	Prototipo en Java estándar, sin Spring Boot ni otro marco de trabajo.
0003	Extraer la arquitectura implementada del *bytecode*, con *ArchUnit*.
0004	Granularidad de paquete para el grafo y las reglas.
0005	Catálogo de tres *Fitness Functions* seleccionadas con los criterios C1–C5.
0006	La arquitectura prevista se declara en un catálogo de propiedades.
0007	Exenciones declaradas: `excluir` y `rol.compartido`.
0008	CR por verificación atómica, reportando también la CR por regla del anteproyecto.
0009	Clasificación combinada `cr+severidad`: el veredicto más restrictivo.
0010	Severidades: ciclos «crítica», capas y concentración «mayor».
0011	Umbral de concentración: percentil 90 del Ce de la línea base.
0012	FF3 mide solo la concentración eferente, no el *hub-like* completo.
0013	Interfaz Swing y lanzador único.
0014	Asistente de catálogo con modelos de lenguaje, como aporte complementario.
0015	El asistente usa por defecto un modelo local.
0016	Validación complementaria por inyección controlada de derivas en sistemas reales.
0017	Aplicaciones en capas como conjunto de validación independiente.
0018	Estudio de evolución histórica: deriva real, no inyectada.
0019	Oráculo externo: contrastar el grafo con JDepend.
0020	Contrastar el catálogo con las mismas reglas escritas en *ArchUnit*.
0021	Guardia de versión de *bytecode* (Java 23 como máximo).
0022	Licencia dual: documento CC BY-NC-ND 4.0 y código Apache 2.0.
0023	Compilar la monografía con Docker (TeX Live completo).
0024	Contraste explícito de la hipótesis y evaluación de utilidad con usuarios.
0025	Reemplazar la literatura gris por fuentes arbitradas en las afirmaciones centrales.
0026	Registro honesto del cronograma y de las revisiones con el director.